

A.D. 1308
unipg
UNIVERSITÀ DEGLI STUDI
DI PERUGIA

Università degli Studi di Perugia

Dipartimento di Matematica e Informatica

Corso di Laurea (Magistrale) in Matematica

Approccio Stocastico alla Crittoanalisi: Metodo Monte Carlo e Statistica Bayesiana

Relatore:
Prof. Alessio Troiani
Prof. Marco Timpanella

Laureando:
Claudia Puliga
Matricola: 386382

Anno Accademico 2025 – 2026

Indice

1 Introduzione **5**

2 Fondamenti di Statistica Computazionale, Ottimizzazione Stocastica e Crittografia **8**
2.1 Processi Markoviani a Stati Finiti 8
2.2 Generalizzazione ai Processi Non Omogenei 9
2.3 Proprietà Strutturali: Irriducibilità e Aperiodicità 9
2.4 Distribuzione Stazionaria ed Equilibrio 10
2.5 Reversibilità e Bilancio Dettagliato 10
2.6 Ottimizzazione Stocastica e Simulated Annealing 11
2.7 Modelli Linguistici e N-grammi 12
2.7.1 L'Assunzione di Markov e l'approssimazione a N-grammi . 13
2.7.2 Stima di Massima Verosimiglianza (MLE) 13
2.8 Valutazione dei Modelli Linguistici 14
2.8.1 Generalizzazione e Dipendenza dal Corpus di Addestramento 14
2.9 Tecniche di Smoothing 14
2.9.1 Add-k Smoothing 15
2.9.2 Interpolazione Lineare 15
2.10 Metodi Monte Carlo e Convergenza 16
2.10.1 L'Algoritmo di Metropolis-Hastings e Random Walk 16
2.11 Ricorrenza, Ergodicità e Teoremi di Convergenza 16
2.12 Teoria della Segretezza e Validità della Soluzione 18
2.12.1 La Validità di una Soluzione Crittografica 18
2.12.2 Equivocazione (Equivocation) e Incertezza 18
2.13 Entropia e Teoria dell'Informazione 18
2.13.1 Il Concetto di Entropia 18
2.13.2 Cross-Entropy e Teorema di Shannon-McMillan-Breiman . 19
2.14 Ridondanza e Distanza di Unicità 19
2.14.1 La Ridondanza del Linguaggio (D) 19
2.14.2 II Calcolo della Distanza di Unicità 20
2.15 Cifrari omofonici 21

3 Metodo MCMC per la crittanalisi di cifrari omofonici **23**
3.1 Costruzione della probabilità tramite N-grammi 23
3.2 Costruzione della funzione "Fitness" o "Score" 25
3.3 Distribuzione di Probabilità Stazionaria $\pi(K)$ 25

3.4 Campionamento stocastico con Metropolis-Hastings 26
3.5 Flusso Operativo dell'Algoritmo: Esecuzione Pratica 27
3.6 Applicazione del Teorema Ergodico e *Cooling Schedule* 28

4 Estensione della teoria al crittogramma Z_{340} 29
4.1 Sviluppo degli Strumenti Computazionali per l'Ottimizzazione . . 29
4.2 L'Anomalia Periodica e lo Studio dei Bigrammi 30
4.3 Trasposizione Geometrica e Decimazione 31
4.4 Il Limite della Molteplicità e il Metodo Crib 32
4.5 Risoluzione Finale, Correzione degli Errori e Validazione 33

5 Limiti di MCMC e Teoria dell'informazione 34
5.1 Il Teorema di Shannon-McMillan-Breiman 34
5.2 Il problema dell'Equivocazione e le Soluzioni Multiple 35
5.3 Stima dell'Entropia dell'Inglese e Limiti Teorici 35
5.4 Validità della Soluzione e Work Characteristic 36
5.5 Il Limite dell'Approssimazione Markoviana e il Collasso Entropico 37
5.6 Limiti di Convergenza MCMC e Mixing Time 38

6 Analisi Sperimentale e Ottimizzazione dei Parametri Stocastici 39
6.1 Setup dell'Esperimento e Architettura Software 39
6.2 Markov Chain Monte Carlo applicato ai trigrammi 41
6.2.1 Analisi Termodinamica: Dinamiche di Raffreddamento per i trigrammi 42
6.2.2 Confronto tra Add-k-Smoothing e Interpolazione Lineare . 45
6.2.3 Impatto dei Random Restarts 47
6.3 Collasso dello Spazio delle Chiavi nei trigrammi tramite Metodo Crib 47
6.4 Evoluzione ai 4-grammi 49
6.4.1 Analisi Termodinamica: Dinamiche di Raffreddamento per i quadrigrammi 51
6.4.2 Il Paradosso dello Smoothing: Add-k vs Interpolazione Lineare 52
6.4.3 Impatto dei Random Restarts 54
6.4.4 Crib 4-grammi 54

A Codice dell'Algoritmo MCMC e Simulated Annealing Base 59

B Analisi del Raffreddamento per trigrammi 64

C Confronto Modelli: Laplace vs Interpolazione Lineare 68

D Analisi dei Riavvii Multipli (Restarts) 72

E Decrittazione con Metodo Crib 77

F Codice Sorgente: Decrittazione a 4-grammi 82

G Analisi del Raffreddamento per quadrigrammi **87**

H Confronto tra Add-k-Smoothing Ibrido e Interpolazione Lineare per 4-Grammi **93**

I Impatto dei Riavvii Multipli (Restarts) a 4-Grammi **99**

J Metodo Crib per 4-grammi **104**

Elenco delle figure

1.1 The Zodiac Killer's 340-character 7

4.1 Il crittogramma Z_{340} originale pubblicato sui giornali nel novembre 1969. 29
4.2 Distribuzione dei bigrammi ripetuti nelle permutazioni casuali dello Z_{340} (a sinistra) rispetto ai testi artificiali (a destra). Sono evidenziate le linee di media e i valori per il periodo 1 e il periodo 19. 31
4.3 Rappresentazione della decimazione $(1, 2)$ applicata a una matrice bidimensionale. 31
4.4 Inserimento delle stringhe dedotte (*crib*) all'interno della griglia trasposta della prima sezione. 32

6.1 Convergenza termodinamica della Log-Fitness al variare del cooling schedule (5 test a confronto). 44
6.2 Confronto tra Metodo di Laplace e Interpolazione Lineare 46
6.3 Impatto dei restart sull'accuratezza dell'algoritmo 47
6.4 Sismografo termodinamico a 4-grammi con applicazione del Floor Penalty. Confronto tra raffreddamento continuo e a epoche con $T_{fin} = 0.5$. 52
6.5 Evoluzione dell'accuratezza a 4-grammi. Confronto tra Laplace Ibrido (con applicazione del Floor Penalty) e Interpolazione Lineare ($\lambda_4 = 0.4, \lambda_3 = 0.3, \lambda_2 = 0.2, \lambda_1 = 0.1$). 53
6.6 Impatto dei riavvii multipli (10 restarts) sulla decrittazione omofonica. Il grafico a doppio asse illustra l'evoluzione della Log-Fitness massima cumulativa (in blu) e il conseguente balzo dell'accuratezza associata (in verde) utilizzando il modello a 4-grammi con Laplace Ibrido. 54

Capitolo 1

Introduzione

L'evoluzione dei metodi stocastici applicati alla crittografia ha avuto origine nella fisica del secondo dopoguerra, quando scienziati come Stanislaw Ulam e John von Neumann, lavorando al Progetto Manhattan a Los Alamos, idearono il metodo Monte Carlo per simulare problemi intrattabili analiticamente come il trasporto dei neutroni all'interno dei materiali fissili come l'uranio-235 o il plutonio-239.

Nel 1953, Nicholas Metropolis formalizzò il primo algoritmo Markov Chain Monte Carlo (MCMC) per campionare le configurazioni di un sistema termodinamico, un impianto teorico che venne poi generalizzato nel 1970 da W.K. Hastings per renderlo applicabile a qualsiasi distribuzione di probabilità.

Questo motore stocastico trovò la sua definitiva affermazione nell'ottimizzazione nel 1983, quando S. Kirkpatrick e colleghi introdussero il Simulated Annealing, ispirandosi al processo metallurgico della ricottura dei metalli. L'obiettivo finale di quest'ultimo era trovare il minimo energetico della struttura atomica del metallo per renderlo più stabile, meno fragile e più facile da lavorare. Di grande rilievo furono gli utilizzi in ambito di ottimizzazione per la ricostruzione delle immagini con la stesura dell'articolo intitolato "Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images", dei fratelli Stuart e Donald Geman che affrontarono il problema della ricostruzione delle immagini pesantemente corrotte dal rumore; oppure, estendendo il suo utilizzo in ambito medico e farmacologico, l'approccio Monte Carlo si è rivelato essenziale per studiare il ripiegamento delle proteine (protein folding) esplorando le numerosissime configurazioni spaziali attraverso cui una catena di amminoacidi passa progressivamente verso il suo stato di minima energia termodinamica.

La potenza di questo impianto termodinamico ha trovato, nell'ambito crittografico, la sua applicazione più conosciuta nella risoluzione del cifrario Z340, spedito dal serial killer Zodiac nel novembre del 1969 e rimasto inviolato per cinquantuno anni.

Questa tesi si concentrerà sull'applicazione crittografica di questo metodo, esplorandone in particolare l'uso in ambito crittanalitico, ovvero come metodo per la "rottura" di un cifrario storico come quello del killer dello Zodiaco.

Fin dai primi giorni, l'unità crittografica dell'FBI tentò di attaccare il crittogramma utilizzando sistemi di elaborazione dati (EDP) per forzare parole chiave note nel testo, ma senza alcun successo. Nel 1989, con l'avvento dell'informatica

domestica, l'American Cryptogram Association pubblicò un programma in BASIC per facilitare la sperimentazione manuale, ma il vero salto computazionale avvenne nel 2006 con lo sviluppo di ZKDecrypto da parte di Brax Sisco, Michael Eaton e Wesley Hopper, un software capace di affrontare i cifrari omofonici esplorando lo spazio delle chiavi tramite un algoritmo euristico noto come tabu hill climbing.

Ispirandosi a questo lavoro, nel 2014 Jarl Van Eycke sviluppò AZdecrypt [2], un risolutore altamente ottimizzato basato sul Simulated Annealing, capace di elaborare le statistiche della lingua inglese analizzando gli N-grammi fino a otto caratteri consecutivi. La convergenza decisiva tra potenza stocastica e intuizione umana si concretizzò tra il 2019 e il 2020, quando fu notata un'anomalia statistica: un picco innaturale di bigrammi ripetuti se il testo veniva analizzato saltando diciannove caratteri alla volta, un evidente indizio di una trasposizione nascosta. Partendo da questa scoperta, il matematico australiano Sam Blake utilizzò il supercomputer Spartan dell'Università di Melbourne per generare e catalogare ben 655.088 possibili schemi di trasposizione geometrica del cifrario. Nel dicembre del 2020, David Oranchak sottopose ad elaborazione queste centinaia di migliaia di varianti al motore MCMC di AZdecrypt, il quale riuscì finalmente a far collassare la distribuzione di probabilità estraendo frammenti di testo in chiaro come "HOPE YOU ARE" e "TRYING TO CATCH ME". Questa decifrazione rivelò che Zodiac aveva diviso il messaggio in sezioni e lo aveva cifrato leggendo in diagonale, spostandosi di una riga in basso e due colonne a destra (salto del cavallo), commettendo peraltro vari errori di trascrizione che avevano depistato i computer per decenni. Grazie a questo sforzo congiunto, che ha fuso la teoria dell'informazione, la termodinamica di Metropolis-Hastings e il calcolo ad alte prestazioni, l'11 dicembre 2020 l'FBI ha potuto confermare ufficialmente la soluzione definitiva del crittogramma Z340 [2].Per arrivare a questo risultato fu indispensabile l'utilizzo e lo sviluppo di AZdecrypt.

Questa tesi si pone come obiettivo quello di descrivere tale algoritmo dal punto di vista teorico, analizzando la sua struttura basata sull'utilizzo delle catene di Markov, dell'importante ruolo del Simulated Annealing e del modello di accettazione di Metropolis. Inoltre si vogliono studiare i metodi d'ottimizzazione più performanti da un punto di vista pratico con l'implementazione di più algoritmi. Sfruttando un approccio sperimentale Trial and Error si vogliono mostrare tutti i risultati relativi all'utilizzo dei 3-grammi e di tutte le componenti che possono contribuire alla costruzione di un algoritmo ottimale, affrontando poi l'importanza della sua evoluzione ai 4-grammi.

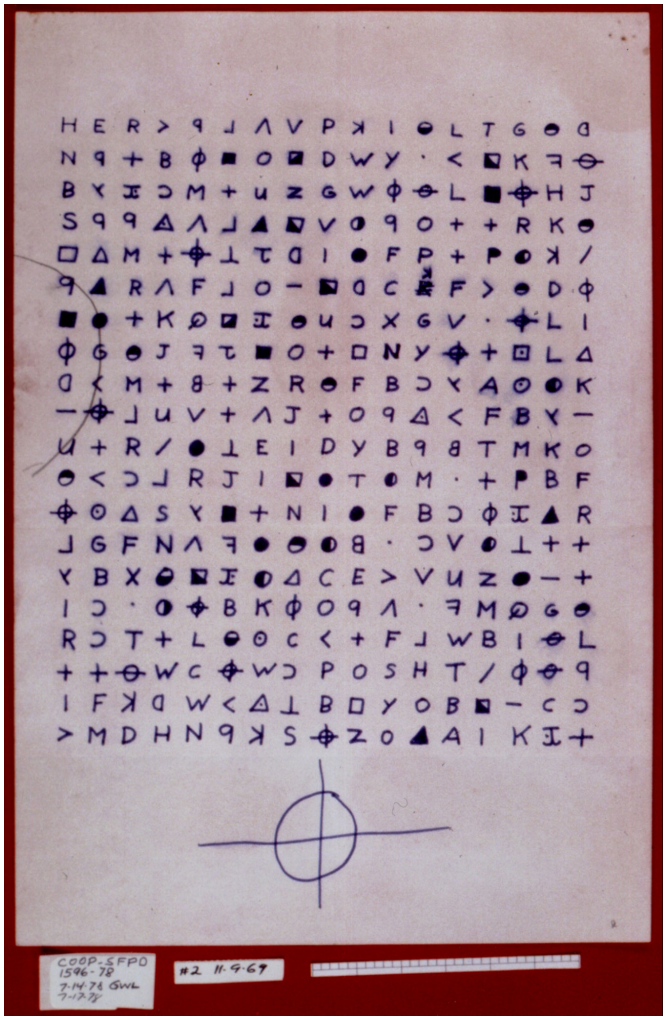


Figura 1.1: The Zodiac Killer's 340-character

Capitolo 2

Fondamenti di Statistica Computazionale, Ottimizzazione Stocastica e Crittografia

2.1 Processi Markoviani a Stati Finiti

La base della teoria che verrà analizzata in questa tesi riguarda le catene di Markov e la loro evoluzione nel tempo. Per riuscire a cogliere al meglio la complessità di questo metodo è necessario conoscere la sua struttura più profonda e la sua complessità. Iniziamo formalizzando rigorosamente la definizione di Catena di Markov e come questa si possa muovere nel tempo, passando da uno stato all'altro [3].

Definizione 2.1 (Catena di Markov Omogenea)
Sia P una matrice $k \times k$ con elementi $\{P_{i,j} : i, j = 1, \dots, k\}$. Un processo stocastico $(X_0, X_1, \dots)$ con spazio degli stati finito $S = \{s_1, \dots, s_k\}$ è detto una **catena di Markov (omogenea)** con matrice di transizione P se, per ogni n, per ogni $i, j \in \{1, \dots, k\}$ e per ogni sequenza di indici passati $i_0, \dots, i_{n-1} \in \{1, \dots, k\}$, si ha [3]:

$$\begin{aligned} &P(X_{n+1} = s_j \mid X_0 = s_{i_0}, \dots, X_n = s_i) \\ &= P(X_{n+1} = s_j \mid X_n = s_i) \\ &= P_{i,j}. \end{aligned}$$

Gli elementi $P_{i,j}$ sono detti *probabilità di transizione* e rappresentano la probabilità condizionale di transizione allo stato s_j al tempo $n+1$ dato che il sistema si trova nello stato s_i al tempo n. Ogni matrice di transizione soddisfa due proprietà [3]:

- $P_{i,j} \geq 0 \quad \forall i, j \in \{1, \dots, k\}$
- $\sum_{j=1}^{k} P_{i,j} = 1 \quad \forall i \in \{1, \dots, k\}$

Teorema 2.1 (Evoluzione Temporale della Distribuzione)
Per una catena di Markov $(X_0, X_1, \dots)$ con spazio degli stati $\{s_1, \dots, s_k\}$, distribuzione iniziale $\mu^{(0)}$ e matrice di transizione P, per ogni $n \geq 1$ la distribuzione $\mu^{(n)}$ al tempo n soddisfa:

$$\mu^{(n)} = \mu^{(0)} P^n$$

dove P^n denota la potenza n-esima della matrice P [3].

2.2 Generalizzazione ai Processi Non Omogenei

Negli algoritmi di ottimizzazione stocastica dinamica (come il Simulated Annealing) che utilizzeremo poi per arrivare alla decifrazione di Z340, ma in generale per decifrare cifrari classici, le regole di transizione devono cambiare al variare di un parametro di controllo (la temperatura). Per fare in modo che questo avvenga non possiamo più considerare la catena di Markov omogenea. Quindi consideriamola con un'altra proprietà [3].

Definizione 2.2 (Catena di Markov Disomogenea)
Sia $P^{(1)}, P^{(2)}, \dots$ una successione di matrici $k \times k$, ciascuna delle quali soddisfa le condizioni di non-negatività e di normalizzazione su riga. Un processo stocastico $(X_0, X_1, \dots)$ con spazio degli stati finito $S = \{s_1, \dots, s_k\}$ è detto una **catena di Markov disomogenea** con matrici di transizione $P^{(1)}, P^{(2)}, \dots$ se per ogni n si ha [3]:

$$P(X_{n+1} = s_j \mid X_0 = s_{i_0}, \dots, X_n = s_i) = P_{i,j}^{(n+1)}$$

2.3 Proprietà Strutturali: Irriducibilità e Aperiodicità

Le proprietà fondamentali per la costruzione delle catene di Markov sono l'irriducibilità, che dal punto di vista crittografico assicura che l'algoritmo abbia una probabilità strettamente positiva di raggiungere una chiave corretta partendo da una qualsiasi chiave generata casualmente, e l'aperiodicità, che assicura che l'algoritmo non si blocchi in un ciclo infinito deterministico [3].

Uno stato s_i comunica con s_j ($s_i \to s_j$) se esiste un intero $n \geq 0$ tale che $(P^n)_{i,j} > 0$. Se $s_i \to s_j$ e $s_j \to s_i$, i due stati intercomunicano ($s_i \leftrightarrow s_j$) [3].

Definizione 4.1 (Irriducibilità)
Una catena di Markov con spazio degli stati $S = \{s_1, \dots, s_k\}$ e matrice di transizione P è detta **irriducibile** se per ogni coppia di stati $s_i, s_j \in S$ si ha che $s_i \leftrightarrow s_j$ [3].

Per ciascuno stato $s_i \in S$ il periodo $d(s_i)$ è definito come [3]:

$$d(s_i) = \gcd\{n \geq 1 : (P^n)_{i,i} > 0\}$$

Definizione 4.2 (Aperiodicità)
Una catena di Markov è detta **aperiodica** se tutti i suoi stati sono aperiodici ($d(s_i) = 1$) [3].

2.4 Distribuzione Stazionaria ed Equilibrio

L'obiettivo dell'impianto MCMC è progettare una catena la cui evoluzione asintotica converga a una misura di probabilità target, ovvero una distribuzione stazionaria [3].

Definizione 5.1 (Distribuzione Stazionaria)
Sia $(X_0, X_1, \dots)$ una catena di Markov con spazio degli stati $\{s_1, \dots, s_k\}$ e matrice di transizione P. Un vettore riga $\pi = (\pi_1, \dots, \pi_k)$ è detto una **distribuzione stazionaria** per la catena se soddisfa [3]:

1. $\pi_i \geq 0 \quad \forall i \in \{1, \dots, k\} \quad$ e $\quad \sum_{i=1}^{k} \pi_i = 1$;
2. $\pi P = \pi$, ovvero $\sum_{i=1}^{k} \pi_i P_{i,j} = \pi_j \quad \forall j \in \{1, \dots, k\}$.

L'esistenza di una distribuzione stazionaria non garantisce, da sola, che la catena vi converga nel tempo. Tuttavia, richiedendo che la catena possieda le proprietà strutturali precedentemente definite (irriducibilità e aperiodicità), si giunge al risultato asintotico fondamentale della teoria dei processi markoviani [3].

Teorema 5.2 (Teorema di convergenza delle catene di Markov)
Sia $(X_0, X_1, \dots)$ una catena di Markov irriducibile e aperiodica con spazio degli stati $S = \{s_1, \dots, s_k\}$, matrice di transizione P e una distribuzione iniziale arbitraria $\mu^{(0)}$. Allora, per qualsiasi distribuzione π che sia stazionaria per la matrice di transizione P, si ha che [3]:

$$\mu^{(n)} \to \pi$$

Questo teorema assicura che, facendo evolvere il sistema per un tempo sufficientemente lungo, la sua distribuzione approssimerà l'equilibrio indipendentemente dallo stato da cui è partito. Un corollario diretto e cruciale di questo comportamento è l'unicità di tale limite [3].

Teorema 5.3 (Unicità della distribuzione stazionaria)
Qualsiasi catena di Markov irriducibile e aperiodica ha esattamente una distribuzione stazionaria [3].

2.5 Reversibilità e Bilancio Dettagliato

Nella progettazione degli algoritmi MCMC, il problema principale non è analizzare una catena data, ma costruire una matrice di transizione P che converga esattamente a una distribuzione stazionaria π desiderata. Per garantire ciò, si sfrutta una condizione di simmetria nota come reversibilità [3].

Definizione 6.1 (Distribuzione Reversibile e Catena Reversibile)
Sia $(X_0, X_1, \dots)$ una catena di Markov con spazio degli stati $S = \{s_1, \dots, s_k\}$ e matrice di transizione P. Una distribuzione di probabilità π su S è detta

reversibile per la catena (o per la matrice di transizione P) se per ogni $i, j \in \{1, \ldots, k\}$ si ha:

$$\pi_i P_{i,j} = \pi_j P_{j,i}$$

La catena di Markov è detta **reversibile** se esiste una distribuzione reversibile per essa [3].

L'equazione $\pi_i P_{i,j} = \pi_j P_{j,i}$ è nota in fisica come principio del *bilancio dettagliato*. L'importanza operativa di questa definizione per la costruzione di algoritmi di campionamento è sancita dal seguente teorema, che fornisce un metodo diretto per verificare la stazionarietà [3].

Teorema 6.1 (La reversibilità implica la stazionarietà)
Sia $(X_0, X_1, \ldots)$ una catena di Markov con spazio degli stati $S = \{s_1, \ldots, s_k\}$ e matrice di transizione P. Se π è una distribuzione reversibile per la catena, allora essa è anche una distribuzione stazionaria per la catena [3].

2.6 Ottimizzazione Stocastica e Simulated Annealing

I concetti di teoria delle catene di Markov e di distribuzioni stazionarie trovano la loro più potente applicazione nella risoluzione di problemi di ottimizzazione combinatoria in spazi degli stati immensi. Il problema generale si formula come segue: si ha un insieme finito $S = \{s_1, \ldots, s_k\}$ e una funzione $f : S \to \mathbb{R}$. L'obiettivo è trovare un elemento $s_i \in S$ che minimizzi (o massimizzi) $f(s_i)$ [3]. Quando la dimensione k dello spazio è enorme, il calcolo esaustivo è impraticabile.

L'idea alla base del metodo noto come *Simulated Annealing* è quella di costruire una catena di Markov la cui distribuzione stazionaria unica concentri la maggior parte della sua probabilità sugli stati $s \in S$ con un valore ottimale di $f(s)$ [3]. Il metodo generale per ottenere una tale distribuzione consiste nell'utilizzare la distribuzione di Gibbs-Boltzmann [3].

Definizione 13.1 (Distribuzione di Gibbs-Boltzmann)
La distribuzione di Gibbs-Boltzmann $\pi_{f,T}$ sull'insieme finito S, con funzione di energia $f : S \to \mathbb{R}$ e parametro di temperatura $T > 0$, è la distribuzione di probabilità su S che assegna a ciascun elemento $s \in S$ la probabilità [3]:

$$\pi_{f,T}(s) = \frac{1}{Z_{f,T}} \exp\left(-\frac{f(s)}{T}\right)$$

dove $Z_{f,T}$ è una costante di normalizzazione, definita come [3]:

$$Z_{f,T} = \sum_{s \in S} \exp\left(-\frac{f(s)}{T}\right)$$

che garantisce che $\sum_{s \in S} \pi_{f,T}(s) = 1$ [3].

Come specificato da Häggström [3], quando l'obiettivo è massimizzare (anziché minimizzare) la funzione f — come accade nella crittanalisi dove si cerca di

massimizzare la probabilità o la funzione di *fitness* — è utile sostituire la distribuzione di Boltzmann con una distribuzione modificata, in cui l'esponente assume la forma positiva $\frac{f(s)}{T}$ invece di $-\frac{f(s)}{T}$ [3].

Il seguente teorema dimostra matematicamente perché una bassa temperatura spinge il sistema verso lo stato ottimale, giustificando teoricamente l'algoritmo di *Simulated Annealing* [3].

Teorema 13.1
Sia S un insieme finito e sia $f : S \to \mathbb{R}$ arbitraria. Per $T > 0$, sia $\alpha(T)$ la probabilità che un elemento casuale Y, scelto secondo la distribuzione di Boltzmann $\pi_{f,T}$ su S, soddisfi [3]:

$$f(Y) = \min_{s \in S} f(s)$$

Allora:

$$\lim_{T \to 0} \alpha(T) = 1$$

Applicando il teorema alla distribuzione modificata per la massimizzazione, il limite per $T \to 0$ spingerà la probabilità di individuare il massimo globale a 1 [3].

Nella pratica del *Simulated Annealing*, si sceglie una successione decrescente di temperature $T_1 > T_2 > T_3 > \dots$ che tende a zero (da cui il termine *annealing*, o ricottura) e si fa evolvere la catena di Markov abbassando progressivamente T [3]. Se il raffreddamento avviene in modo sufficientemente lento, la probabilità di trovarsi in uno stato che ottimizza la funzione f tenderà a 1 al crescere del tempo n.

2.7 Modelli Linguistici e N-grammi

Prevedere un discorso futuro è difficile, ancora di più lo è prevedere le parole che qualcuno potrebbe usare. Per riuscire in questo intento però, si possono introdurre dei modelli linguistici, conosciuti come N-grammi o LM [4]. Sono modelli di Machine Learning che mirano a prevedere le parole successive. Più in particolare, un modello linguistico assegna una probabilità a ciascuna possibile parola (o lettera) successiva o, equivalentemente, fornisce una distribuzione di probabilità sulle possibili parole (o lettere) successive. I grandi modelli linguistici imparano un'enorme quantità di nozioni sulla lingua, inglese e non, unicamente venendo addestrati a prevedere le parole successive a partire da quelle precedenti. Altri utilizzi di questa conoscenza probabilistica risiedono nella correzione di errori grammaticali o di ortografia. Il modello linguistico più semplice è l'N-gramma, ovvero una sequenza di N caratteri, che può essere semplificato a un bigramma o un trigramma. L'obiettivo è stimare la probabilità di un carattere dati gli N-1 precedenti e di conseguenza assegnare probabilità a intere sequenze. L'utilizzo degli N-grammi dipende molto dal training set che ne garantisce la correttezza. Implementeremo la struttura degli N-grammi alle sequenze di lettere per formare un testo. [4]

Applicando la regola della catena alla probabilità (*chain rule*), la probabilità congiunta di una sequenza di n lettere $x_{1:n}$ è calcolabile scomponendola nel

prodotto delle probabilità condizionate [4]:

$$P(x_{1:n}) = P(x_1 \dots x_n) = \prod_{k=1}^{n} P(x_k \mid x_{1:k-1})$$

Dove la lettera k-esima dipende dalle $k-1$ precedenti. Tuttavia, calcolare l'esatta probabilità di una lettera condizionata all'intera sequenza precedente è computazionalmente inattuabile, a causa della natura creativa del linguaggio che genera contesti troppo elevati.

2.7.1 L'Assunzione di Markov e l'approssimazione a N-grammi

L'intuizione alla base dei modelli a N-grammi è approssimare la storia precedente considerando solo le ultime lettere. Utilizzando l'assunzione di Markov, il modello assume che la probabilità di un'unità dipenda solo da un passato limitato [4].

In un modello a bigrammi (dove $N = 2$), si approssima la probabilità condizionata considerando unicamente l'unità immediatamente precedente:

$$P(x_n \mid x_{1:n-1}) \approx P(x_n \mid x_{n-1})$$

Per il caso generale a N-grammi, si considerano le $N-1$ parole precedenti:

$$P(x_n \mid x_{1:n-1}) \approx P(x_n \mid x_{n-N+1:n-1})$$

Sostituendo l'approssimazione markoviana nella regola della catena, la probabilità di un'intera sequenza si stima come segue (esemplificato per i bigrammi) [4]:

$$P(x_{1:n}) \approx \prod_{k=1}^{n} P(x_k \mid x_{k-1})$$

2.7.2 Stima di Massima Verosimiglianza (MLE)

Per stimare empiricamente queste probabilità condizionate si ricorre alla Stima di Massima Verosimiglianza (MLE), estraendo i conteggi C da un corpus di addestramento e normalizzandoli in frequenze relative.

Per un modello a bigrammi, la probabilità MLE è il rapporto tra il conteggio del bigramma e la somma di tutti i bigrammi che condividono lo stesso prefisso (pari al conteggio dell'unigramma del prefisso) [4]:

$$P(x_n \mid x_{n-1}) = \frac{C(x_{n-1}x_n)}{\sum_x C(x_{n-1}x)} = \frac{C(x_{n-1}x_n)}{C(x_{n-1})}$$

Nel caso generale degli N-grammi, la formula MLE è:

$$P(x_n \mid x_{n-N+1:n-1}) = \frac{C(x_{n-N+1:n-1}x_n)}{C(x_{n-N+1:n-1})}$$

Osservazione:
Nella pratica crittanalitica e nei linguaggi di programmazione, moltiplicare probabilità stimate produce fenomeni di *underflow* numerico. Tutte le computazioni vengono quindi traslate nello spazio logaritmico, trasformando le moltiplicazioni in addizioni, così da non lavorare con numeri troppo piccoli [4]:

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

2.8 Valutazione dei Modelli Linguistici

2.8.1 Generalizzazione e Dipendenza dal Corpus di Addestramento

Un aspetto cruciale nella costruzione della funzione obiettivo riguarda la scelta dei dati su cui le probabilità vengono calcolate. Il modello a N-grammi, come molti modelli statistici, è strettamente dipendente dal corpus di addestramento. Un'implicazione diretta di questa caratteristica è che le probabilità spesso codificano fatti e strutture specifiche di un dato corpus di addestramento [4].

I modelli statistici risultano essere predittori sostanzialmente inutili se gli insiemi di addestramento e i dati da analizzare sono discordanti, come potrebbero esserlo le opere di Shakespeare confrontate con gli articoli del *Wall Street Journal*.

Di conseguenza, per affrontare questo problema nella costruzione dei modelli a N-grammi, è un passo fondamentale assicurarsi di utilizzare un corpus di addestramento che possieda un genere simile a quello del compito che si sta cercando di risolvere. Come stabilito dalla teoria, per costruire un modello linguistico destinato alla traduzione di documenti legali, è necessario un corpus di addestramento composto da documenti legali; allo stesso modo, per costruire un modello per un sistema di *question-answering*, è necessario un corpus di addestramento costituito da domande [4].

Questa regola teorica risulta di vitale importanza in ambito crittanalitico: se si ipotizza che un crittogramma nasconda un messaggio appartenente a un dominio semantico iper-specifico o a un micro-linguaggio tecnico, il modello a N-grammi deve essere obbligatoriamente addestrato su un corpus appartenente a quel medesimo dominio, pena l'inutilità statistica del valutatore.

2.9 Tecniche di Smoothing

L'utilizzo esclusivo della stima MLE presenta gravi limiti quando il modello incontra N-grammi mai osservati nel training set (problema delle sequenze a probabilità zero). Per prevenire il collasso delle metriche probabilistiche, si applicano algoritmi di *smoothing* o *discounting*, che arginano una parte della massa di probabilità degli eventi osservati per ridistribuirla agli eventi non visti [4].

2.9.1 Add-k Smoothing

Un metodo utile quanto semplice è sottrarre e ridistribuire meno massa di probabilità aggiungendo un valore frazionario k anziché 1. Questa tecnica prende il nome di Add-k smoothing e si definisce tramite la formula [4]:

$$P_{Add-k}(x_n \mid x_{n-1}) = \frac{C(x_{n-1}x_n) + k}{C(x_{n-1}) + kV}$$

2.9.2 Interpolazione Lineare

Un'alternativa valida per risolvere il problema della frequenza zero consiste nello sfruttare modelli probabilistici di ordine inferiore. Se si intende calcolare la probabilità di un trigramma $P(x_n \mid x_{n-2}x_{n-1})$ ma non si dispone di occorrenze per quella particolare sequenza, è possibile stimarne la probabilità basandosi sul bigramma $P(x_n \mid x_{n-1})$ o, in assenza anche di quest'ultimo, sull'unigramma $P(x_n)$. In altre parole, l'utilizzo di un contesto più ridotto aiuta il modello a generalizzare meglio in situazioni in cui l'apprendimento sui contesti ampi risulta limitato.

Il metodo più diffuso per combinare questa gerarchia di N-grammi prende il nome di *Interpolazione Lineare*. Essa consiste nel calcolare una nuova probabilità mettendo insieme (attraverso una somma pesata) le probabilità di trigrammi, bigrammi e unigrammi [4]. La formula per l'interpolazione lineare semplice stima la probabilità del trigramma come segue:

$$\hat{P}(x_n \mid x_{n-2}x_{n-1}) = \lambda_1 P(x_n) + \lambda_2 P(x_n \mid x_{n-1}) + \lambda_3 P(x_n \mid x_{n-2}x_{n-1})$$

Affinché l'equazione rappresenti una media pesata e una corretta distribuzione di probabilità, la somma dei coefficienti deve essere rigorosamente pari a 1 ($\sum_i \lambda_i = 1$).

In una versione avanzata dell'interpolazione lineare, i pesi λ vengono condizionati dal contesto. Se si dispone di conteggi molto precisi per un determinato bigramma, si può presumere che anche le stime dei trigrammi basati su di esso siano più attendibili. È quindi possibile assegnare a tali trigrammi un peso λ maggiore. In questo approccio, ogni parametro λ diventa una funzione del contesto precedente (indicato come x_{n-2}^{n-1}):

$$\hat{P}(x_n \mid x_{n-2}x_{n-1}) =$$
$$\lambda_1(x_{n-2}^{n-1})P(x_n) + \lambda_2(x_{n-2}^{n-1})P(x_n \mid x_{n-1}) + \lambda_3(x_{n-2}^{n-1})P(x_n \mid x_{n-2}x_{n-1})$$

I valori ottimali dei parametri λ vengono acquisiti sfruttando un *held-out corpus*, ovvero un set di dati di validazione tenuto separato dai dati di addestramento primario. Fissate le probabilità degli N-grammi, si cercano i valori di λ che massimizzano la verosimiglianza (*likelihood*) del corpus di validazione. Tale ottimizzazione viene tipicamente eseguita tramite l'algoritmo EM (*Expectation-Maximization*), un processo iterativo capace di convergere verso valori localmente ottimali.

2.10 Metodi Monte Carlo e Convergenza

2.10.1 L'Algoritmo di Metropolis-Hastings e Random Walk

Per minimizzare la *cross-entropy* esplorando l'immenso spazio delle chiavi crittografiche, si ricorre a metodi stocastici. L'algoritmo di Metropolis-Hastings parte da una densità obiettivo (target) f, definita dal nostro *score*, e utilizza una densità condizionale $q(y \mid x)$ (distribuzione strumentale o di proposta) [6]. L'algoritmo produce una catena di Markov $(X^{(t)})$ attraverso la seguente transizione [6]:

Algoritmo A.24 (Metropolis-Hastings) [6]
Dato $x^{(t)}$:

1. Generare $Y_t \sim q(y \mid x^{(t)})$.
2. Assumere
$$X^{(t+1)} = \begin{cases} Y_t & \text{con probabilità } \rho(x^{(t)}, Y_t) \\ x^{(t)} & \text{con probabilità } 1 - \rho(x^{(t)}, Y_t) \end{cases}$$
dove
$$\rho(x, y) = \min \left\{ \frac{f(y)}{f(x)} \frac{q(x \mid y)}{q(y \mid x)}, 1 \right\}$$

Un approccio valido per la costruzione pratica di tale algoritmo consiste nell'esplorazione locale dell'intorno del valore corrente della catena. Poiché la distribuzione candidata q è autorizzata a dipendere dallo stato corrente $X^{(t)}$, si può simulare il candidato Y_t tramite una perturbazione aleatoria ϵ_t indipendente da $X^{(t)}$ [6]:

$$Y_t = X^{(t)} + \epsilon_t$$

La catena di Markov associata a questa specifica formulazione prende il nome di *random walk* (passeggiata aleatoria) sullo spazio degli stati $\mathcal{E}$ [6]. Nel contesto crittografico, la perturbazione ϵ_t consiste nello scambio casuale di elementi della chiave di decifrazione.

2.11 Ricorrenza, Ergodicità e Teoremi di Convergenza

Affinché il random walk possa validamente approssimare la soluzione ottima, è fondamentale stabilire il comportamento limite della catena generata dall'algoritmo, dimostrando che essa converga indipendentemente dalle condizioni iniziali [6].

Trattando l'ottimizzazione di chiavi crittografiche, lo spazio degli stati $S = \{s_1, \ldots, s_k\}$ è finito. In questo contesto, le condizioni per garantire la convergenza si semplificano notevolmente rispetto alla teoria generale per gli spazi continui e infiniti. A questo riguardo una prima importante nozione è quella di ricorrenza.

Definizione 2.3 (Catena ricorrente e positivamente ricorrente)
Uno stato è detto ricorrente se, partendo da esso, la catena vi farà ritorno con probabilità 1. Se, inoltre, il tempo atteso di ritorno in quello stato è finito, lo stato è detto positivamente ricorrente. Una catena in cui tutti gli stati sono positivamente ricorrenti prende il nome di catena positivamente ricorrente [3].

Per le catene di Markov su spazi degli stati finiti il problema della ricorrenza si semplifica. In particolare, è dimostrato formalmente (Lemma 5.1 [3]) che per qualsiasi catena di Markov finita, irriducibile e aperiodica, se la catena parte da uno stato s_i, la probabilità di raggiungere (o tornare a) un qualsiasi stato s_j è sempre pari a 1, ovvero $P(T_{i,j} < \infty) = 1$. Inoltre, il tempo atteso di primo passaggio (o di ritorno) $\tau_{i,j} = E[T_{i,j}]$ è sempre strettamente finito, ovvero $E[T_{i,j}] < \infty$ [3]. Pertanto, tutti gli stati di una tale catena soddisfano le condizioni di ricorrenza positiva.

Unendo a questa proprietà il concetto di aperiodicità (che garantisce l'assenza di cicli deterministici bloccanti), si ottengono le condizioni sufficienti per l'esistenza e l'unicità di una misura stazionaria. Tale concetto è formalizzato dal teorema fondamentale di convergenza per le catene di Markov:

Teorema 5.2 (Convergenza delle catene di Markov)
Sia (X_n) una catena di Markov irriducibile e aperiodica con spazio degli stati finito S e matrice di transizione P, avente distribuzione stazionaria unica π. Allora, per qualsiasi distribuzione iniziale, la distribuzione della catena al tempo n converge a π per $n \to \infty$ [3]. In termini di probabilità di transizione [3]:

$$\lim_{n\to\infty} P^n(x, y) = \pi(y)$$

Date le osservazioni $X_1, \ldots, X_n$ generate dalla catena di Markov, si analizza il comportamento limite delle somme parziali al crescere di n. Il risultato principale che garantisce la corretta stima delle funzioni obiettivo (come la funzione fitness) è la Legge dei Grandi Numeri per le catene di Markov, conosciuto come Teorema Ergodico [6].

Teorema 6.63 (Teorema Ergodico per spazi finiti)
Sia (X_n) una catena di Markov finita, irriducibile e aperiodica, avente distribuzione stazionaria unica π. Allora, per ogni funzione limitata $h : S \to \mathbb{R}$, le "medie empiriche" calcolate sui campioni della catena convergono con probabilità 1 alle "medie teoriche" calcolate rispetto alla distribuzione stazionaria [6]:

$$\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} h(X_i) = \sum_{x\in S} h(x)\pi(x)$$

Questo teorema è fondamentale per l'algoritmo MCMC: esso garantisce che, esplorando lo spazio delle chiavi per un numero sufficiente di iterazioni, il comportamento medio campionato dal nostro algoritmo stocastico approssimerà accuratamente il comportamento della distribuzione desiderata.

2.12 Teoria della Segretezza e Validità della Soluzione

2.12.1 La Validità di una Soluzione Crittografica

L'esplorazione dello spazio delle chiavi tramite l'algoritmo MCMC garantisce matematicamente la convergenza verso una chiave ottimale, ma solleva un valido interrogativo crittografico: come stabilire se la soluzione trovata sia l'unica corretta? Come esplicitato da Shannon [7], se un sistema e una chiave proposti risolvono un crittogramma per una lunghezza di materiale considerevolmente maggiore della *distanza di unicità*, la soluzione è affidabile (*trustworthy*). Se il materiale è dello stesso ordine di grandezza o più corto, la soluzione è altamente sospetta [7].

2.12.2 Equivocazione (Equivocation) e Incertezza

Nel suo lavoro fondativo, Claude Shannon modella la crittanalisi come l'elaborazione di un segnale perturbato. In un sistema di segretezza, l'analista intercetta un crittogramma E di lunghezza N e deve calcolare le probabilità a posteriori dei vari messaggi M e delle chiavi K [7].

Per misurare l'incertezza residua sul messaggio o sulla chiave, Shannon definisce l'*equivocazione* della chiave $H_E(K)$ e del messaggio $H_E(M)$ come [7]:

$$H_E(K) = \sum_{E,K} P(E,K) \log P_E(K)$$

$$H_E(M) = \sum_{E,M} P(E,M) \log P_E(M)$$

All'aumentare della lunghezza N del materiale intercettato, l'equivocazione della chiave è una funzione non crescente (non aumenta all'aumentare delle informazioni disponibili) [7].

2.13 Entropia e Teoria dell'Informazione

2.13.1 Il Concetto di Entropia

La costruzione di una funzione obiettivo basata sulle probabilità richiama concetti informatico-teorici. Il concetto centrale è l'entropia, che fornisce una misura della quantità di informazione (o incertezza) e un limite inferiore teorico alla compressione dei dati [7].

Dato un insieme χ su cui varia una variabile aleatoria X (es. i caratteri o le parole di un vocabolario) e una funzione di probabilità $p(x)$, l'entropia $H(X)$ è definita come [7]:

$$H(X) = -\sum_{x \in \chi} p(x) \log p(x)$$

Estendendo il concetto alle sequenze testuali, si definisce il tasso di entropia (*entropy rate*) per una sequenza di lunghezza n appartenente a un linguaggio L

[7]:

$$\frac{1}{n}H(x_{1:n}) = -\frac{1}{n}\sum_{x_{1:n}\in L} p(x_{1:n})\log p(x_{1:n})$$

Per misurare la vera entropia di una lingua, si considerano sequenze di lunghezza infinita:

$$H(L) = \lim_{n\to\infty} -\frac{1}{n}\sum_{X\in L} p(x_{1:n})\log p(x_{1:n})$$

2.13.2 Cross-Entropy e Teorema di Shannon-McMillan-Breiman

Quando l'effettiva distribuzione di probabilità p che ha generato i dati non è nota, ci si affida a un modello m (nel nostro caso, il modello a N-grammi) che funge da approssimazione di p. L'entropia incrociata (*cross-entropy*) $H(p, m)$ del modello m sulla distribuzione p è [7]:

$$H(p, m) = \lim_{n\to\infty} -\frac{1}{n}\sum_{X\in L} p(x_1, \ldots, x_n)\log m(x_1, \ldots, x_n)$$

Calcolare questa somma su tutte le sequenze possibili di una lingua è inattuabile. Tuttavia, il teorema di Shannon-McMillan-Breiman stabilisce che, se il processo stocastico che genera la lingua è stazionario ed ergodico, l'entropia incrociata può essere approssimata valutando una singola sequenza sufficientemente lunga:

$$H(p, m) = \lim_{n\to\infty} -\frac{1}{n}\log m(x_1, x_2, \ldots, x_n)$$

Questa equazione è fondamentale per la crittanalisi stocastica: essa dimostra che la media delle log-probabilità calcolate dal nostro modello su un singolo crittogramma decifrato fornisce una stima matematicamente rigorosa della *cross-entropy* [7]. Definendo l'approssimazione della *cross-entropy* su una sequenza fissa X di lunghezza N come:

$$H(X) = -\frac{1}{N}\log P(x_1x_2\ldots x_N)$$

risulta evidente che il nostro *fitness score* logaritmico $S(K) = \log P(x_1x_2\ldots x_N)$ non è altro che il negativo della *cross-entropy* moltiplicato per la lunghezza del testo. Massimizzare il *fitness score* equivale quindi a minimizzare la *cross-entropy* del testo decifrato, spingendo il modello a trovare la decifrazione che più si avvicina alla vera distribuzione della lingua naturale.

2.14 Ridondanza e Distanza di Unicità

2.14.1 La Ridondanza del Linguaggio (D)

Nella teoria della segretezza, un parametro cruciale associato ad una lingua è la ridondanza D. Questo parametro misura la quantità di "vincolo statistico"

imposto dalla lingua stessa o, in altri termini, quanto un testo può essere ridotto in lunghezza senza perdere alcuna informazione [7].

Matematicamente, Shannon definisce la ridondanza totale D_N per un messaggio di N lettere come la differenza tra la massima incertezza teorica e l'entropia effettiva del messaggio [7]:

$$D_N = \log G - H(M)$$

dove G è il numero totale di messaggi possibili di lunghezza N, e $H(M)$ è l'incertezza (entropia) effettiva nella scelta di uno di questi messaggi.

La ridondanza per singola lettera D si ricava dividendo la ridondanza totale per il numero di lettere [7]:

$$D = \frac{D_N}{N}$$

2.14.2 II Calcolo della Distanza di Unicità

La ridondanza è ciò che permette di isolare una soluzione [7]. Shannon dimostra che, in un sistema chiuso, il decremento dell'equivocazione della chiave dopo aver intercettato N lettere non è mai superiore alla ridondanza D_N di quelle N lettere:

$$H(K) - H_E(K) \leq D_N$$

Analizzando un "cifrario casuale" (*random cipher*), Shannon individua il punto in cui l'equivocazione della chiave $H_E(K)$ tende a zero, ovvero il punto in cui la soluzione del crittogramma diventa unica [7]. Questo valore critico di N, definito "distanza di unicità", è approssimato dalla seguente equazione:

$$N \approx \frac{H(K)}{D}$$

In questa relazione:

- $H(K)$ è un numero che misura la "dimensione" dello spazio delle chiavi. Se tutte le chiavi sono a priori equiprobabili, $H(K)$ è semplicemente il logaritmo del numero di chiavi possibili.
- D è la ridondanza per lettera della lingua originaria (misurata nella stessa base logaritmica).

È possibile costruire sistemi di segretezza con chiave finita per determinati "linguaggi" in cui l'equivocazione non si avvicina a zero per $N \to \infty$ [7]. In questi sistemi ideali, a prescindere dalla quantità di dati intercettati, il nemico non ottiene una soluzione unica al cifrario, ma è lasciato con molteplici alternative tutte dotate di probabilità ragionevoli. In ambito crittografico vengono quindi generate una serie di chiavi "spurie", ovvero che portano ad una decifrazione sensata ma non corretta. Questo limite è invalicabile.

Nei prossimi capitoli applicheremo la teoria studiata fino ad ora per costruire l'impalcatura teorica di un codice per la crittanalisi di cifrari classici. In particolare, si analizzerà il crittogramma Z_{340}. E' quindi necessario capire la teoria crittografica che caratterizza l'applicazione pratica.

2.15 Cifrari omofonici

Definizione:

Un sistema crittografico è una quintupla $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$ in cui sono soddisfatte le seguenti condizioni [9]:

1. $\mathcal{P}$ è un insieme finito di possibili testi in chiaro;
2. $\mathcal{C}$ è un insieme finito di possibili testi cifrati;
3. $\mathcal{K}$, lo spazio delle chiavi, è un insieme finito di possibili chiavi;
4. Per ogni $K \in \mathcal{K}$, esiste una regola di crittografia $e_K \in \mathcal{E}$ e una corrispondente regola di decrittazione $d_K \in \mathcal{D}$. Ogni $e_K : \mathcal{P} \to \mathcal{C}$ e $d_K : \mathcal{C} \to \mathcal{P}$ sono funzioni tali che $d_K(e_K(x)) = x$ per ogni elemento del testo in chiaro $x \in \mathcal{P}$.

Questa definizione formale, introdotta da Douglas Stinson [9], costituisce il modello standard per i cifrari deterministici. I cifrari omofonici rappresentano un'estensione probabilistica di questo modello classico, l'algoritmo di cifratura implica una scelta aleatoria dell'omofono (e dunque la regola e_K non agisce più come una singola funzione deterministica), essi superano questa formulazione di base, evolvendosi ed introducendo un grado di libertà necessario per contrastare l'analisi delle frequenze.

Uno dei modelli più elementari di crittosistema è definito utilizzando la cifratura a sostituzione semplice (o monoalfabetica).

Definizione (Cifrario a Sostituzione semplice):

Siano $\mathcal{P} = \mathcal{C} = \mathbb{Z}_{26}$. $\mathcal{K}$ consiste in tutte le possibili permutazioni dei 26 simboli $0, 1, \ldots, 25$. Per ogni permutazione $\pi \in \mathcal{K}$, definiamo $e_\pi(x) = \pi(x)$, e definiamo $d_\pi(y) = \pi^{-1}(y)$, dove π^{-1} è la permutazione inversa di π [9].

Una naturale evoluzione del crittosistema a sostituzione semplice, ideata per contrastare l'analisi delle frequenze, è la cifratura omofonica, in cui l'alfabeto del testo cifrato è strettamente più grande dell'alfabeto del testo in chiaro. In questo sistema, a ogni singolo elemento dell'alfabeto in chiaro viene associato un insieme di due o più simboli cifrati, denominati omofoni.

Condizione matematica necessaria affinché la decifrazione sia ben definita è che gli insiemi di omofoni associati a lettere diverse siano rigorosamente disgiunti, formando una partizione dell'alfabeto cifrato $\mathcal{C}$. In assenza di tale condizione, uno stesso simbolo cifrato potrebbe corrispondere a più lettere in chiaro, rendendo l'operazione di decifratura confusionaria ed irrisolvibile.

A differenza della sostituzione monoalfabetica, la cifratura omofonica introduce una componente probabilistica: il mittente sceglie casualmente uno degli omofoni disponibili per cifrare un dato carattere. L'obiettivo crittografico di tale mappatura "uno a molti" è l'appiattimento delle frequenze relative dei simboli nel testo cifrato, al fine di mitigare le vulnerabilità del crittosistema contro gli attacchi basati sull'analisi statistica delle frequenze.

Tuttavia questo appiattimento agisce unicamente sulle frequenze delle singole lettere (unigrammi). Il cifrario omofonico non è in grado di mascherare le frequenze e le correlazioni di ordine superiore, come la struttura tipica di bigrammi, trigrammi, ecc. Questa limitazione teorica è fondamentale, poiché costituisce esattamente la debolezza strutturale che verrà sfruttata dall'attacco crittanalitico a N-grammi discusso nei Capitolo 3 e 4.

Un esempio di sostituzione omofonica può essere questa:

$$e \to \{i, \clubsuit, \diamondsuit, \heartsuit, \spadesuit\}$$

La differenza tra questi due crittosistemi risiede nello spazio delle chiavi. Nel modello del cifrario a sostituzione semplice, gli spazi del testo in chiaro e del testo cifrato coincidono, per cui si definisce $\mathcal{P} = \mathcal{C} = \mathbb{Z}_{26}$. La trasformazione crittografica si basa su una corrispondenza biunivoca tra gli elementi dei due insiemi. Quindi lo spazio delle chiavi può essere calcolato come [9]:

$$|\mathcal{K}| = 26!$$

Questo valore è superiore a 4.0×10^{26}, un numero estremamente grande che rende computazionalmente irrealizzabile un attacco di forza bruta, sebbene il sistema rimanga altamente vulnerabile all'analisi delle frequenze.

Diversamente il cifrario omofonico introduce una disuguaglianza tra le cardinalità degli alfabeti, ponendo la condizione $|\mathcal{C}| > |\mathcal{P}|$. In questo scenario, l'operazione di cifratura non è più modellabile attraverso una singola permutazione, bensì tramite una mappatura "uno a molti" in cui l'algoritmo sceglie in modo probabilistico un elemento da un sottoinsieme di simboli cifrati.

Siano $|\mathcal{P}| = n$ la cardinalità dell'alfabeto in chiaro (es. $n = 26$, lettere della lingua inglese) e $|\mathcal{C}| = m$ la cardinalità dell'alfabeto cifrato. Supponendo che la chiave K definisca l'assegnazione di c_i omofoni all'elemento i-esimo del testo in chiaro (con $1 \leq i \leq n$), deve valere la condizione:

$$\sum_{i=1}^{n} c_i = m$$

La dimensione dello spazio delle chiavi $|\mathcal{K}|$ non è più calcolabile tramite un semplice fattoriale, ma corrisponde al numero di modi in cui è possibile partizionare m elementi distinti in n sottoinsiemi di dimensioni prefissate $c_1, c_2, \ldots, c_n$. Questa quantità è descritta dalla formula del coefficiente multinomiale:

$$|\mathcal{K}_{omofonico}| = \frac{m!}{c_1! \cdot c_2! \cdot \cdots \cdot c_n!}$$

Se si rimuove il vincolo sulle frequenze prefissate, permettendo ad ogni simbolo del testo cifrato di assumere in modo indipendente il valore di una qualsiasi delle n lettere, l'approccio combinatorio cambia. Questa possibilità rappresenta un limite superiore teorico (upper bound) per lo spazio delle chiavi. Vedremo nei capitoli successivi che applicando questa teoria al crittogramma Z_{340} lo spazio esplorabile ammonta a: 26^{63}.

Capitolo 3

Metodo MCMC per la crittanalisi di cifrari omofonici

In questo capitolo verrà analizzata la costruzione teorica e computazionale di un algoritmo basato sui metodi Markov Chain Monte Carlo (MCMC) per la crittanalisi di cifrari omofonici. Da ogni crittogramma è teoricamente possibile estrarre il testo in chiaro conoscendo la chiave di decifratura corretta; tuttavia, nell'analisi di crittogrammi storici, tale chiave è l'incognita del problema.

Si definiscano innanzitutto gli insiemi fondamentali che strutturano il crittosistema:

- $C = \{c_1, \ldots, c_m\}$: l'alfabeto del testo cifrato, composto da m simboli. Nel caso specifico dei cifrari omofonici , si ha tipicamente $m > 26$.
- $P = \{x_1, \ldots, x_n\}$: l'alfabeto della lingua del testo in chiaro (nel caso dell'inglese standard , $n = 26$).

Dato un crittogramma di lunghezza L, il testo cifrato è rappresentato come una sequenza di simboli $c = (c_1, \ldots, c_L)$ con $c_i \in C$. La chiave di decifrazione può essere modellata come una funzione suriettiva:

$$K : C \longrightarrow P \quad \text{con} \quad c_i \longmapsto K(c_i) = x_i \quad \text{con} \quad x_i \in P$$

L'intero messaggio decifrato diviene quindi $K(c) = x = (x_1 \ldots x_L)$.

L'obbiettivo crittanalitico è individuare la chiave ottimale $K^* \in \mathcal{K}$ che massimizzi la probabilità che x corrisponda al testo in chiaro originale. Per procedere, è fondamentale stabilire le assunzioni sul dominio linguistico del messaggio, fattore che determina l'addestramento del modello a N-grammi (come teorizzato nella Sezione 2.8.1).

3.1 Costruzione della probabilità tramite N-grammi

Il testo in chiaro P viene modellato come un processo stocastico di Markov di ordine $N-1$. Applicando la regola della catena , la probabilità congiunta dell'intera sequenza decifrata si scompone nel prodotto delle probabilità condizionate:

$$\mathbb{P}(x_{1:L}) = \mathbb{P}(x_1)\mathbb{P}(x_2|x_1)\dots\mathbb{P}(x_L|x_{1:L-1}) = \prod_{i=1}^{L}\mathbb{P}(x_i|x_{1:i-1})$$

Come analizzato nella Sezione 2.7.1, calcolare le esatte probabilità condizionate sull'intero passato è computazionalmente inattuabile. Interviene quindi l'assunzione markoviana, che limita la dipendenza di ogni carattere a una finestra locale. Nel caso dei bigrammi ($N = 2$) o trigrammi ($N = 3$):

$$\mathbb{P}(x_i|x_{1:i-1}) \approx \mathbb{P}(x_i|x_{i-1}) \quad (\textbf{BIGRAMMA})$$
$$\mathbb{P}(x_L|x_{1:L-1}) \approx \mathbb{P}(x_L|x_{L-N+1:L-1}) \quad (\textbf{N-GRAMMA})$$

Estendendo l'approssimazione al caso generale degli N-grammi, la probabilità diviene:

$$\mathbb{P}(x_{1:L}) = \prod_{i=1}^{L}\mathbb{P}(x_i|x_{1:i-1}) \approx \prod_{i=1}^{L}\mathbb{P}(x_i|x_{i-N+1:i-1})$$

Per calcolare empiricamente tali probabilità, si ricorre alla Stima di Massima Verosimiglianza (MLE) sui conteggi C estratti dal corpus di addestramento:

$$\mathbb{P}(x_L|x_{L-1}) = \frac{C(x_{L-1}x_L)}{\sum_x C(x_{L-1}x)} = \frac{C(x_{L-1}x_L)}{C(x_{L-1})} \in [0,1]$$

Estendendo al caso generale degli N-grammi troviamo:

$$\mathbb{P}(x_i|x_{i-N+1}) = \frac{C(x_{i-N+1:i})}{C(x_{i-N+1:i-1})} \in [0,1]$$

La probabilità di x nel caso generale applicato agli N-grammi diventa quindi:

$$\mathbb{P}(x) \approx \prod_{i=1}^{L}\mathbb{P}_{\text{MLE}}(x_i|x_{i-N+1:i-1}) = \sum_{i=1}^{L}\frac{C(x_{i-N+1:i})}{C(x_{i-N+1:i-1})}$$

Per prevenire i fenomeni di *underflow* numerico derivanti dalla moltiplicazione di migliaia di probabilità marginali, le computazioni vengono traslate nello spazio logaritmico, convertendo i prodotti in sommatorie.

Nella decrittazione pratica, la determinazione del parametro N risulta fondamentale. Solitamente si sceglie di lavorare con 5-grammi o 6-grammi, poichè l'impiego di queste lunghezze rappresenta un ottimo compromesso tra il costo computazionale richiesto al sistema (che cresce esponenzialmente al crescere di N) e la potenza risolutiva necessaria all'algoritmo per arrivare correttamente al testo in chiaro.

3.2 Costruzione della funzione "Fitness" o "Score"

Per orientare la ricerca nello spazio delle chiavi, si definisce una funzione obiettivo, denominata "score" o "fitness", che valuta la bontà di una data chiave K. Più alto è il punteggio, maggiore sarà la probabilità che la chiave scelta sia corretta. Come dimostrato nel Capitolo 2, massimizzare questa funzione equivale a minimizzare la *cross-entropy* tra il modello linguistico e il testo decifrato. Pertanto, la funzione $S(K)$ viene definita direttamente come la log-probabilità del testo stesso:

$$S(K) = \log \mathbb{P}(K(c)) = \log \mathbb{P}(x) = \sum_{i=1}^{L} \log \mathbb{P}(x_i | x_{i-N+1:i-1})$$

Il problema crittanalitico si riduce formalmente a un problema di ottimizzazione combinatoria, in cui si ricerca la chiave k^* tale che :

$$K^* = \arg\max_{K \in \mathcal{K}} S(K)$$

Nell'impiego sui crittogrammi storici, il massimo globale matematico potrebbe non coincidere esattamente con la chiave originaria a causa di rumore introdotto dall'autore (errori di ortografia, omissioni, omofonia imperfetta). Tuttavia, la massimizzazione di $S(K)$ è il canale necessario per l'avvicinamento asintotico alla soluzione.

3.3 Distribuzione di Probabilità Stazionaria $\pi(K)$

Data l'immensità dello spazio delle chiavi K, l'approccio di forza bruta è impraticabile. Si introduce quindi la metodologia del *Simulated Annealing* per costruire una catena di Markov la cui distribuzione stazionaria converga verso le chiavi che ottimizzano $S(K)$. Richiamando la Definizione 13.1, si applica la distribuzione di Gibbs-Boltzmann:

$$\pi_{S,T}(k) = \frac{1}{Z_{S,T}} \exp\left(\frac{S(k)}{T}\right)$$

Dove:

- T è la temperatura (parametro di controllo che regola la dispersione della distribuzione).
- Z è la funzione di partizione, che garantisce la normalizzazione della probabilità.

Con $Z_{S,T}$ una costante di normalizzazione , definita come :

$$Z_{S,T} = \sum_{k \in K} \exp\left(\frac{S(k)}{T}\right)$$

questa garantisce che

$$\sum_{k \in K} \pi_{S,T}(k) = 1$$

Il limite computazionale di questa distribuzione risiede nel denominatore $Z_{S,T}$, il cui calcolo richiederebbe la valutazione di tutte le possibili chiavi dell'universo. Per campionare le chiavi eludendo il calcolo di $Z_{S,T}$, è imperativo ricorrere all'algoritmo di Metropolis-Hastings.

3.4 Campionamento stocastico con Metropolis-Hastings

L'algoritmo costruisce una catena di Markov in cui la probabilità di transizione da una chiave K a una chiave perturbata K' è data dal prodotto tra la distribuzione di proposta q e la probabilità di accettazione α:

$$\mathbb{P}(K \to K') = q(K'|K) \cdot \alpha(K, K')$$

Integrando una distribuzione strumentale simmetrica tale che $q(K'|K) = q(K|K')$, si garantisce il principio del bilancio dettagliato (Teorema 6.1) essenziale per la convergenza della distribuzione stazionaria :

$$\pi(K)\mathbb{P}(K \to K') = \pi(K')\mathbb{P}(K' \to K)$$

Sostituendo la riga precedente si ottiene:

$$\pi(K)q(K'|K)\alpha(K, K') = \pi(K')q(K|K')\alpha(K', K)$$

Sfruttando la simmetria si semplifica q e la formula diventa:

$$\frac{\alpha(K, K')}{\alpha(K', K)} = \frac{\pi(K')\cancel{q(K|K')}}{\pi(K)\cancel{q(K'|K)}} \implies \frac{\alpha(K, K')}{\alpha(K', K)} = \frac{\pi(K')}{\pi(K)}$$

Massimizzando il tasso di accettazione si ottiene:

$$\alpha(K, K') = \min\left(1, \frac{\pi(K')}{\pi(K)}\right)$$

Sostituendo l'equazione di Gibbs:

$$\frac{\pi(K')}{\pi(K)} = \frac{\cancel{\frac{1}{Z}} \exp\left(\frac{S(K')}{T}\right)}{\cancel{\frac{1}{Z}} \exp\left(\frac{S(K)}{T}\right)} = \exp\left(\frac{S(K') - S(K)}{T}\right) = \exp\left(\frac{\Delta S}{T}\right)$$

Da Metropolis-Hastings si ottiene quindi:

$$\alpha(K, K') = \min\left(1, \exp\left(\frac{\Delta S}{T}\right)\right)$$

(Dove ΔS è la differenza di fitness tra le due chiavi adiacenti).

3.5 Flusso Operativo dell'Algoritmo: Esecuzione Pratica

L'impianto teorico fin qui descritto si traduce in un ciclo computazionale iterativo. Nella pratica, l'algoritmo stocastico procede attraverso i seguenti step operativi:

- **Inizializzazione e Valutazione:** Il sistema genera casualmente una chiave di decifrazione iniziale K e fissa il parametro della temperatura T ad un valore molto elevato. Il crittogramma viene decodificato e il testo risultante viene valutato tramite il modello a N-grammi per calcolare il punteggio iniziale $S(K)$.

- **Perturbazione dello Spazio delle Chiavi:** Il sistema genera una nuova chiave candidata K' apportando una minima modifica stocastica a K. È in questa fase che si concretizza la differenza strutturale dei cifrari omofonici rispetto a quelli a sostituzione semplice. Mentre in quest'ultimi la perturbazione consiste nella permutazione di due lettere (con probabilità $q_{scambio}(K'|K) = 1/\binom{26}{2}$), nei cifrari omofonici la mappatura è "uno a molti". L'algoritmo seleziona casualmente uno degli m simboli cifrati e gli riassegna in modo indipendente una delle 26 lettere dell'alfabeto in chiaro. La probabilità di transizione strumentale assume quindi la forma $q_{omofonico}(K'|K) = \frac{1}{m} \cdot \frac{1}{26}$.

- **Calcolo dell'Accettazione:** Il testo viene decifrato con la nuova chiave K' per calcolarne il punteggio $S(K')$. Si determina il gradiente di *fitness* $\Delta S = S(K') - S(K)$ e si calcola la probabilità di accettazione di Metropolis $\alpha = \min(1, \exp(\Delta S/T))$. Essendo la distribuzione q simmetrica ($q(K'|K) = q(K|K')$), essa si semplifica nel rapporto di Metropolis-Hastings, preservando intatta l'infallibilità matematica dell'algoritmo.

- **Scelta Stocastica:** Il computer genera un numero pseudocasuale u estratto da una distribuzione uniforme continua nell'intervallo $[0, 1]$. Se $u \leq \alpha$, la nuova chiave K' viene accettata e diventa la chiave corrente; se $u > \alpha$, la chiave K' viene rifiutata e il sistema mantiene la vecchia chiave K.

- **Iterazione e Raffreddamento:** Il processo si ripete per migliaia di iterazioni per far assestare la catena, dopodiché la temperatura T viene ridotta moltiplicandola per il fattore di decadimento λ, dando inizio a un nuovo ciclo.

La dinamica di questo algoritmo ha una stretta dipendenza dal ruolo del generatore pseudocasuale e dalla temperatura. All'inizio dell'esecuzione, essendo T molto elevata, il termine $\exp(\Delta S/T)$ restituisce una probabilità alta anche in presenza di un ΔS fortemente negativo. Di conseguenza, il generatore casuale permetterà al sistema di accettare frequentemente chiavi che peggiorano temporaneamente la decifrazione. Questa mobilità stocastica è essenziale per evitare che l'algoritmo rimanga intrappolato in un massimo locale.

Proseguendo con le iterazioni e abbassando la temperatura, la medesima mossa peggiorativa produrrà una probabilità α prossima allo zero. L'algoritmo diventa progressivamente più selettivo, fino a trasformarsi in un ottimizzatore deterministico che accetta esclusivamente le chiavi in grado di incrementare il punteggio $S(K)$, guidando la decifrazione verso il massimo globale.

3.6 Applicazione del Teorema Ergodico e *Cooling Schedule*

La validità della decifrazione poggia sulla convergenza della catena di Markov, che risulta:

1. **Irriducibile:** la perturbazione $q(K'|K)$ garantisce che ogni stato in $\mathcal{K}$ sia raggiungibile.
2. **Aperiodica:** poiché $\alpha(K, K') < 1$, vi è una probabilità non nulla di rimanere nello stato corrente ($X^{(t+1)} = X^{(t)}$).
3. **Ricorrente positiva.**

Per la Legge dei Grandi Numeri per i processi markoviani (Teorema Ergodico 6.63), si ha la certezza della convergenza asintotica:

$$\lim_{t\to\infty} \mathbb{P}(X_t = K) = \pi(K)$$

Il parametro termodinamico T viene fatto decadere nel tempo secondo un fattore di raffreddamento $\lambda \in [0.99, 1]$.

Analizzando il limite per $T \to 0$:

- **Caso $\Delta S < 0$ (Peggioramento):** $\lim_{T\to 0} \exp\left(\frac{-|\Delta S|}{T}\right) = 0$.
  La probabilità di accettare una chiave peggiore si annulla progressivamente.
- **Caso $\Delta S > 0$ (Miglioramento):** L'esponenziale diverge a $+\infty$, per cui il minimo satura a 1. L'algoritmo diventa deterministico (similmente all'*Hill Climbing*) e accetta sempre la mossa.

Quindi per $T \to 0$ si ha:

$$\lim_{T\to 0} \pi(K) = \begin{cases} \frac{1}{|\mathcal{K}^*|} & \text{se } K \in \mathcal{K}^* \text{ (massimi globali)} \\ 0 & \text{altrimenti} \end{cases}$$

L'equilibrio termodinamico assicura che, mentre le alte temperature iniziali permettono alla catena di "scavalcare" i massimi locali accettando mosse peggiorative, il raffreddamento la guidi verso le configurazioni ottimali. L'azione combinata del Teorema Ergodico e del *cooling schedule* assicura matematicamente la convergenza dell'algoritmo verso la chiave ottimale k^*. Resta tuttavia da stabilire se tale massimo matematico coincida univocamente con la chiave storica corretta, limite che verrà affrontato nel Capitolo 5 tramite la Teoria dell'Informazione.

Capitolo 4

Estensione della teoria al crittogramma Z_{340}

4.1 Sviluppo degli Strumenti Computazionali per l'Ottimizzazione

Il primo Agosto del 1969 fu pubblicato sui giornali il primo crittogramma del killer dello Zodiaco, fu immediatamente decifrato da due civili usando il metodo delle frequenze. Il secondo cifrario, noto come Z_{340} apparve nei giornali il 12 Novembre del 1969, a differenza del primo questo rimase un mistero per 51 anni. Il motivo di tale mistero fu dovuto alla complessità della cifratura, resa tale da Zodiac stesso per mettere in difficoltà la polizia.



2 San Francisco Chronicle Thurs., Nov. 13, 1969

Zodiac 'Legally Sane'

Cop Sure Clues Will Snag Him

By Fred Avery

The detective in charge of the manhunt for the Zodiac killer who now boasts he has slain seven persons said yesterday: "I consider him to be legally sane."

Chief of Inspectors Martin Lee made the statement at a morning press conference on the Zodiac case held at San Francisco's Hall of Justice.

"He (Zodiac) knows right from wrong. He knows that what he has been doing is forbidden. He knows the law is after him," Lee said.

Catching Zodiac, Lee said, is only a matter of time.

"Our knowledge of this man is increasing . . . I am confident we will get him," Lee said.

CLUES

Through physical clues Zodiac has clumsily left behind at crime scenes and bits and pieces of information about himself he has inadvertently revealed in letters sent to The Chronicle, police feel sure there will be no doubt they have the right man in custody when an arrest is made.

Lee would not discuss what these clues and information are. All he would say is that "considerable evidence of many different kinds" has been amassed by the 20 San Francisco, Napa and Vallejo

Zodiac's message—cryptologist said it had definite patterns

Figura 4.1: Il crittogramma Z_{340} originale pubblicato sui giornali nel novembre 1969.

Il Capitolo 3 ha delineato la struttura teorica e algoritmica del motore stocastico MCMC, progettato per l'ottimizzazione della decifrazione di cifrari omofonici. L'applicazione di questo approccio al crittogramma Z_{340} ha tuttavia prodotto per decenni risultati negativi, dimostrando empiricamente che il sistema utilizzato non si limitava a una semplice sostituzione omofonica ma costituiva un sistema crittografico ibrido che comprendeva più metodi di cifratura. [2]. La decifrazione, formalizzata nel dicembre 2020 da David Oranchak, Sam Blake e Jarl Van Eycke, ha rivelato che l'ostacolo principale all'algoritmo era l'inserimento di una complessa trasposizione geometrica, complicata ulteriormente da errori ortografici e di cifratura che hanno alterato i risultati statistici per anni.[2].

Per affrontare un simile livello di complessità, è stato fondamentale lo sviluppo di *AZdecrypt*, un risolutore creato nel 2014 da Van Eycke e basato su un algoritmo di *Simulated Annealing* altamente ottimizzato [2]. Sfruttando una moderna CPU a 8 core, AZdecrypt è in grado di risolvere fino a 200 cifrari omofonici al secondo con un tasso di successo superiore al 99% [2]. Una delle evoluzioni più rilevanti per l'analisi dello Z_{340} è stata sviluppata nel 2019 dal programmatore Louie Helm, il quale ha ideato un metodo modulare per caricare in memoria le matrici di transizione per gli 8-grammi linguistici [2]. Fino a quel momento, l'uso di N-grammi di tali dimensioni richiedeva fino a 256 GB di RAM; il nuovo approccio ha permesso di comprimere i dati in meno di 16 GB, rendendo l'esplorazione computazionale accessibile su computer meno potenti [2]. A questo si è aggiunta un'ulteriore modifica proposta da Geoffrey LaTurner, che ha limitato il calcolo statistico affinché non attraversasse i confini tra le righe, agevolando l'individuazione di frammenti di testo trasposti [2].

4.2 L'Anomalia Periodica e lo Studio dei Bigrammi

La prima traccia matematica sull'effettivo utilizzo di una regola di trasposizione è stato riscontrato nell'analisi delle periodicità interne al testo cifrato [2]. Considerando il testo nella sua lettura sequenziale standard (periodo 1), lo Z_{340} presenta solamente 25 bigrammi ripetuti [2]. Per comprendere l'anomalia di questo dato, i ricercatori hanno condotto un esperimento su 1.000 testi cifrati omofonici artificiali della stessa lunghezza, i quali hanno mostrato una media di 34,5 bigrammi ripetuti [2]. I 25 bigrammi dello Z_{340} risultavano invece distanti appena 1,36 deviazioni standard dalla media delle permutazioni casuali del testo, suggerendo una totale assenza di dipendenza markoviana lungo la direzione orizzontale [2].

Calcolando le ripetizioni a distanze relative (salti) variabili, l'analisi ha evidenziato una particolarità statistica: a una distanza di 19 caratteri (periodo 19), il conteggio dei bigrammi ripetuti registra un picco anomalo pari a 37 occorrenze [2]. Ulteriori manipolazioni topologiche della griglia, come la trasposizione dell'ultima colonna prima della prima, hanno innalzato questo valore a 45 ripetizioni [2]. Tale anomalia ha fornito la prova decisiva che il testo fosse stato sottoposto ad una trasposizione geometrica periodica , oltre che ad una suddivisione in quadranti [2].

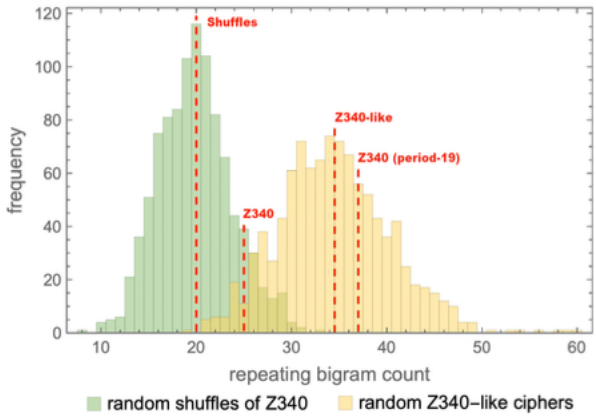


Figura 4.2: Distribuzione dei bigrammi ripetuti nelle permutazioni casuali dello Z_{340} (a sinistra) rispetto ai testi artificiali (a destra). Sono evidenziate le linee di media e i valori per il periodo 1 e il periodo 19.

4.3 Trasposizione Geometrica e Decimazione

Sfruttando l'anomalia del periodo 19, la ricerca si è orientata sull'esplorazione di schemi di trasposizione bidimensionale applicati alla griglia originale di 20 righe per 17 colonne [2]. La strategia risolutiva consiste in una decimazione doppiamente periodica propria (*doubly-periodic proper decimation*) [2]. Fisicamente, l'operazione equivale ad una lettura diagonale con passo asimmetrico: ci si sposta di una riga verso il basso e due colonne verso destra, ripartendo dal lato opposto in caso di superamento del bordo [2]. Questo metodo è stato denominato *salto del cavallo*.

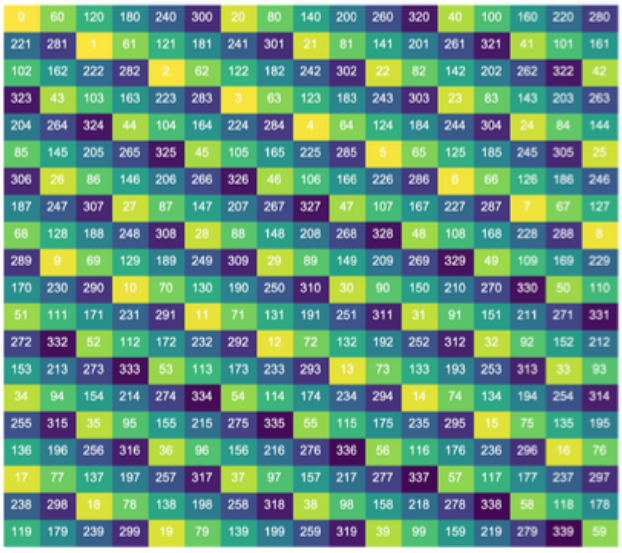


Figura 4.3: Rappresentazione della decimazione $(1, 2)$ applicata a una matrice bidimensionale.

Matematicamente, definita una griglia di dimensioni $N \times M$, la decimazione (n, m) di un array di elementi C è espressa dalla formula:

$$dec_{n,m}(C_k) = C_{n \cdot k \bmod N,\ m \cdot k \bmod M}$$

per $k \in [0, N \times M - 1]$ [2].

Nello scenario dello Z_{340}, i parametri individuati sono $n = 1$ e $m = 2$, tuttavia l'algoritmo non è stato applicato all'intera matrice simultaneamente [2]. Il crittogramma risulta diviso orizzontalmente in quattro partizioni indipendenti: un blocco superiore da 9 righe, un blocco centrale da 9 righe e due righe finali (la 19 e la 20) trattate singolarmente come sezioni indipendenti [2].

4.4 Il Limite della Molteplicità e il Metodo Crib

Il presupposto della presenza di decimazioni ha generato una libreria di oltre 655.088 varianti di trasposizione [2]. Elaborando questo insieme di dati in AZdecrypt, il sistema ha restituito un *output* parziale per la prima sezione da 9 righe contenente frammenti perfettamente coerenti con lo stile dell'autore, come "HOPE YOU ARE" e "TRYING TO CATCH ME" [2]. Nonostante l'individuazione di queste frasi, l'algoritmo falliva nel convergere autonomamente sul resto del testo a causa dell'alta molteplicità del segmento [2]. Valutando 63 simboli unici su un testo di soli 153 caratteri, la prima sezione presentava una molteplicità di 0,41, dilatando di molto lo spazio delle chiavi esplorabili rispetto allo 0,13 del precedente cifrario Z_{408} [2].

Per forzare il collasso della distribuzione verso l'ottimo globale, è stato applicato il metodo *Crib* [2]. Fissando forzatamente i frammenti in chiaro precedentemente individuati alle loro posizioni esatte nel crittogramma, i ricercatori hanno individuato i valori di 27 dei 63 simboli unici, vincolando definitivamente il 43% della chiave e illuminando il 54% del testo della prima sezione [2].

	H	O	P	E	Y	O	U	A	R	E	H					G
H	+	M	8	|	C	V	@	K	E	B	+	*	5	k	.	L
		T		O						N	T	R	Y	I	N	G
d	R	[	U	V	F	F	z	9	<	>	#	Z	3	P	>	L
T	O	C	A	T	C	H	M	E	T	H	A			A		
[	M	p	0	G	p	+	2	]	G	+	I	%	W	0	&	D
T	M				T	H		T		S	H	O			H	
#	2	b	^	D	[	+	4	[	5	J	+	V	W	]	+	k
C	H		R	I		G			P	A					T	A
p	+	f	Z	P	Y	L	R	/	8	K	j	R	k	.	#	K
B			T	M	E			M				A				
_	R	q	#	2	|	<	z	2	9	^	%	0	F	1	*	H
	O		T	H	E	G	A	S	C	H	A	M	B	E	R	
S	M	F	;	+	B	L	K	J	p	+	I	2	_	c	T	f
E	C		A				T						E			M
B	p	z	0	U	N	y	G	]	y	7	t	-	c	Y	A	2
									C	E	A			T	H	
N	:	^	j	*	X	z	6	d	p	c	I	d	d	G	+	4

Figura 4.4: Inserimento delle stringhe dedotte (*crib*) all'interno della griglia trasposta della prima sezione.

Utilizzando la nuova implementazione in AZdecrypt capace di inserire automaticamente gli spazi bianchi tra le parole calcolando la verosimiglianza linguistica tramite N-grammi, il motore stocastico ha operato in uno spazio significativamente più ristretto, convergendo in tempi minimi sul testo restante e confermando la decifrazione completa del crittogramma [2]. La correttezza di questa prima soluzione è stata confermata dai riferimenti storici presenti nel testo, il messaggio decifrato citava esplicitamente: "Non ero io quello in TV" e "Non ho paura della camera a gas", richiamando inequivocabilmente una celebre telefonata giunta al programma televisivo di Jim Dunbar poche settimane prima della spedizione del cifrario [2].

4.5 Risoluzione Finale, Correzione degli Errori e Validazione

L'individuazione di tutte le corrispondenze omofone tramite la prima sezione ha permesso di attaccare i blocchi successivi, rivelando asimmetrie o errori introdotti dall'autore al momento della cifratura [2]. Nella seconda sezione da 9 righe, il riallineamento perdeva coerenza: l'autore aveva accidentalmente omesso un simbolo cifrato sulla quindicesima riga globale, e la sequenza in chiaro "LIFE IS" era stata ignorata durante il processo di trasposizione originale [2].

Per ripristinare la regolarità della decimazione, è stato applicato uno scorrimento circolare verso destra (*circular right-shift*) sulla riga compromessa [2]. Corretti tali errori, il messaggio finale si è rivelato totalmente: *"Spero che vi stiate divertendo un mondo a cercare di prendermi; non ero io quello in TV... Non temo la morte perché so che la mia nuova vita sarà facile in Paradiso."* [2].

Per confermare la validità assoluta del risultato, il 5 dicembre 2020 i ricercatori hanno inviato i risultati matematici alla *Cryptanalysis and Racketeering Records Unit* (CRRU) dell'FBI, la quale, l'11 dicembre, ha confermato formalmente l'avvenuta risoluzione indipendente del cifrario [2].

Per avvalorare in modo rigoroso la validità del risultato ottenuto, l'analisi può rifarsi al concetto di *distanza di unicità* introdotto da Claude Shannon [2]. Applicando la teoria dell'informazione alle specifiche caratteristiche crittografiche dello Z_{340} (omofonia, partizionamento, trasposizione ed errori), il crittologo Joachim von zur Gathen ha calcolato che l'entropia del sistema richiedeva una distanza di unicità pari a un massimo di 152 caratteri [2]. Poiché il messaggio Z_{340} è costituito da 340 caratteri, esso si colloca nettamente al di sopra di questo limite teorico, garantendo matematicamente che la soluzione individuata tramite l'ottimizzazione stocastica MCMC sia l'unica decrittazione plausibile del crittogramma [2].

Capitolo 5

Limiti di MCMC e Teoria dell'informazione

L'implementazione dell'algoritmo Metropolis-Hastings e del Simulated Annealing garantisce, per il Teorema Ergodico, la convergenza asintotica verso una chiave ottimale che massimizzi la funzione fitness logaritmica. Tuttavia, l'ottimizzazione stocastica risolve esclusivamente il problema computazionale della ricerca nello spazio delle chiavi, non può assicurare che il massimo matematico individuato coincida con la chiave corretta. Per stabilire i limiti di validità di questo attacco crittanalitico, è necessario ricorrere alla teoria della segretezza formulata da Claude E. Shannon [7].

5.1 Il Teorema di Shannon-McMillan-Breiman

L'algoritmo MCMC esplora lo spazio delle chiavi valutando ogni possibile decifrazione tramite una funzione fitness logaritmica $S(K)$ che ne restituisce un punteggio. Sebbene intuitivamente si cerchi la sequenza di caratteri che massimizzi l'accuratezza della decifrazione attraverso l'uso della funzione fitness, la validità matematica di questo approccio si basa sulla Teoria dell'Informazione e, in particolare, sul Teorema di Shannon-McMillan-Breiman (noto anche come Proprietà di Equipartizione Asintotica o AEP).

Sia $\mathcal{X}$ un processo stocastico stazionario ed ergodico che modella la lingua naturale, con distribuzione di probabilità reale p. L'entropia del linguaggio è definita come:

$$H(\mathcal{X}) = \lim_{n\to\infty} -\frac{1}{n} \sum_{x_{1:n}} p(x_{1:n}) \log p(x_{1:n})$$

Nella decifrazione pratica, la vera distribuzione p è ignota e viene approssimata tramite un modello a N-grammi m. La divergenza tra la vera distribuzione e il modello è misurata dalla *Cross-Entropy* (Entropia Incrociata) $H(p, m)$.

Il Teorema di Shannon-McMillan-Breiman stabilisce che, per una singola sequenza sufficientemente lunga generata dal processo, la media delle log-probabilità converge quasi certamente alla cross-entropy:

$$-\frac{1}{n} \log m(X_1, X_2, \ldots, X_n) \xrightarrow{n\to\infty} H(p, m)$$

Poiché la nostra funzione di fitness è definita esattamente come:

$$S(K) = \sum \log m(x_i | x_{i-N+1:i-1})$$

risulta evidente che:

$$S(K) \approx -n \cdot H(p, m)$$

Questa equazione fornisce la dimostrazione formale del nostro metodo: massimizzare il punteggio della funzione fitness logaritmica calcolata tramite N-grammi equivale a minimizzare l'entropia incrociata tra il testo decifrato e il modello linguistico acquisito. Si può quindi affermare che l'MCMC è stocastico (casuale) nella generazione dei campioni, ma non è una casualità "cieca" bensì guidata dalla probabilità degli N-grammi che spingono il testo candidato verso la vera distribuzione statistica della lingua inglese.

5.2 Il problema dell'Equivocazione e le Soluzioni Multiple

Come introdotto nella Sezione 2.12.2, esiste un legame tra l'architettura di un sistema crittografico e quella di un canale di comunicazione rumoroso. In questo modello, il testo cifrato funge da segnale disturbato, mentre l'incertezza residua del crittanalista viene calcolata rigorosamente attraverso la funzione di equivocazione di Shannon [7]. Essendo l'equivocazione una funzione non crescente, l'algoritmo MCMC si scontra con un limite fisico cruciale legato alla lunghezza N del testo intercettato.

Quando il numero di caratteri intercettati N è piccolo rispetto all'enorme dimensione dello spazio delle chiavi, l'equivocazione della chiave $H_E(K)$ rimane elevata. In questo scenario si presentano "soluzioni multiple" al crittogramma, ovvero stati con densità di probabilità equivalenti [7]. Nei cifrari omofonici, a causa della mappatura uno a molti che incrementa notevolmente il numero di chiavi possibili, questa fase di alta incertezza si estende a testi molto più lunghi rispetto ad una semplice sostituzione.

Se l'algoritmo MCMC venisse eseguito con queste modalità, la sua ricerca stocastica convergerebbe inevitabilmente verso uno dei tanti massimi locali, restituendo una chiave spuria che produce un testo in chiaro sensato ma storicamente errato. Solo quando N è sufficientemente grande, superando la soglia critica del sistema, l'equivocazione crolla e la catena di Markov può convergere verso l'unica decifrazione statisticamente e storicamente corretta.

5.3 Stima dell'Entropia dell'Inglese e Limiti Teorici

La transizione da uno stato di soluzioni multiple a una soluzione unica è guidata dalla ridondanza della lingua, che definisce il limite teorico invalicabile della Distanza di Unicità (Sezione 2.14.2). Per quantificare questo limite e applicarlo alla crittanalisi, è necessario calcolare l'esatta ridondanza dell'inglese.

Nel suo lavoro del 1951, *"Prediction and Entropy of Printed English"*, Shannon dimostrò che, all'aumentare dell'ordine degli N-grammi, l'entropia dell'inglese decresce significativamente. Considerando un alfabeto di 26 lettere, la massima entropia teorica è $\log_2(26) \approx 4.7$ bit per lettera. Tuttavia, le forti dipendenze a lungo raggio riducono l'entropia reale $H(L)$ dell'inglese a un valore compreso tra 1.0 e 1.5 bit per lettera. La ridondanza statistica D si ottiene per differenza [8]:

$$D = \log_2(26) - H(L) \approx 4.7 - 1.2 = 3.5 \text{ bit per lettera}$$

Questo parametro permette di calcolare numericamente la Distanza di Unicità applicandola direttamente al caso di studio dello Z_{340}. Come illustrato nella Sezione 2.15, se si considera il limite superiore teorico per lo spazio delle chiavi di questo cifrario — in cui ciascuno dei 63 simboli cifrati unici può assumere in modo indipendente il valore di una qualsiasi delle 26 lettere — lo spazio esplorabile ammonta a 26^{63} (upper bound) possibili combinazioni. La dimensione dell'incertezza della chiave $H(K)$ in bit risulta quindi:

$$H(K) = \log_2(26^{63}) = 63 \cdot \log_2(26) \approx 63 \cdot 4.7 \approx 296.1 \text{ bit}$$

Sostituendo questo valore nella formula teorica della distanza di unicità si ottiene:

$$N \approx \frac{296.1}{3.5} \approx 84.6 \text{ caratteri}$$

Questo calcolo dimostra formalmente che, considerando lo Z_{340} esclusivamente come una pura sostituzione omofonica (senza considerare le trasposizioni), la ridondanza del linguaggio non è sufficiente a vincolare un'unica soluzione sotto la soglia di circa 85 caratteri. In porzioni di testo più brevi, il paesaggio termodinamico esplorato dall'algoritmo MCMC presenterebbe infiniti picchi di fitness equivalenti, portando quindi a decifrazioni ingannevoli. Inoltre, l'aggiunta di ulteriori approcci crittografici, quali la trasposizione geometrica e gli errori di cifratura originali, contribuiscono ad innalzare ulteriormente questo limite di unicità fino a 152 caratteri, come verrà sottolineato nella sezione successiva.

5.4 Validità della Soluzione e Work Characteristic

Questa severa dipendenza dalla lunghezza del testo ha fornito la verifica matematica definitiva per la decifrazione del crittogramma Z_{340} [2]. L'entropia dello spazio degli stati del cifrario di Zodiac richiedeva una distanza di unicità massima di 152 caratteri; poiché il messaggio era costituito da 340 caratteri, la decifrazione estratta dall'algoritmo stocastico è risultata matematicamente univoca e inattaccabile, escludendo l'ipotesi di una soluzione spuria [2].

Tuttavia, come evidenziato dalla teoria dell'informazione, il superamento della distanza di unicità garantisce solo l'esistenza teorica di un'unica soluzione, ma non quantifica la fattibilità pratica del suo ritrovamento [7]. Per questo Shannon introduce la funzione $W(N)$, ovvero la *Work Characteristic*, che misura la quantità media di lavoro necessaria per determinare la chiave [7].

Sebbene per $N > 152$ la soluzione dello Z_{340} fosse teoricamente unica, il metodo della prova ed errore completo (forza bruta) richiederebbe tempi nell'ordine di 3×10^{12} anni anche testando un milione di chiavi al secondo [7]. È proprio in questa divario tra la vulnerabilità teorica (garantita dalla ridondanza) e l'impossibilità computazionale della forza bruta (espressa dalla *Work Characteristic*) che il metodo MCMC si rivela essenziale, sfruttando la discesa termodinamica per isolare la soluzione riducendo drasticamente il lavoro computazionale $W(N)$.

5.5 Il Limite dell'Approssimazione Markoviana e il Collasso Entropico

Sebbene il Teorema di Shannon-McMillan-Breiman giustifichi l'uso della funzione fitness basata sugli N-grammi, questo approccio presenta una vulnerabilità teorica quando applicato ai cifrari omofonici. Tale vulnerabilità deriva dalla natura stessa dell'assunzione di Markov, la quale approssima la probabilità dell'intera sequenza valutando esclusivamente finestre locali di lunghezza N [4].

Nei cifrari omofonici, lo spazio del testo cifrato è strettamente maggiore dello spazio del testo in chiaro ($|\mathcal{C}| > |\mathcal{P}|$). Questo squilibrio fornisce all'algoritmo stocastico un eccesso di gradi di libertà. Guidata dalla pura massimizzazione della probabilità locale, la catena di Markov tende a sfruttare una "scorciatoia" matematica: mappa un numero sproporzionato di simboli cifrati verso le lettere a più alta frequenza della lingua inglese (come E, T, A, O).

Il risultato di questa anomalia è la generazione di testi degeneri (ad esempio, sequenze ripetitive come "THE THE THE" o sovrabbondanza di bigrammi come "EE" o "TH"). Questi testi ottengono paradossalmente un punteggio di fitness locale (N-gram score) altissimo, pur essendo linguisticamente insensati. Questo fenomeno è noto come *collasso entropico*.

Per prevenire la convergenza dell'MCMC verso questi falsi positivi statistici, l'ottimizzazione locale degli N-grammi deve essere vincolata dalle leggi termodinamiche globali del linguaggio. Secondo la teoria dell'informazione, un testo naturale non è definito solo dalle probabilità di transizione tra lettere adiacenti, ma possiede una macro-distribuzione globale la cui incertezza è quantificabile tramite l'entropia del singolo carattere (unigramma):

$$H = -\sum_{x \in \mathcal{P}} p(x) \log p(x)$$

La lingua inglese, pur presentando evidenti squilibri nelle frequenze assolute, mantiene un'entropia globale degli unigrammi ben definita (stimabile teoricamente intorno a 2.8 - 2.9 nats se si utilizza il logaritmo naturale). Una sequenza in cui l'algoritmo ha forzato troppi omofoni sulle lettere frequenti vedrà la propria entropia abbassarsi drasticamente, poiché la distribuzione $p(x)$ si concentrerà su pochissimi elementi, allontanandosi dalla vera distribuzione della lingua.

Questa limitazione teorica ben nota nello sviluppo di risolutori stocastici avanzati come AZdecrypt [2], impone l'introduzione di una funzione di regolarizzazione esterna. Affinché l'algoritmo di Metropolis-Hastings esplori esclusivamente le

configurazioni linguisticamente valide senza bloccarsi in falsi positivi, è matematicamente necessario imporre una barriera energetica (sotto forma di penalizzazione del gradiente ΔS) che respinga istantaneamente qualsiasi chiave candidata la cui entropia globale scenda al di sotto della soglia teorica del linguaggio.

5.6 Limiti di Convergenza MCMC e Mixing Time

Sebbene il Teorema Ergodico garantisca che l'algoritmo MCMC visiti la distribuzione stazionaria nel limite $t \to \infty$, l'applicazione pratica è vincolata dalla velocità con cui la catena dimentica il suo stato iniziale. Questo parametro è formalizzato come *Mixing Time* (Tempo di Mescolamento) [5].

Definita la distanza in variazione totale tra la distribuzione della catena al tempo t e la distribuzione stazionaria π, il tempo di mescolamento $\tau(\epsilon)$ è il numero minimo di passi necessari affinché la distanza diventi inferiore ad una soglia ϵ [5]:

$$\tau(\epsilon) = \min\{t : \|P^t(x, \cdot) - \pi\|_{TV} \leq \epsilon\}$$

Nel contesto crittografico, il mixing time di una catena che esplora lo spazio delle chiavi omofoniche dipende profondamente da due fattori: la lunghezza del testo e la pianificazione dello schema di raffreddamento (*cooling schedule*). Se il testo è eccessivamente corto, il paesaggio termodinamico si appiattisce, generando massimi locali multipli causati da coincidenze statistiche. In questo scenario, la catena di Markov impiega un tempo esponenziale per "saltare" le valli di probabilità tra due massimi locali. Inoltre, se la temperatura T nel Simulated Annealing viene abbassata troppo rapidamente rispetto al mixing time naturale della catena, il sistema subisce un congelamento termico prematuro (shock termico). Invece di convergere alla distribuzione π, la catena collassa in una ricerca avida (*Hill Climbing*) e si blocca nel primo sub-ottimo locale incontrato, fallendo l'esplorazione globale dello spazio delle chiavi [1].

Capitolo 6

Analisi Sperimentale e Ottimizzazione dei Parametri Stocastici

6.1 Setup dell'Esperimento e Architettura Software

Al fine di validare empiricamente l'impianto teorico delineato nei capitoli precedenti, è stato sviluppato un motore di decrittazione stocastica in linguaggio Python. L'implementazione ha previsto una prima fase sperimentale basata su modelli a 3-grammi, per poi evolvere verso un'analisi più complessa a 4-grammi.

L'architettura del software si articola in due macro-fasi: inizialmente, il sistema agisce da generatore, acquisendo un testo in chiaro in lingua inglese e applicando una cifratura omofonica; successivamente, esegue l'attacco crittanalitico vero e proprio impiegando il metodo Markov Chain Monte Carlo (MCMC). Per ragioni di efficienza computazionale, la cardinalità dell'alfabeto cifrato è stata limitata a 50 simboli, costituiti dalle 26 lettere standard dell'alfabeto inglese e da 24 omofoni extra assegnati in modo proporzionale alle lettere a maggior frequenza statistica.

La valutazione delle chiavi candidate è governata dalla funzione *fitness*, costruita a partire dal modello linguistico stocastico. Tale modello è stato addestrato calcolando le probabilità tramite la Stima di Massima Verosimiglianza (MLE) sul corpus di riferimento *text8*, un dataset che comprende circa 100 milioni di caratteri.

Per assicurare la coerenza semantica del risultato e scongiurare il fenomeno del collasso entropico – ovvero la convergenza dell'algoritmo verso sequenze che massimizzano la funzione fitness ma risultano del tutto prive di significato linguistico – la funzione obiettivo è stata rigorosamente regolarizzata. Come discusso nel Capitolo 5, si è introdotto un vincolo globale basato sull'Entropia di Shannon (H) [7]. Sfruttando le stime entropiche caratteristiche della lingua inglese, è stata imposta una severa penalizzazione algoritmica per tutti quegli stati che presentano un livello di entropia incompatibile con il linguaggio naturale, intervenendo

ogni qualvolta si registri $H < 2.5$ [7]:

```
counts = collections.Counter(text)
entropy = 0.0
for count in counts.values():
    p = count / len(text)
    entropy -= p * math.log(p)
if entropy < 2.5:
    score -= 5000.0
```

Listing 6.1: Regolarizzazione della fitness tramite Entropia di Shannon

Per mitigare l'impatto di eventuali errori ortografici o anomalie statistiche nel crittogramma, sono state considerate due diverse tecniche di regolarizzazione: la tecnica dell'add-k smoothing (con $k = 1$, chiamata anche Metodo di Laplace) e la tecnica dell'interpolazione lineare.

Per quanto concerne l'algoritmo di *Simulated Annealing*, l'architettura generale (Appendice A) prevede un modello di raffreddamento ad epoche (o blocchi). I parametri termodinamici sono stati inizializzati con una temperatura di partenza $T_{start} = 15.0$ e una temperatura finale $T_{fin} = 0.5$. Questa configurazione permette alla catena di stabilizzarsi procedendo con un fattore di decadimento (*cooling rate*) pari a $\lambda = 0.90$ e con 15 000 iterazioni per singola epoca, per un volume computazionale complessivo di 495 000 iterazioni.

Il lavoro sperimentale si è successivamente focalizzato sull'ottimizzazione dell'algoritmo, analizzando inizialmente i modelli a 3-grammi per poi estendere la teoria ai 4-grammi con l'implementazione delle necessarie modifiche strutturali. Le analisi si sono concentrate in particolar modo sui seguenti aspetti:

- **Cooling schedule**: studio delle dinamiche di raffreddamento del sistema, al fine di calibrare un *Simulated Annealing* ottimale che permetta al processo stocastico di sfuggire ai massimi locali, garantendo la convergenza asintotica verso l'ottimo globale.
- **Tecniche di smoothing**: indagine comparativa per la costruzione della probabilità smussata ottimale, valutando le differenze prestazionali tra il metodo di Laplace e l'Interpolazione Lineare.
- **Riavvii multipli (Random Restarts)**: introduzione di esecuzioni indipendenti multiple, necessarie per mitigare il rumore stocastico e l'imprevedibilità esplorativa delle catene di Markov.

Risulta infine fondamentale dimostrare come questa impalcatura stocastica si presti alla costruzione di un algoritmo ibrido, un passaggio indispensabile per massimizzare l'accuratezza e raggiungere una decifrazione definitiva. A tale scopo, viene introdotto il metodo *Crib*, concepito per portare a compimento l'operazione innescata dall'MCMC sfruttandone i risultati parziali. Il principio teorico alla base di questo metodo consiste nel fissare le frazioni di testo (parole o stringhe di senso compiuto) emerse nei passaggi stocastici precedenti; questo ancoraggio permette di ridurre drasticamente le dimensioni dello spazio delle chiavi, spingendo l'ultima fase della ricerca MCMC verso la decrittazione esatta.

6.2 Markov Chain Monte Carlo applicato ai trigrammi

Il codice generale (Appendice A) implementa la logica appena descritta: l'obiettivo è quello di cifrare un testo in inglese e poi procedere con l'applicazione dell'algoritmo crittanalitico per valutarne la capacità di decifrazione. In questa fase è fondamentale cercare parole di senso compiuto, che possano essere utilizzate nel secondo step per ridurre lo spazio delle chiavi. Un esempio di output ottenuto è il seguente:

Testo originale:

```
IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
 THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINT
 ABOUTMEIAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEI
 TWILLSENDMETOPARADISEALLTHESOONERBECAUSEIN
OWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVERYO
 NEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHE
 YAREAFRAIDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWT
 HATMYNEWLIFEWILLBEANEASYONEINPARADISEDEATH
```

Simboli unici nel crittogramma: 41

Modello linguistico a Trigrammi:

```
--- AVVIO TENTATIVO 1 DI 5 ---
Risultato parziale:
IHAMEYAUEREHATESSSNTRADDUTITTRYITSTICATCHLETHATTAS... (Score: -816.00)

--- AVVIO TENTATIVO 2 DI 5 ---
Risultato parziale:
LOSUNDSTGIRASANINEITISSSTELEMIDLENTRERTEAONEOSTERE... (Score: -855.50)

--- AVVIO TENTATIVO 3 DI 5 ---
Risultato parziale:
SNCOSSCENERUNTRINEOFECHHEISIAESSINFROSFOULDINTFESE... (Score: -828.10)

--- AVVIO TENTATIVO 4 DI 5 ---
Risultato parziale:
CATUNDTHTINATERAMEOPITTTHECESIDCEMPRESPEAINEASPESE... (Score: -830.22)

--- AVVIO TENTATIVO 5 DI 5 ---
Risultato parziale:
LNCASSCHTEROTHRESEETECIIHILIAESLISTRISTIONDINDTOSE... (Score: -842.39)
```

RISULTATO FINALE DELLA CATENA MCMC (Migliore dei tentativi):

OBIETTIVO:

IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINT
ABOUTMEIAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEI
TWILLSENDMETOPARADISEALLTHESOONERBECAUSEIN
OWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVERYO
NEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHE
YAREAFRAIDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWT
HATMYNEWLIFEWILLBEANEASYONEINPARADISEDEATH

OTTENUTO:

IHAMEYAUEREHATESSSNTRADDUTITTRYITSTICATCHLE
THATTASTCLEATTHECTSHOTTHECHBRENSSUMEMINST
ABOUTLENALSATADRANINDCHESASCHALBERBECAURTI
CTISSRETILECAMERAINSEASSTHESIATERBECEUSEIN
ATHATTENAUSHRSATERTOTORTDARLETHEREAREETERY
ONEESSEHASSITHISSTHENTHEYREACHMAREINRESOTH
EYAREADRAIIIDIEETHEALSATEDRAIIBECAUSENTNIT
THATLYNETSIDETESSBEATEASYNTEISMARAINSTIEATH

ACCURATEZZA: 59.35%

Essendo l'MCMC un metodo stocastico, il valore dell'accuratezza può restituire risultati estremamente diversi da prova a prova. Con 5 ripetizioni si ottiene una media dell'accuratezza di circa 36,45%. Questo punteggio può sembrare disastroso, ma è perfettamente in linea con la teoria precedentemente analizzata. È normale che l'accuratezza abbia evidenti variazioni, potendo passare dal 22.85% al 59.35% in due esecuzioni consecutive. La natura imprevedibile di questo sistema è dovuta all'immensità dello spazio delle chiavi da esplorare.

Nonostante non si ottengano risultati perfetti, si possono comunque apprezzare diverse parole di senso compiuto individuate durante la decrittazione, come: "I", "CATCH", "ABOUT", "BECAUSE". Queste sono sufficienti per implementare il metodo *Crib* e forzare la decrittazione.

Prima di applicare il metodo *Crib* è opportuno, però, presentare i diversi metodi con cui si è lavorato per ottimizzare questo modello.

6.2.1 Analisi Termodinamica: Dinamiche di Raffreddamento per i trigrammi

Al fine di ottimizzare l'accuratezza della decrittazione, risulta fondamentale determinare la strategia di raffreddamento più adatta. Si è pertanto condotto uno studio empirico confrontando diversi approcci con la modalità Trial and Error: mantenendo costanti le temperature iniziali e finali, sono stati fatti variare i tipi di raffreddamento e i rispettivi tassi di decadimento termico per valutarne l'impatto sul sistema. Il comportamento dell'algoritmo è stato analizzato monitorando l'evoluzione della funzione *fitness* corrente. L'andamento atteso è paragonabile ad un tracciato sismografico: alle alte temperature le fluttuazioni energetiche sono

molto evidenti; successivamente, al diminuire della temperatura, la varianza si riduce notevolmente, accompagnata da una crescita della funzione verso il massimo globale. L'obiettivo è calibrare un raffreddamento sufficientemente lento da consentire la stabilizzazione della catena di Markov, garantendo contemporaneamente una mobilità stocastica adeguata ad accettare peggioramenti temporanei per sfuggire ai massimi locali.

Si mettono a confronto i seguenti metodi di raffreddamento:

- **Raffreddamento continuo ottimale**: la temperatura viene abbassata a ogni iterazione con un *cooling-rate* di 0.99995. Si tratta di un decadimento abbastanza lento da permettere alla catena di stabilizzarsi gradualmente, evitando shock termici importanti.

- **Raffreddamento continuo veloce**: la temperatura decresce ad ogni iterazione con un *cooling-rate* di 0.999. Questo decadimento risulta essere eccessivamente rapido, non permette alla catena di stabilizzarsi e la costringe a bloccarsi nel primo massimo locale incontrato senza possibilità di scavalcarlo, riducendo di fatto l'algoritmo a un semplice Hill Climbing.

- **Raffreddamento ad epoche** (0.78, 0.85, 0.90): prima di abbassare la temperatura si eseguono 15 000 iterazioni. Il numero di epoche è quindi strettamente legato al *cooling-rate* scelto. Nello specifico: con 0.78 servono 14 epoche per raggiungere la temperatura finale, per un totale di 210 000 iterazioni; con 0.85 sono necessarie 21 epoche, totalizzando 315 000 iterazioni; infine, con 0.90 occorrono 33 epoche, per un totale di 495 000 iterazioni.

Implementando queste configurazioni (Appendice B), si ottiene il seguente grafico:

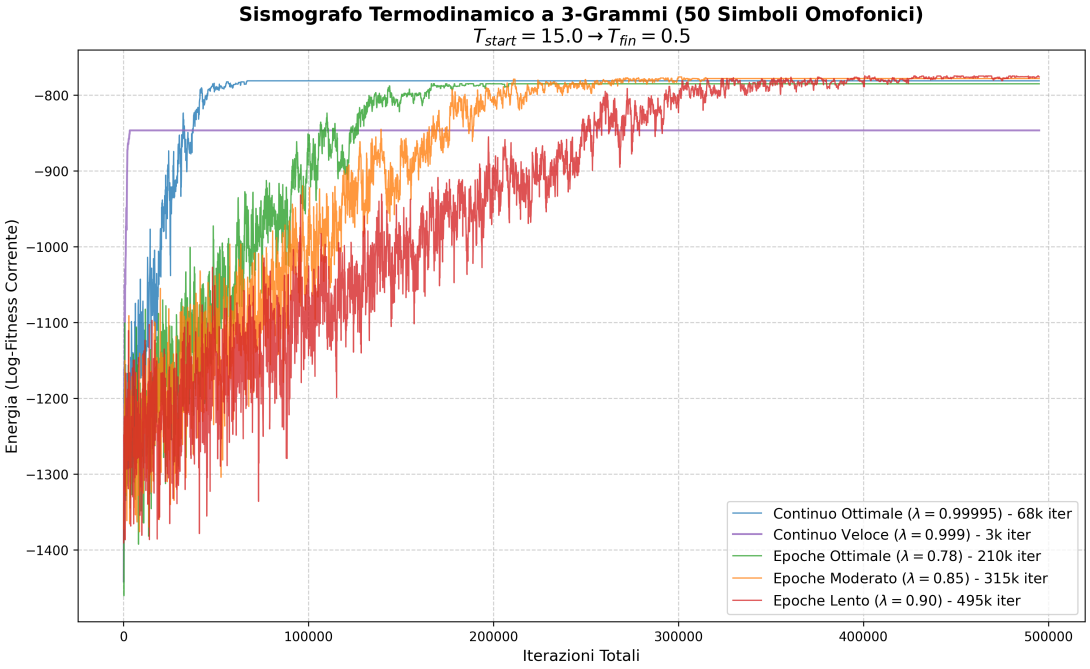


Figura 6.1: Convergenza termodinamica della Log-Fitness al variare del cooling schedule (5 test a confronto).

L'analisi del grafico evidenzia come il raffreddamento continuo veloce converga quasi istantaneamente verso un massimo locale, esaurendo la sua esplorazione dopo appena 3 000 iterazioni. Notevolmente diverso è il comportamento delle altre configurazioni. Il raffreddamento continuo ottimale mostra come anche un numero inferiore di iterazioni possa fornire buoni risultati: dopo 68 000 iterazioni si assesta su un massimo locale che restituisce comunque un punteggio eccellente, persino superiore a quello raggiunto dal raffreddamento ad epoche con $\lambda = 0.78$, il quale, nonostante le sue 210 000 iterazioni, non riesce ad avvicinarsi al massimo globale.

Risultati significativamente migliori si ottengono con i raffreddamenti ad epoche moderato ($\lambda = 0.85$) e lento ($\lambda = 0.90$). Entrambi superano i massimi locali precedenti mostrando un andamento regolare e riuscendo ad assorbire, anche dopo 200 000 iterazioni, notevoli peggioramenti termodinamici funzionali alla scoperta di nuovi massimi. Pur essendo entrambi ottimi ottimizzatori, è fondamentale sottolineare come il raffreddamento ad epoche lento ottenga le prestazioni migliori: superate le 250 000 iterazioni, quando il modello moderato è ormai troppo "freddo" per concedersi ampie variazioni, il modello lento conserva una temperatura sufficientemente alta per continuare l'esplorazione, raggiungendo i picchi massimi. È altresì rilevante osservare che, perfino alla soglia delle 495 000 iterazioni, il sistema mostra ancora un margine, seppur minimo, di ricerca. Ciò suggerisce che, prolungando ulteriormente il processo di raffreddamento e incrementando il numero di iterazioni, questo *cooling-schedule* si confermerebbe l'ottimizzatore ideale per la ricerca del massimo globale.

Per un confronto più rigoroso, la tabella seguente riassume i massimi globali e le medie dei massimi (calcolate sul 5% e sul 10% dei valori più alti dell'intera

funzione).

Configurazione	**Max Assoluto**	**Media Max (5%)**	**Plateau Finale**
Continuo Ottimale ($\lambda = 0.99995$)	-781.22	-782.07	-783.47
Continuo Veloce ($\lambda = 0.999$)	-846.62	-846.62	-849.71
Epoche Ottimale ($\lambda = 0.78$)	-785.03	-785.05	-786.23
Epoche Moderato ($\lambda = 0.85$)	-776.13	-777.76	-779.27
Epoche Lento ($\lambda = 0.90$)	**-774.79**	**-775.13**	**-776.29**

Tabella 6.1: Analisi statistica delle dinamiche di raffreddamento (Log-Fitness) per il modello a trigrammi con $T_{fin} = 0.5$.

I dati statistici riportati in tabella rappresentano i migliori risultati estratti su un campione di 5 esecuzioni indipendenti (restarts). I valori confermano in modo evidente la superiorità complessiva del raffreddamento ad epoche lento.

6.2.2 Confronto tra Add-k-Smoothing e Interpolazione Lineare

Una fase cruciale nella costruzione dell'algoritmo (Appendice C) riguarda la definizione della probabilità smussata, necessaria per mitigare l'impatto di eventuali errori ortografici presenti nel testo sull'economia statistica del modello. Il metodo di Laplace (o *add-k smoothing* con $k = 1$) evita che agli N-grammi assenti nel corpus di addestramento venga assegnata probabilità zero. Questo approccio impedisce di scartare a priori combinazioni corrette ma statisticamente non rilevanti o semanticamente sbagliate. L'approccio basato sull'interpolazione lineare, pur avendo il medesimo obiettivo, si differenzia per l'assegnazione di pesi specifici ai diversi N-grammi. Trattandosi di un cifrario omofonico, la strategia adottata è quella di discostarsi il più possibile dalle frequenze dei singoli unigrammi, privilegiando il peso associato ai bigrammi e, nello specifico di questo studio, ai trigrammi. Attraverso un approccio empirico (*trial and error*), sono state eseguite diverse prove per valutare l'impatto dei pesi assegnati a ciascun N-gramma.

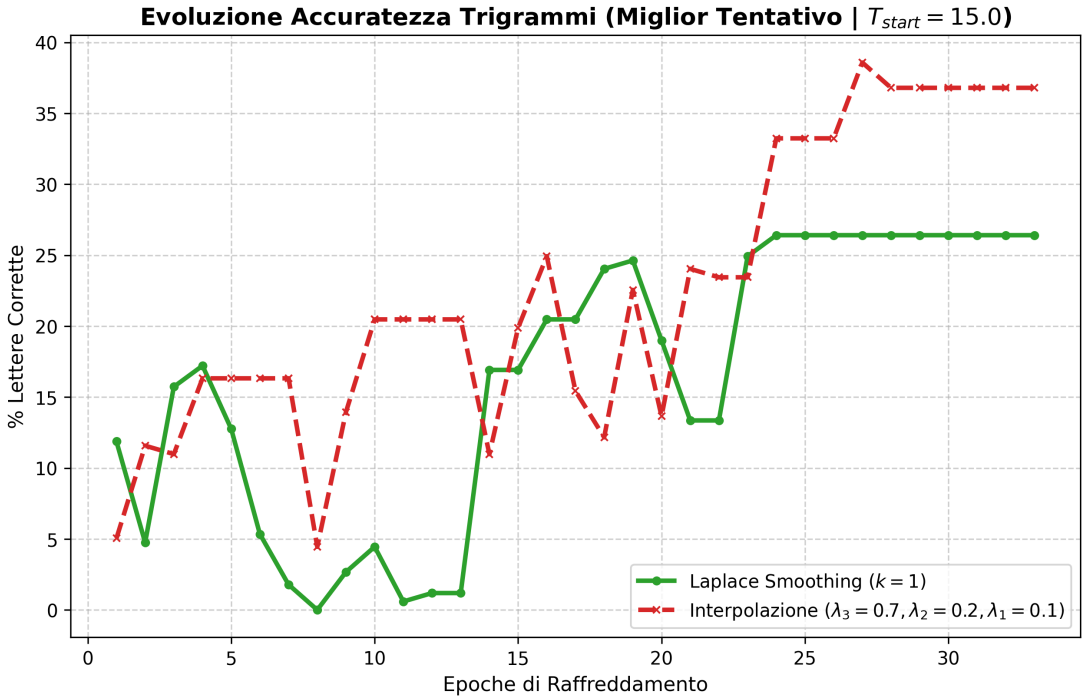


Figura 6.2: Confronto tra Metodo di Laplace e Interpolazione Lineare

Per valutare l'efficacia di entrambi i sistemi, il grafico illustra l'andamento dell'accuratezza relativa alla decrittazione ottenuta. Si può osservare come inizialmente l'interpolazione lineare restituisca un'accuratezza superiore, successivamente, dopo 15 epoche, il metodo di Laplace ha un notevole incremento raggiungento una buona accuratezza. Dopo 25 epoche i due metodi si distaccano , rendendo evidente la superiorità del metodo d'Interpolazione Lineare che raggiunge il 46% d'accuratezza contro il 26% finale raggiunto dal metodo di Laplace.

Pesi $(\lambda_3, \lambda_2, \lambda_1)$	**Laplace Smoothing**	**Interpolazione Lineare**
0.8, 0.1, 0.1	28.72%	52.23%
0.7, 0.2, 0.1	40.36%	42.73%
0.6, 0.3, 0.1	28.49%	19.88%

Tabella 6.2: Confronto dell'accuratezza tra Laplace Smoothing e interpolazione lineare al variare dei parametri λ.

Tenendo sempre in considerazione il fatto che si sta lavorando con un metodo stocastico, i risultati ottenuti su 5 *restart* permettono di affermare che l'interpolazione lineare, con il peso massimo assegnato ai trigrammi, risulta essere l'approccio più performante in termini di accuratezza, superando i risultati ottenuti con lo *smoothing* di Laplace.

6.2.3 Impatto dei Random Restarts

Come discusso in precedenza, il comportamento non deterministico del modello rende necessario ripetere la ricerca della chiave partendo da configurazioni iniziali differenti. Si introducono pertanto i *restart* (riavvii multipli), con l'obiettivo di valutarne l'impatto sul processo e di determinare il numero ottimale di esecuzioni per massimizzare i risultati.

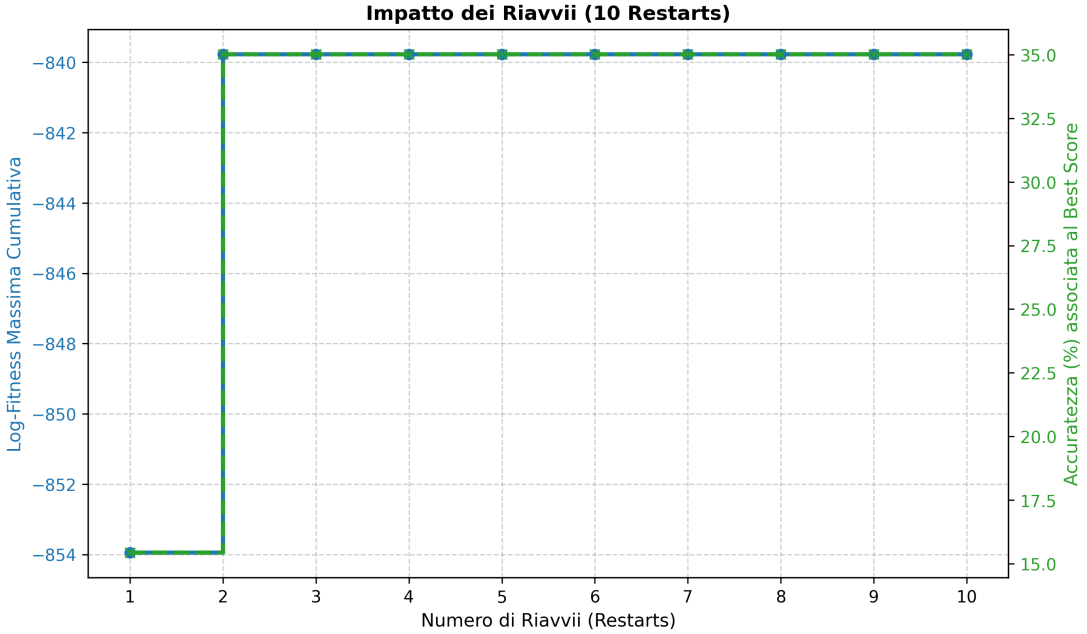


Figura 6.3: Impatto dei restart sull'accuratezza dell'algoritmo

Dal grafico si evince come il primo riavvio produca un risultato del tutto insoddisfacente, sia in termini di punteggio della funzione *fitness* sia di accuratezza della decrittazione. Il secondo *restart*, al contrario, registra un picco di accuratezza del 36%. Considerando che l'accuratezza media del processo si attesta proprio attorno al 36,45% e che, a seguito di questo picco, la funzione massima cumulativa si stabilizza, si potrebbe dedurre che due soli *restart* siano sufficienti per ottimizzare il sistema. Tuttavia, un'analisi empirica più approfondita eseguita tramite l'algoritmo (Appendice D) ha dimostrato che impostare 5 *restart* rappresenta il compromesso ideale tra la massimizzazione dell'accuratezza e il costo computazionale richiesto per iterare l'operazione.

6.3 Collasso dello Spazio delle Chiavi nei trigrammi tramite Metodo Crib

L'integrazione del metodo Crib trasforma il modello in un sistema ibrido, rivelandosi l'approccio ideale per portare a termine la decrittazione e massimizzarne l'accuratezza. L'output generato dal codice (Appendice E) restituisce i seguenti

risultati:

Testo originale :

```
IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
 THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINT
 ABOUTMEIAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEI
 TWILLSENDMETOPARADISEALLTHESOONERBECAUSEIN
OWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVERYO
 NEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHE
 YAREAFRAIDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWT
 HATMYNEWLIFEWILLBEANEASYONEINPARADISEDEATH
```

Simboli unici nel crittogramma: 46

Modello linguistico a Trigrammi:

— AVVIO TENTATIVO 1 DI 10 —
Risultato parziale:
IHOLEYOUAREHATINGLITNISSUNINTRYINGTICATCHRETHATTAS... (Score: -781.25)
— AVVIO TENTATIVO 2 DI 10 —
Risultato parziale:
IHOLEYOUAREHATINGLITNISSUNINTRYINGTICATCHRETHATTAS... (Score: -781.25)
— AVVIO TENTATIVO 3 DI 10 —
Risultato parziale:
IHOLEYOUAREHATINGLITNISSUNINTRYINGTICATCHRETHATTAS... (Score: -781.25)
— AVVIO TENTATIVO 4 DI 10 —
Risultato parziale:
IHOLEYOUAREHATINGLITNISSUNINTRYINGTICATCHRETHATTAS... (Score: -781.25)
— AVVIO TENTATIVO 5 DI 10 —
Risultato parziale:
IHOLEYOUAREHATINGLITNISSUNINTRYINGTICATCHRETHATTAS... (Score: -781.25)

RISULTATO FINALE DELLA CATENA MCMC (Migliore dei tentativi):

OBIETTIVO :

```
IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
 THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINT
```

```
ABOUTMEIAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEI
TWILLSENDMETOPARADISEALLTHESOONERBECAUSEIN
OWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVERYO
NEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHE
YAREAFRAIDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWT
HATMYNEWLIFEWILLBEANEASYONEINPARADISEDEATH
```

OTTENUTO :

```
IHOLEYOUAREHATINGLITNISSUNINTRYINGTICATCHRE
 THATTASNTREONTHETTSHITTHICHBRENGSULALIENT
 ABOUTREEARNITASRAITISTHEGANCHARBERBECAUSEI
 TTILLNENTRETILARATENEALLTHESIONERBECAUSSIN
OTHATESNOUGHNLATENTITORDSORRETHEREANESTERYI
 NEELSEHASNOTHENGTHENTHEYREACHLARATINESITHE
 YARSASRAITISTEATHEARNITASRAETBECAUSSIDNOTT
 HATRYNETLESSTILLBEANEANYONEENLARATINETEATH
```

ACCURATEZZA: 73.29%

Si osserva un notevole incremento prestazionale: il passaggio da un'accuratezza media del 36,45% al 73,29% conferma empiricamente l'efficacia di questa strategia. Nonostante il risultato sia da considerarsi ottimo, il testo ottenuto non risulta ancora perfettamente leggibile. Per superare la soglia del 90% di accuratezza e giungere ad una decrittazione quasi completa, è necessario far evolvere il modello linguistico. Il salto dai 3-grammi ai 4-grammi (quadrigrammi) fornirà al sistema il vincolo statistico addizionale richiesto per portare a termine l'operazione in modo ottimale. Si procede pertanto con lo sviluppo e l'analisi di una nuova architettura algoritmica.

6.4 Evoluzione ai 4-grammi

Il codice relativo all'implementazione dei quadrigrammi (Appendice F) rappresenta la naturale evoluzione del modello a trigrammi. Il passaggio a $N = 4$ ha richiesto modifiche strutturali significative, a partire da un nuovo addestramento del corpus testuale. A causa dell'esponenziale dilatazione dello spazio statistico, è emersa la necessità di gestire specificatamente i tetragrammi mai osservati nel corpus di addestramento. A tal fine, si è introdotta una *Floor Penalty*: anziché adottare uno *smoothing* di Laplace standard, si è optato per un approccio ibrido che assegna a ogni 4-gramma sconosciuto una probabilità fittizia invariabile. Tale penalità logaritmica è sufficientemente severa da punire le sequenze non valide, ma limitata in modo da non generare asintoti negativi che porterebbero a un collasso matematico e termodinamico.

Questo metodo accentua la rigidità del paesaggio energetico, aumentando la probabilità che il sistema rimanga intrappolato in massimi locali (specialmente in presenza di errori ortografici nel crittogramma), ma garantisce una maggiore

precisione globale. Diventa pertanto fondamentale affiancare all'analisi termodinamica uno studio approfondito sul numero di *restarts* necessari per superare le barriere energetiche locali, oltre a valutare empiricamente se questo modello ibrido a *Floor Penalty* si associ meglio all'*add-k smoothing* o all'Interpolazione Lineare. L'intero processo culminerà con l'applicazione del metodo *Crib* per massimizzare l'accuratezza definitiva.

Testo originale:

```
IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
 THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINT
 ABOUTMEIAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEI
 TWILLSENDMETOPARADISEALLTHESOONERBECAUSEIN
OWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVERYO
 NEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHE
 YAREAFRAIDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWT
 HATMYNEWLIFEWILLBEANEASYONEINPARADISEDEATH
```

Simboli unici nel crittogramma: 50

Modello linguistico a Tetragrammi (con Floor Penalty):

```
--- AVVIO TENTATIVO 1 DI 5 ---
Risultato parziale:
SANDSTINCOSIEUSAREISTIOONESEMOTTORETHETHASSFICTSET... (Score: -3477.54)

--- AVVIO TENTATIVO 2 DI 5 ---
Risultato parziale:
IHOSEEOUEREHATINGLOTSOFFUNINTREENGTOCATCHSETHETTAS... (Score: -3279.50)

--- AVVIO TENTATIVO 3 DI 5 ---
Risultato parziale:
GINSTORSTSTREAGALFRECREESAGANSORALITHESHINTERTSEEC... (Score: -3425.98)

--- AVVIO TENTATIVO 4 DI 5 ---
Risultato parziale:
IHOWSTORERSHESINGLOTSOFFRNINTRTENGTOTATTHISTHETTES... (Score: -3327.83)

--- AVVIO TENTATIVO 5 DI 5 ---
Risultato parziale:
RNHATSPITATARER0FOPLEPSSIEREEASESFEREDIENSTHATIERE... (Score: -3492.25)
```

RISULTATO FINALE DELLA CATENA MCMC (Migliore dei tentativi):

OBIETTIVO:

```
IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
```

THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINT
ABOUTMEIAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEI
TWILLSENDMETOPARADISEALLTHESOONERBECAUSEIN
OWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVERYO
NEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHE
YAREAFRAIDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWT
HATMYNEWLIFEWILLBEANEASYONEINPARADISEDEATH

OTTENUTO:

IHOSEEQUEREHATINGLOTSOFFUNINTREENGTOCATCHSE
THETTASNTSTONTHETTSHOTTHICHBRANGSUSASOINT
ABOUTSTIASNOTEFREEDOFTHEGASCHASBERBECAUSEI
TTILLSENDSETOSEREDISEALLTHESOONERBECAUSEEN
OTHETEENOUGHSLATESTOTOROFORSETHEREASEETEREO
NEELSEHASNOTHINGTHENTHEEREECHSAREDASESOTHE
EAREEFREEDOFDEATHEASNOTAFREADBECAUSTIONOTT
HETSENETLEFETILLBEANEASEONEENSAREDASEDEATH

ACCURATEZZA: 79.82%

Il passaggio dal modello a trigrammi a quello a quadrigrammi evidenzia un notevole incremento prestazionale. Su un campione di 5 ripetizioni, l'accuratezza media si attesta attorno al 68.7%. La quantità di frammenti di senso compiuto estratti correttamente risulta significativamente maggiore, ponendo basi estremamente solide per un'agile e produttiva applicazione del metodo *Crib*.

6.4.1 Analisi Termodinamica: Dinamiche di Raffreddamento per i quadrigrammi

Analogamente a quanto eseguito per i trigrammi, si è proceduto con il medesimo studio termodinamico sui raffreddamenti per valutarne l'evoluzione con i quadrigrammi (Appendice G). Dal grafico seguente si evince un netto cambiamento delle dinamiche di convergenza: in questo scenario, il raffreddamento ad epoche moderato risulta essere il più performante, raggiungendo un ottimo massimo locale già dopo 100 000 iterazioni. Il raffreddamento ad epoche lento mantiene invece una marcata mobilità anche oltre le 200 000 iterazioni, indicando un potenziale margine di crescita qualora si aumentasse il numero complessivo di iterazioni. Il raffreddamento continuo ottimale continua a restituire risultati di rilievo: sebbene il sistema decada termicamente in fretta, la delicatezza del raffreddamento gli consente di assestarsi costantemente su ottimi valori. Infine, il raffreddamento continuo veloce si conferma del tutto inadeguato, bloccandosi in modo quasi istantaneo su un massimo locale sub-ottimale.

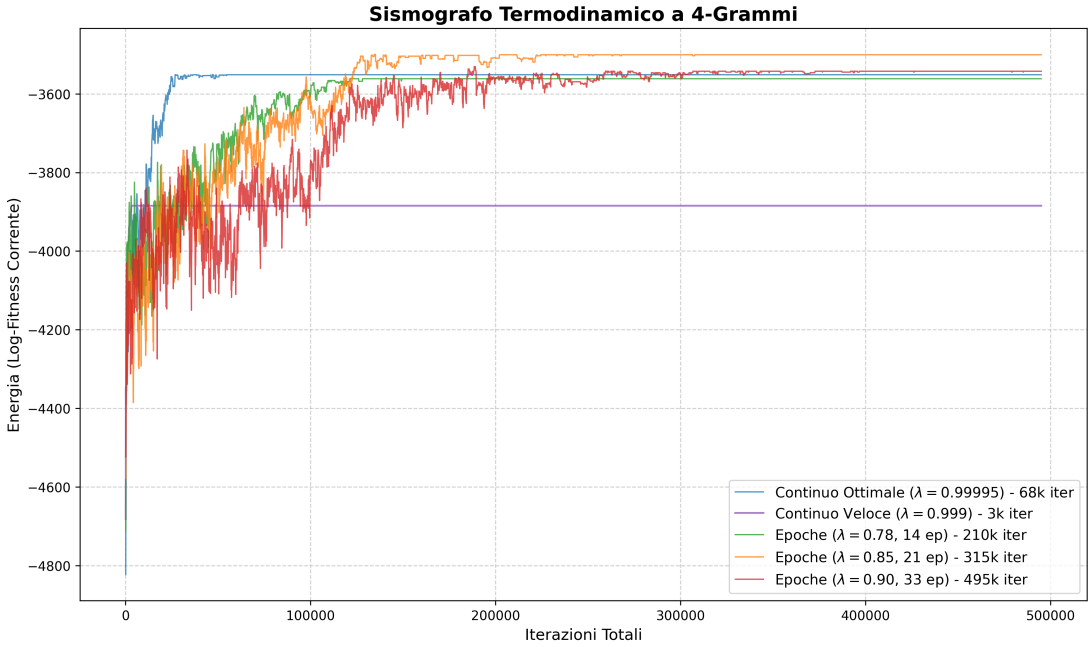


Figura 6.4: Sismografo termodinamico a 4-grammi con applicazione del Floor Penalty. Confronto tra raffreddamento continuo e a epoche con $T_{fin} = 0.5$.

Come per l'analisi a trigrammi, la tabella sottostante riporta lo studio dei massimi assoluti e delle medie dei massimi registrati durante i test. Coerentemente con il tracciato visivo, i risultati migliori si ottengono impiegando il raffreddamento ad epoche con *cooling-rate* pari a 0.85. È doveroso precisare che la scala dei punteggi (*Log-Fitness*) differisce drasticamente da quella osservata per i trigrammi: tali valori dipendono intrinsecamente dall'ordine dell'N-gramma scelto e, soprattutto, dall'impatto severo della *Floor Penalty*. Si osserva infine che optare per un decadimento più lento ($\lambda = 0.90$) non comprometterebbe in modo significativo l'accuratezza finale raggiunta.

Configurazione	**Max Assoluto**	**Media Max (5%)**	**Plateau Finale**
Continuo Ottimale ($\lambda = 0.99995$)	-3551.36	-3551.36	-3551.36
Continuo Veloce ($\lambda = 0.999$)	-3884.65	-3884.65	-3884.65
Epoche ($\lambda = 0.78$, 14 ep)	-3561.50	-3561.50	-3561.50
Epoche ($\lambda = 0.85$, 21 ep)	**-3499.50**	**-3500.51**	**-3500.66**
Epoche ($\lambda = 0.90$, 33 ep)	-3530.36	-3541.96	-3542.49

Tabella 6.3: Analisi Termodinamica MCMC a 4-Grammi con Floor Penalty ($T_{fin} = 0.5$)

6.4.2 Il Paradosso dello Smoothing: Add-k vs Interpolazione Lineare

Come discusso in precedenza, l'introduzione della *Floor Penalty* nei modelli a 4-grammi presenta delle criticità. Invece di associare una probabilità fittizia e

invariabile alle sequenze sconosciute, un'alternativa teorica consiste nell'impiegare l'Interpolazione Lineare, la quale, assegnando pesi differenziati ai vari N-grammi, possiede il potenziale per ottimizzare le prestazioni complessive del sistema.

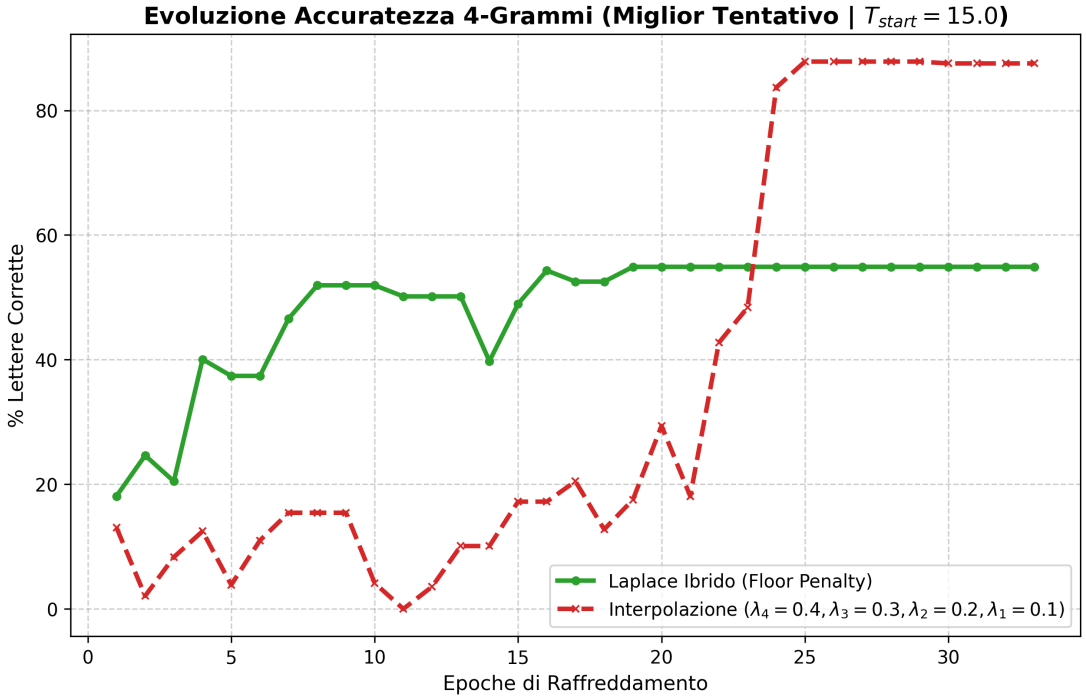


Figura 6.5: Evoluzione dell'accuratezza a 4-grammi. Confronto tra Laplace Ibrido (con applicazione del Floor Penalty) e Interpolazione Lineare ($\lambda_4 = 0.4, \lambda_3 = 0.3, \lambda_2 = 0.2, \lambda_1 = 0.1$).

Il grafico dimostra come l'interpolazione lineare raggiunga un'accuratezza decisamente superiore rispetto al metodo di Laplace Ibrido. Attraverso una distribuzione dei pesi che privilegia i quadrigrammi e i trigrammi, riducendo al minimo l'impatto degli unigrammi, l'accuratezza supera l'85% dopo 25 epoche. Al fine di determinare la configurazione ottimale dei parametri λ, si è proceduto con l'analisi quantitativa riportata nella seguente tabella:

Configurazione Pesi ($\lambda_4, \lambda_3, \lambda_2, \lambda_1$)	**Laplace Ibrido**	**Interpolazione Lineare**
Pesi: 0.5, 0.2, 0.2, 0.1	30.86%	**88.72%**
Pesi: 0.5, 0.3, 0.1, 0.1	**79.82%**	23.74%
Pesi: 0.6, 0.2, 0.1, 0.1	51.98%	**89.02%**
Pesi: 0.7, 0.2, 0.05, 0.05	47.48%	**93.18%**

Tabella 6.4: Accuratezza massima raggiunta a 4-grammi: confronto tra Laplace Ibrido e Interpolazione Lineare al variare dei parametri λ.

La tabella evidenzia l'accuratezza massima registrata su un campione di 5 *restarts*, confrontando direttamente i due approcci (Appendice H). Tenendo sempre in considerazione che una significativa varianza stocastica nell'accuratezza è un

fenomeno assolutamente fisiologico quando si opera con le catene di Markov, i dati confermano empiricamente che la strategia migliore consiste nel massimizzare il peso dei quadrigrammi, riducendo al minimo l'incidenza di unigrammi e bigrammi, e assegnando un peso intermedio ai trigrammi.

6.4.3 Impatto dei Random Restarts

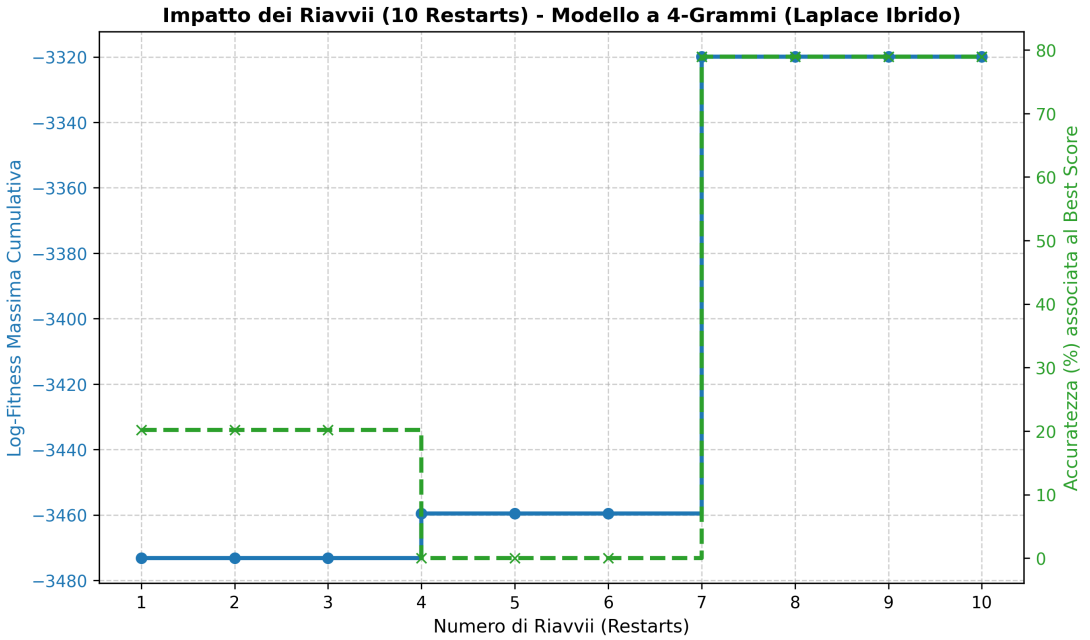


Figura 6.6: Impatto dei riavvii multipli (10 restarts) sulla decrittazione omofonica. Il grafico a doppio asse illustra l'evoluzione della Log-Fitness massima cumulativa (in blu) e il conseguente balzo dell'accuratezza associata (in verde) utilizzando il modello a 4-grammi con Laplace Ibrido.

Il grafico evidenzia come un elevato numero di *restart* determini un significativo incremento dell'accuratezza. Data l'intrinseca imprevedibilità delle catene di Markov e l'impatto della *Floor Penalty* – la quale assegna punteggi estremamente penalizzanti alle configurazioni spurie per forzare l'inserimento di sole parole sensate – risulta indispensabile aumentare le iterazioni di riavvio per ottimizzare la decrittazione complessiva. L'introduzione di questi vincoli, necessari per operare efficacemente con i 4-grammi, modella un paesaggio energetico molto più rigido: è questa la ragione analitica per cui l'architettura a quadrigrammi richiede un numero di *restart* superiore rispetto all'analisi a trigrammi, al fine di eludere i numerosi minimi locali.

6.4.4 Crib 4-grammi

Nonostante l'elevata rigidità topologica del paesaggio energetico, indotta dall'impiego dei tetragrammi con *Floor Penalty*, un'esecuzione esplorativa priva di

ancoraggi (*blind decryption*) supportata da riavvii multipli ha consentito di raggiungere un picco di accuratezza dell'84.57%. Dall'analisi del testo sub-ottimale così generato è stato possibile isolare molteplici frammenti di senso compiuto (Appendice J).

Tali frammenti (*IHOPE, LOTSOFFUN, BRINGSUP, ABOUT, BECAUSE, ALLTHESOONERBECAUSE, ENOUGHS, EVERYONEELSEHASNOTHING, REACH, LIFE, ONEIN*) sono stati fissati nel motore MCMC sotto forma di *crib* vincolanti, permettendo di "congelare" 36 dei 46 simboli crittografici univoci presenti nel testo. Questa drastica riduzione della dimensionalità dello spazio delle chiavi ha guidato efficacemente la catena di Markov verso la convergenza finale, innalzando l'accuratezza al 90.80%.

Metrica	**Decrittazione Cieca**	**Decrittazione con Ancoraggio (Crib)**
Simboli crittografici fissati	0 su 46	36 su 46
Tentativi (Restarts)	5	3
Miglior Log-Fitness	-3310.20	-3348.43
Accuratezza Finale	**84.57%**	**90.80%**

Tabella 6.5: Confronto delle prestazioni MCMC prima e dopo l'applicazione dei crib estratti dal primo output.

Di seguito si riporta il confronto testuale tra i vari passaggi. I blocchi sono stati impaginati per facilitarne la lettura, evidenziando in rosso i frammenti di testo e le parole correttamente decifrate.

1. Testo Obiettivo (Plaintext):

IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME
THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME
IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO
PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES
TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY
REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID
BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN
PARADISEDEATH

2. Output Decrittazione senza Crib (84.57%):

IHOPEEOUAREHATINGLOTSOFFUNINTREINGTOCATCHRE
THATWASNTREENTHETTSHOWTHECHBRINGSUPAPOENTABOUTRE
IARNETAFRAITOFTHEGASCHARBERBECAUSEITTILLSENTRETO
PARATISEALLTHESOONERBECAUSEINOTHATEENOUGHSLATES
TOTOREFORRETHEREASEETEREONEELSEHASNOTHINGWHENTHEE
REACHPARATISESETHEEAREAFRAITOFTEATHEARNOTAFRAIT
BECAUSEEENOTTHATRENETLIFETILLBEANEASYONEIN
PARATISETEATH

3. Output Decrittazione con Crib (90.80%):

IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHSE
THATTASNTSEONTHETVSHOTTHICHBRINGSUPAPOINTABOUTSE
IASNOTAFRAITOFTHEGASCHASBERBECAUSEITTILLSENTSETO
PARATISEALLTHESOONERBECAUSEINOTHAVEENOUGHSLAVES
TOTORSFORSETHEREASEEVERYONEELSEHASNOTHINGWHENTHEY
REACHPARATISESOTHEYAREAFRAITOFTEATHIASNOTAFRAIT
BECAUSEISNOTTHATSYNETLIFETILLBEANEASYONEIN
PARATISETEATH

Conclusioni

Il presente lavoro di tesi ha esplorato e confermato l'efficacia dei metodi stocastici, in particolare dell'algoritmo Markov Chain Monte Carlo (MCMC) e del Simulated Annealing, applicati alla crittanalisi dei cifrari omofonici storici, prendendo come riferimento applicativo lo studio del crittogramma Z_{340}. Partendo dai fondamenti della Teoria dell'Informazione di Shannon, è stato possibile tradurre il problema della decrittazione in un problema di ottimizzazione combinatoria e minimizzazione dell'entropia.

L'analisi sperimentale ha dimostrato come la potenza di elaborazione sia insufficiente se non affiancata da una calibrazione probabilistica e statistica. Il passaggio da un modello linguistico a 3-grammi ad uno a 4-grammi ha evidenziato la necessità di affrontare il problema della frequenza zero (zero-frequency problem) ricercando soluzioni funzionali al contesto. L'introduzione della Floor Penalty, pur rivelandosi superiore in termini di capacità predittiva rispetto allo smoothing di Laplace o all'interpolazione lineare, ha generato un paesaggio energetico estremamente rigido e caratterizzato da un elevato numero di picchi e minimi locali (rugged energy landscape). L'algoritmo ha superato questo ostacolo geometrico dimostrando sperimentalmente la necessità di due parametri fondamentali: un cooling-schedule a epoche lento (che previene il congelamento termico prematuro nei massimi locali) e l'impiego massiccio di restarts multipli per aggirare il rumore stocastico creatosi nella catena di Markov.

Tuttavia, il traguardo sperimentale più significativo di questa analisi risiede nella dimostrazione dei limiti dell' esplorazione stocastica "alla cieca" e nella conseguente formalizzazione di un algoritmo ibrido. Sebbene l'MCMC riesca a far emergere frammenti testuali statisticamente e semanticamente validi, l'eccesso di gradi di libertà tipico della cifratura omofonica lo espone al rischio del collasso entropico. La soluzione definitiva si è concretizzata nell'integrazione del metodo Crib. Iniettando vincoli deterministici (il "congelamento di determinate parole") all'interno del motore stocastico, il sistema ibrido forza il collasso dello spazio delle chiavi, riducendolo e abbattendo l'equivocazione residua, spingendo la catena di Markov verso la convergenza globale esatta con un'accuratezza superiore al 90%.

L'efficacia di questo algoritmo ibrido – che unisce l'esplorazione probabilistica ad ampio spettro con lo sfruttamento deterministico dei minimi locali – supera l'ambito puramente crittanalitico. La logica dell'algoritmo sviluppato in questa tesi trova naturale applicazione in molteplici campi della scienza computazionale caratterizzati da spazi di ricerca vasti e fortemente vincolati. Tra questi spiccano la biologia computazionale (per l'allineamento stocastico di sequenze genomiche e il protein folding), l'ottimizzazione combinatoria industriale (problemi di routing

e scheduling su larga scala), e il Natural Language Processing (per il recupero di informazioni in testi corrotti o fortemente rumorosi).

In conclusione, l'interazione tra termodinamica stocastica e vincoli informazionali non si limita a fornire la chiave di lettura per enigmi storici apparentemente inviolabili, ma si conferma un approccio matematico e computazionale estremamente versatile, capace di estrarre ordine e significato dal caos in sistemi complessi ad alta entropia.

Appendice A

Codice dell'Algoritmo MCMC e Simulated Annealing Base

Il seguente algoritmo riporta l'implementazione completa in Python del modello linguistico a trigrammi, la funzione fitness è basata sull'entropia di Shannon e l'algoritmo Metropolis-Hastings con Simulated Annealing per la decrittazione del cifrario omofonico.

```
import gensim.downloader as api
import random
import math
import string
import collections
from collections import defaultdict

# ============================================================
# FASE 1: MODELLO LINGUISTICO A TRIGRAMMI
# ============================================================
def train_language_model(k_smoothing=1.0):
    print("Caricamento del corpus 'text8' in corso...")
    dataset = api.load("text8")

    trigram_counts = defaultdict(int)
    bigram_counts = defaultdict(int)
    alphabet = set(string.ascii_uppercase)

    prev1 = None
    prev2 = None
    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    if prev1 is not None and prev2 is not None:
                        trigram = prev2 + prev1 + char
                        bigram = prev2 + prev1
                        trigram_counts[trigram] += 1
                        bigram_counts[bigram] += 1
                    prev2 = prev1
                    prev1 = char
```

```
33
34      V = 26
35      log_probs = {}
36      for a in string.ascii_uppercase:
37          for b in string.ascii_uppercase:
38              for c in string.ascii_uppercase:
39                  prob = (trigram_counts[a+b+c] + k_smoothing) / (
                        bigram_counts[a+b] + k_smoothing * V)
40                  log_probs[a+b+c] = math.log(prob)
41
42      print("Modello linguistico a Trigrammi pronto\n")
43      return log_probs
44
45  # ============================================================
46  # FASE 2: FUNZIONE FITNESS CON ENTROPIA
47  # ============================================================
48  def calculate_fitness(text, log_probs):
49      score = 0.0
50      for i in range(len(text) - 2):
51          trigram = text[i:i+3]
52          if trigram in log_probs:
53              score += log_probs[trigram]
54          else:
55              score -= 20.0
56
57      # Calcolo dell'Entropia di Shannon per evitare la ridondanza
            dei trigrammi
58      counts = collections.Counter(text)
59      entropy = 0.0
60      for count in counts.values():
61          p = count / len(text)
62          entropy -= p * math.log(p)
63
64      if entropy < 2.5:
65          score -= 5000.0
66
67      return score
68
69  # ============================================================
70  # FASE 3: CIFRARIO OMOFONICO
71  # ============================================================
72  def encrypt_homophonic(plaintext):
73      #symbol_string = "

            "
74      symbol_string = "Inserisci qui i tuoi simboli"
75      pool = list(symbol_string)
76      random.shuffle(pool)
77
78      extra_homophones = {
79          'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
80          'S': 1, 'R': 1, 'H': 1
81      }
82
83      cipher_dict = {}
84      for char in string.ascii_uppercase:
```

```
85          num_symbols = 1 + extra_homophones.get(char, 0)
86          cipher_dict[char] = [pool.pop() for _ in range(
                num_symbols)]
87
88      ciphertext = ""
89      for char in plaintext:
90          if char in cipher_dict:
91              ciphertext += random.choice(cipher_dict[char])
92
93      return ciphertext, cipher_dict
94
95  def apply_key(ciphertext, key):
96      return ''.join(key.get(c, c) for c in ciphertext)
97
98  # ============================================================
99  # FASE 4: METROPOLIS-HASTINGS E SIMULATED ANNEALING
100 # ============================================================
101 def decrypt_mcmc_sa_epoch(ciphertext, log_probs, T_start=15.0,
        T_fin=0.5, cooling_rate=0.90, iter_per_epoch=15000):
102     epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
            log(cooling_rate))
103
104     unique_symbols = list(set(ciphertext))
105     alphabet = list(string.ascii_uppercase)
106
107     current_key = {sym: random.choice(alphabet) for sym in
            unique_symbols}
108     current_text = apply_key(ciphertext, current_key)
109     current_score = calculate_fitness(current_text, log_probs)
110
111     best_text = current_text
112     best_score = current_score
113
114     T = T_start
115     history = []
116     global_iter = 0
117
118     for epoch in range(epochs_needed):
119         for _ in range(iter_per_epoch):
120             sym = random.choice(unique_symbols)
121             old_val = current_key[sym]
122             new_val = random.choice(alphabet)
123
124             if old_val == new_val:
125                 continue
126
127             current_key[sym] = new_val
128             new_text = apply_key(ciphertext, current_key)
129             new_score = calculate_fitness(new_text, log_probs)
130
131             delta_S = new_score - current_score
132
133             if delta_S > 0:
134                 alpha = 1.0
135             else:
136                 if T < 1e-10:
```

```
                    alpha = 0.0
                else:
                    alpha = math.exp(delta_S / T)

            if random.random() < alpha:
                current_score = new_score
                if current_score > best_score:
                    best_score = current_score
                    best_text = new_text
            else:
                current_key[sym] = old_val

            global_iter += 1
            if global_iter % 1000 == 0:
                history.append(current_score)

        T *= cooling_rate

    return best_text, best_score, history, epochs_needed

# ============================================================
# ESECUZIONE
# ============================================================
if __name__ == "__main__":
    plaintext = (
        "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME"
        "THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME"
        "IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO"
        "PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES"
        "TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY"
        "REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID"
        "BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN"
        "PARADISEDEATH"
    )

    ciphertext, real_key = encrypt_homophonic(plaintext)
    log_probs = train_language_model(k_smoothing=1.0)

    restarts = 5
    global_best_score = float('-inf')
    global_best_text = ""

    T_INIZIALE = 15.0
    T_FINALE = 0.5
    LAMBDA_COOLING = 0.90
    ITER_EPOCA = 15000

    for run in range(restarts):
        text, score, _, epoche_usate = decrypt_mcmc_sa_epoch(
            ciphertext, log_probs, T_start=T_INIZIALE, T_fin=
                T_FINALE,
            cooling_rate=LAMBDA_COOLING, iter_per_epoch=
                ITER_EPOCA
        )
        if score > global_best_score:
            global_best_score = score
```

```
191             global_best_text = text
192
193     matches = sum(1 for a, b in zip(plaintext, global_best_text)
            if a == b)
194     accuracy = (matches / len(plaintext)) * 100
195     print(f"\nACCURATEZZA FINALE ASSOLUTA: {accuracy:.2f}% delle
            lettere corrette")
```

Appendice B

Analisi del Raffreddamento per trigrammi

In questa appendice è riportato il codice per l'analisi e la visualizzazione del comportamento termodinamico dell'algoritmo al variare dei parametri di raffreddamento.

```
import random
import math
import string
import collections
import matplotlib.pyplot as plt
import numpy as np
import os
import gensim.downloader as api

def train_language_model(n_gram, k_smoothing=1.0):
    dataset = api.load("text8")
    ngram_counts = collections.defaultdict(int)
    prefix_counts = collections.defaultdict(int)
    alphabet = set(string.ascii_uppercase)
    buffer = []

    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    buffer.append(char)
                    if len(buffer) == n_gram:
                        ngram = "".join(buffer)
                        prefix = ngram[:-1]
                        ngram_counts[ngram] += 1
                        prefix_counts[prefix] += 1
                        buffer.pop(0)

    V = 26
    log_probs = {}
    for ngram, count in ngram_counts.items():
        prefix = ngram[:-1]
```

```
34          prob = (count + k_smoothing) / (prefix_counts[prefix] +
                k_smoothing * V)
35          log_probs[ngram] = math.log(prob)
36      return log_probs
37
38  def calculate_fitness(text, log_probs, n_gram):
39      score = 0.0
40      penalty = -15.0 if n_gram >= 4 else -20.0
41      for i in range(len(text) - (n_gram - 1)):
42          chunk = text[i:i+n_gram]
43          score += log_probs.get(chunk, penalty)
44      return score
45
46  def encrypt_homophonic(plaintext):
47      #symbol_string = "
            "
48      symbol_string = "Inserisci qui i tuoi simboli"
49      pool = list(symbol_string)
50      random.shuffle(pool)
51      extra_homophones = {'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, '
            N': 2, 'S': 1, 'R': 1, 'H': 1}
52      cipher_dict = {}
53      for char in string.ascii_uppercase:
54          num_symbols = 1 + extra_homophones.get(char, 0)
55          cipher_dict[char] = [pool.pop() for _ in range(
                num_symbols)]
56      ciphertext = ""
57      for char in plaintext:
58          if char in cipher_dict:
59              ciphertext += random.choice(cipher_dict[char])
60      return ciphertext, cipher_dict
61
62  def apply_key(ciphertext, key):
63      return ''.join(key.get(c, c) for c in ciphertext)
64
65  def mcmc_epoch_dynamic(ciphertext, log_probs, n_gram, T_start,
        T_fin, cooling_rate, iter_per_epoch):
66      epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
            log(cooling_rate))
67      total_iterations = epochs_needed * iter_per_epoch
68      unique_symbols = list(set(ciphertext))
69      alphabet = list(string.ascii_uppercase)
70      current_key = {sym: random.choice(alphabet) for sym in
            unique_symbols}
71      current_score = calculate_fitness(apply_key(ciphertext,
            current_key), log_probs, n_gram)
72      T = T_start
73      history = []
74      global_iter = 0
75      for epoch in range(epochs_needed):
76          for _ in range(iter_per_epoch):
77              sym = random.choice(unique_symbols)
78              old_val = current_key[sym]
79              new_val = random.choice(alphabet)
80              if old_val != new_val:
```

```
                current_key[sym] = new_val
                new_score = calculate_fitness(apply_key(
                    ciphertext, current_key), log_probs, n_gram)
                delta_S = new_score - current_score
                alpha = 1.0 if delta_S > 0 else math.exp(delta_S
                    / T)
                if random.random() < alpha:
                    current_score = new_score
                else:
                    current_key[sym] = old_val
            global_iter += 1
            if global_iter % 100 == 0:
                history.append(current_score)
        T *= cooling_rate
    return history, total_iterations, epochs_needed

def mcmc_continuous_dynamic(ciphertext, log_probs, n_gram,
    T_start, T_fin, cooling_rate):
    total_iterations = math.ceil(math.log(T_fin / T_start) / math
        .log(cooling_rate))
    unique_symbols = list(set(ciphertext))
    alphabet = list(string.ascii_uppercase)
    current_key = {sym: random.choice(alphabet) for sym in
        unique_symbols}
    current_score = calculate_fitness(apply_key(ciphertext,
        current_key), log_probs, n_gram)
    T = T_start
    history = []
    for i in range(total_iterations):
        sym = random.choice(unique_symbols)
        old_val = current_key[sym]
        new_val = random.choice(alphabet)
        if old_val != new_val:
            current_key[sym] = new_val
            new_score = calculate_fitness(apply_key(ciphertext,
                current_key), log_probs, n_gram)
            delta_S = new_score - current_score
            alpha = 1.0 if delta_S > 0 else math.exp(delta_S / T)
            if random.random() < alpha:
                current_score = new_score
            else:
                current_key[sym] = old_val
        T *= cooling_rate
        if (i + 1) % 100 == 0:
            history.append(current_score)
    return history, total_iterations

if __name__ == "__main__":
    plaintext = (
        "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHMETHATWA"
        "SNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTMEIAMNOTAFRAI"
        "DOFTHEGASCHAMBERBECAUSEITWILLSENDMETOPARADISEALLTHESO"
        "ONERBECAUSEINOWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVE"
        "RYONEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHEYAREAFRA"
        "IDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWTHATMYNEWLIFEWILLBEA"
        "NEASYONEINPARADISEDEATH"
```

```
    )
    ciphertext, _ = encrypt_homophonic(plaintext)

    N_GRAM_SIZE = 4
    T_START = 15.0
    T_FIN = 0.1
    ITER_EP = 15000

    log_probs = train_language_model(n_gram=N_GRAM_SIZE)
    tests_data = []

    h_c1, i_c1 = mcmc_continuous_dynamic(ciphertext, log_probs,
        N_GRAM_SIZE, T_START, T_FIN, cooling_rate=0.99995)
    tests_data.append({"hist": h_c1, "iters": i_c1, "label": "
        Continuo Ottimale ($\lambda=0.99995$)", "color": "#1f77b4"
        , "lw": 1.0})

    h_e1, i_e1, ep_e1 = mcmc_epoch_dynamic(ciphertext, log_probs,
         N_GRAM_SIZE, T_START, T_FIN, cooling_rate=0.78,
        iter_per_epoch=ITER_EP)
    tests_data.append({"hist": h_e1, "iters": i_e1, "label": "
        Epoche Ottimale ($\lambda=0.78$)", "color": "#2ca02c", "lw
        ": 1.0})

    max_iters = max(test["iters"] for test in tests_data)
    max_len = max_iters // 100
    x_axis = np.arange(100, max_iters + 1, 100)

    plt.figure(figsize=(14, 8))
    for test in tests_data:
        hist = test["hist"]
        if len(hist) < max_len:
            hist.extend([hist[-1]] * (max_len - len(hist)))
        label_with_iters = f"{test['label']} - {test['iters
            ']//1000}k iter"
        plt.plot(x_axis, hist, label=label_with_iters, color=test
            ["color"], linewidth=test["lw"], alpha=0.8)

    plt.title(f"Sismografo Termodinamico a {N_GRAM_SIZE}-Grammi",
        fontsize=15, fontweight='bold')
    plt.xlabel("Iterazioni Totali", fontsize=12)
    plt.ylabel("Energia (Log-Fitness Corrente)", fontsize=12)
    plt.legend(loc='lower right', fontsize=11)
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.savefig('grafico_sismografo.png', dpi=300, bbox_inches='
        tight')
```

Appendice C

Confronto Modelli: Laplace vs Interpolazione Lineare

Codice utilizzato per confrontare le performance dell'algoritmo implementando lo smoothing di Laplace con K=1 e l'interpolazione lineare per il modello linguistico a trigrammi.

```
import gensim.downloader as api
import random
import math
import string
import collections
from collections import defaultdict
import matplotlib.pyplot as plt
import numpy as np
import os

def train_comparative_models_trigrams(l3=0.7, l2=0.2, l1=0.1):
    dataset = api.load("text8")
    uni_counts = defaultdict(int)
    bi_counts = defaultdict(int)
    tri_counts = defaultdict(int)
    alphabet = string.ascii_uppercase

    total_unigrams = 0
    prev1 = prev2 = None

    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    uni_counts[char] += 1
                    total_unigrams += 1
                    if prev1 is not None:
                        bi_counts[prev1+char] += 1
                        if prev2 is not None:
                            tri_counts[prev2+prev1+char] += 1
                    prev2, prev1 = prev1, char

```

```
    V = 26
    log_probs_laplace = {}
    log_probs_interp = {}
    k = 1.0

    for tri, count in tri_counts.items():
        bi_pre = tri[:-1]
        prob_add = (count + k) / (bi_counts[bi_pre] + k * V)
        log_probs_laplace[tri] = math.log(prob_add)

    for a in alphabet:
        for b in alphabet:
            for c in alphabet:
                tri = a + b + c
                bi_pre = a + b
                bi_suf = b + c

                p3 = tri_counts[tri] / bi_counts[bi_pre] if
                    bi_counts[bi_pre] > 0 else 0
                p2 = bi_counts[bi_suf] / uni_counts[b] if
                    uni_counts[b] > 0 else 0
                p1 = uni_counts[c] / total_unigrams

                prob_interp = (l3 * p3) + (l2 * p2) + (l1 * p1)
                if prob_interp > 0:
                    log_probs_interp[tri] = math.log(prob_interp)

    return log_probs_laplace, log_probs_interp

def calculate_fitness(text, log_probs, fallback_penalty):
    score = 0.0
    for i in range(len(text) - 2):
        score += log_probs.get(text[i:i+3], fallback_penalty)
    counts = collections.Counter(text)
    entropy = 0.0
    for count in counts.values():
        p = count / len(text)
        entropy -= p * math.log(p)
    if entropy < 2.5:
        score -= 5000.0
    return score

def encrypt_homophonic(plaintext):
    #symbol_string = "

       "
    symbol_string = "Inserisci qui i tuoi simboli"
    pool = list(symbol_string)
    random.shuffle(pool)
    extra_homophones = {'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, '
       N': 2, 'S': 1, 'R': 1, 'H': 1}
    cipher_dict = {}
    for char in string.ascii_uppercase:
        num_symbols = 1 + extra_homophones.get(char, 0)
        cipher_dict[char] = [pool.pop() for _ in range(
           num_symbols)]
```

```
84      return "".join(random.choice(cipher_dict[c]) for c in
            plaintext)
85
86  def apply_key(ciphertext, key):
87      return ''.join(key.get(c, c) for c in ciphertext)
88
89  def decrypt_mcmc_tracked_epochs(ciphertext, log_probs, plaintext,
        fallback_penalty, T_start=15.0, T_fin=0.5, cooling_rate=0.90,
        iter_per_epoch=15000):
90      epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
            log(cooling_rate))
91      unique_symbols = list(set(ciphertext))
92      alphabet = list(string.ascii_uppercase)
93      current_key = {sym: random.choice(alphabet) for sym in
            unique_symbols}
94      best_text = apply_key(ciphertext, current_key)
95      current_score = calculate_fitness(best_text, log_probs,
            fallback_penalty)
96      best_score = current_score
97      T = T_start
98      history_acc = []
99
100     for epoch in range(epochs_needed):
101         for _ in range(iter_per_epoch):
102             sym = random.choice(unique_symbols)
103             old_val = current_key[sym]
104             new_val = random.choice(alphabet)
105             if old_val == new_val: continue
106             current_key[sym] = new_val
107             new_text = apply_key(ciphertext, current_key)
108             new_score = calculate_fitness(new_text, log_probs,
                    fallback_penalty)
109             delta_S = new_score - current_score
110             alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                    else math.exp(delta_S / T))
111             if random.random() < alpha:
112                 current_score = new_score
113                 if current_score > best_score:
114                     best_score = current_score
115                     best_text = new_text
116             else:
117                 current_key[sym] = old_val
118         T *= cooling_rate
119         matches = sum(1 for a, b in zip(plaintext, best_text) if
                a == b)
120         history_acc.append((matches / len(plaintext)) * 100)
121     return history_acc, epochs_needed
122
123 if __name__ == "__main__":
124     plaintext = (
125         "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME"
126         "THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME"
127         "IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO"
128         "PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES"
129         "TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY"
130         "REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID"
```

```
        "BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN"
        "PARADISEDEATH"
    )

    ciphertext = encrypt_homophonic(plaintext)
    log_probs_laplace, log_probs_interp =
       train_comparative_models_trigrams()

    runs = 5
    T_START, T_FIN, LAMBDA_COOL, ITER_EP = 15.0, 0.5, 0.90, 15000

    results_laplace = []
    for r in range(runs):
        acc, _ = decrypt_mcmc_tracked_epochs(ciphertext,
           log_probs_laplace, plaintext, fallback_penalty=-20.0,
           T_start=T_START, T_fin=T_FIN, cooling_rate=LAMBDA_COOL
           , iter_per_epoch=ITER_EP)
        results_laplace.append(acc)
    best_laplace = results_laplace[np.argmax([res[-1] for res in
       results_laplace])]

    results_interp = []
    for r in range(runs):
        acc, _ = decrypt_mcmc_tracked_epochs(ciphertext,
           log_probs_interp, plaintext, fallback_penalty=-50.0,
           T_start=T_START, T_fin=T_FIN, cooling_rate=LAMBDA_COOL
           , iter_per_epoch=ITER_EP)
        results_interp.append(acc)
    best_interp = results_interp[np.argmax([res[-1] for res in
       results_interp])]

    plt.figure(figsize=(10, 6))
    plt.plot(best_laplace, label="Laplace Smoothing (K=1)", color
       ='blue', linewidth=2)
    plt.plot(best_interp, label="Interpolazione Lineare", color='
       orange', linewidth=2)
    plt.title("Evoluzione Accuratezza a 3-Grammi")
    plt.xlabel("Epoche")
    plt.ylabel("Accuratezza (%)")
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.savefig('grafico_confronto_modelli_3grammi.png', dpi=300,
        bbox_inches='tight')
```

Appendice D

Analisi dei Riavvii Multipli (Restarts)

Questo script implementa i molteplici riavvii della catena di Markov (Restarts) e traccia l'aumento dell'accuratezza in base ai minimi locali evitati.

```
import gensim.downloader as api
import random
import math
import string
import collections
from collections import defaultdict
import matplotlib.pyplot as plt

# ============================================================
# FASE 1: MODELLO LINGUISTICO A TRIGRAMMI
# ============================================================
def train_language_model(k_smoothing=1.0):
    print("Caricamento del corpus 'text8' in corso...")
    dataset = api.load("text8")

    trigram_counts = defaultdict(int)
    bigram_counts = defaultdict(int)
    alphabet = set(string.ascii_uppercase)

    prev1 = None
    prev2 = None
    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    if prev1 is not None and prev2 is not None:
                        trigram = prev2 + prev1 + char
                        bigram = prev2 + prev1
                        trigram_counts[trigram] += 1
                        bigram_counts[bigram] += 1
                    prev2 = prev1
                    prev1 = char

```

```
    V = 26
    log_probs = {}
    for a in string.ascii_uppercase:
        for b in string.ascii_uppercase:
            for c in string.ascii_uppercase:
                prob = (trigram_counts[a+b+c] + k_smoothing) / (
                    bigram_counts[a+b] + k_smoothing * V)
                log_probs[a+b+c] = math.log(prob)

    print("Modello linguistico a Trigrammi pronto\n")
    return log_probs

# ============================================================
# FASE 2: FUNZIONE FITNESS CON ENTROPIA
# ============================================================
def calculate_fitness(text, log_probs):
    score = 0.0
    for i in range(len(text) - 2):
        trigram = text[i:i+3]
        if trigram in log_probs:
            score += log_probs[trigram]
        else:
            score -= 20.0

    # Calcolo dell'Entropia di Shannon per evitare la ridondanza
       dei trigrammi
    counts = collections.Counter(text)
    entropy = 0.0
    for count in counts.values():
        p = count / len(text)
        entropy -= p * math.log(p)

    if entropy < 2.5:
        score -= 5000.0

    return score

# ============================================================
# FASE 3: CIFRARIO OMOFONICO
# ============================================================
def encrypt_homophonic(plaintext):
    #symbol_string = "

       "
    symbol_string = "Inserisci qui i tuoi simboli"
    pool = list(symbol_string)
    random.shuffle(pool)

    extra_homophones = {
        'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
        'S': 1, 'R': 1, 'H': 1
    }

    cipher_dict = {}
    for char in string.ascii_uppercase:
        num_symbols = 1 + extra_homophones.get(char, 0)
```

```
87          cipher_dict[char] = [pool.pop() for _ in range(
                num_symbols)]
88
89      ciphertext = ""
90      for char in plaintext:
91          if char in cipher_dict:
92              ciphertext += random.choice(cipher_dict[char])
93
94      return ciphertext, cipher_dict
95
96  def apply_key(ciphertext, key):
97      return ''.join(key.get(c, c) for c in ciphertext)
98
99  # ============================================================
100 # FASE 4: METROPOLIS-HASTINGS E SIMULATED ANNEALING
101 # ============================================================
102 def decrypt_mcmc_sa_epoch(ciphertext, log_probs, T_start=15.0,
        T_fin=0.5, cooling_rate=0.90, iter_per_epoch=15000):
103     epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
            log(cooling_rate))
104     unique_symbols = list(set(ciphertext))
105     alphabet = list(string.ascii_uppercase)
106     current_key = {sym: random.choice(alphabet) for sym in
            unique_symbols}
107     current_text = apply_key(ciphertext, current_key)
108     current_score = calculate_fitness(current_text, log_probs)
109     best_text = current_text
110     best_score = current_score
111     T = T_start
112     history = []
113
114     for epoch in range(epochs_needed):
115         for _ in range(iter_per_epoch):
116             sym = random.choice(unique_symbols)
117             old_val = current_key[sym]
118             new_val = random.choice(alphabet)
119             if old_val == new_val: continue
120             current_key[sym] = new_val
121             new_text = apply_key(ciphertext, current_key)
122             new_score = calculate_fitness(new_text, log_probs)
123             delta_S = new_score - current_score
124             alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                    else math.exp(delta_S / T))
125             if random.random() < alpha:
126                 current_score = new_score
127                 if current_score > best_score:
128                     best_score = current_score
129                     best_text = new_text
130             else:
131                 current_key[sym] = old_val
132         T *= cooling_rate
133     return best_text, best_score, history, epochs_needed
134
135 if __name__ == "__main__":
136     plaintext = (
137         "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHMETHATWA"
```

```
        "SNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTMEIAMNOTAFRAI"
        "DOFTHEGASCHAMBERBECAUSEITWILLSENDMETOPARADISEALLTHESO"
        "ONERBECAUSEINOWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVE"
        "RYONEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHEYAREAFRA"
        "IDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWTHATMYNEWLIFEWILLBEA"
        "NEASYONEINPARADISEDEATH"
    )
    ciphertext, real_key = encrypt_homophonic(plaintext)
    log_probs = train_language_model(k_smoothing=1.0)

    restarts = 10
    global_best_score = float('-inf')
    global_best_text = ""
    current_best_acc = 0.0

    x_axis = list(range(1, restarts + 1))
    y_fitness_cumulativa = []
    y_accuratezza_correlata = []

    for run in range(restarts):
        text, score, _, epoche_usate = decrypt_mcmc_sa_epoch(
            ciphertext, log_probs, T_start=15.0, T_fin=0.5,
               cooling_rate=0.90, iter_per_epoch=15000
        )
        matches_run = sum(1 for a, b in zip(plaintext, text) if a
            == b)
        acc_run = (matches_run / len(plaintext)) * 100

        if score > global_best_score:
            global_best_score = score
            global_best_text = text
            current_best_acc = acc_run

        y_fitness_cumulativa.append(global_best_score)
        y_accuratezza_correlata.append(current_best_acc)

    fig, ax1 = plt.subplots(figsize=(10, 6))
    color1 = 'tab:blue'
    ax1.set_xlabel('Numero di Riavvii (Restarts)')
    ax1.set_ylabel('Log-Fitness Massima Cumulativa', color=color1
       )
    ax1.step(x_axis, y_fitness_cumulativa, where='post', color=
       color1, linewidth=2.5, marker='o')
    ax1.tick_params(axis='y', labelcolor=color1)
    ax1.set_xticks(x_axis)
    ax1.grid(True, linestyle='--', alpha=0.6)

    ax2 = ax1.twinx()
    color2 = 'tab:green'
    ax2.set_ylabel('Accuratezza (%) associata al Best Score',
       color=color2)
    ax2.step(x_axis, y_accuratezza_correlata, where='post', color
       =color2, linewidth=2.5, linestyle='--', marker='x')
    ax2.tick_params(axis='y', labelcolor=color2)
    plt.title("Impatto dei Riavvii (10 Restarts)")
    plt.savefig('grafico_esperimento_restarts.png', dpi=300,
```

```
        bbox_inches='tight')
```

Appendice E

Decrittazione con Metodo Crib

Codice utilizzato per fissare specifiche parole trovate utilizzando l'appendice A, riduce drasticamente lo spazio di ricerca delle chiavi, garantendo la convergenza all'ottimo globale.

```
import gensim.downloader as api
import random
import math
import string
import collections
from collections import defaultdict

# ============================================================
# FASE 1: MODELLO LINGUISTICO A TRIGRAMMI
# ============================================================
def train_language_model(k_smoothing=1.0):
    print("Caricamento del corpus 'text8' in corso...")
    dataset = api.load("text8")

    trigram_counts = defaultdict(int)
    bigram_counts = defaultdict(int)
    alphabet = set(string.ascii_uppercase)

    prev1 = None
    prev2 = None
    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    if prev1 is not None and prev2 is not None:
                        trigram = prev2 + prev1 + char
                        bigram = prev2 + prev1
                        trigram_counts[trigram] += 1
                        bigram_counts[bigram] += 1
                    prev2 = prev1
                    prev1 = char

    V = 26
    log_probs = {}
```

```
    for a in string.ascii_uppercase:
        for b in string.ascii_uppercase:
            for c in string.ascii_uppercase:
                prob = (trigram_counts[a+b+c] + k_smoothing) / (
                    bigram_counts[a+b] + k_smoothing * V)
                log_probs[a+b+c] = math.log(prob)

    print("Modello linguistico a Trigrammi pronto\n")
    return log_probs

# ============================================================
# FASE 2: FUNZIONE FITNESS CON ENTROPIA
# ============================================================
def calculate_fitness(text, log_probs):
    score = 0.0
    for i in range(len(text) - 2):
        trigram = text[i:i+3]
        if trigram in log_probs:
            score += log_probs[trigram]
        else:
            score -= 20.0

    # Calcolo dell'Entropia di Shannon per evitare la ridondanza
        dei trigrammi
    counts = collections.Counter(text)
    entropy = 0.0
    for count in counts.values():
        p = count / len(text)
        entropy -= p * math.log(p)

    if entropy < 2.5:
        score -= 5000.0

    return score

# ============================================================
# FASE 3: CIFRARIO OMOFONICO
# ============================================================
def encrypt_homophonic(plaintext):
    #symbol_string = "

        "
    symbol_string = "Inserisci qui i tuoi simboli"
    pool = list(symbol_string)
    random.shuffle(pool)

    extra_homophones = {
        'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
        'S': 1, 'R': 1, 'H': 1
    }

    cipher_dict = {}
    for char in string.ascii_uppercase:
        num_symbols = 1 + extra_homophones.get(char, 0)
        cipher_dict[char] = [pool.pop() for _ in range(
            num_symbols)]
```

```
87
88      ciphertext = ""
89      for char in plaintext:
90          if char in cipher_dict:
91              ciphertext += random.choice(cipher_dict[char])
92
93      return ciphertext, cipher_dict
94
95  def apply_key(ciphertext, key):
96      return ''.join(key.get(c, c) for c in ciphertext)
97
98  # ============================================================
99  # FASE 4: METROPOLIS-HASTINGS E SIMULATED ANNEALING
100 # ============================================================
101 def decrypt_mcmc_sa_crib_epoch(ciphertext, log_probs, T_start
        =15.0, T_fin=0.5, cooling_rate=0.90, iter_per_epoch=15000,
        locked_keys=None):
102     if locked_keys is None:
103         locked_keys = {}
104
105     epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
            log(cooling_rate))
106     unique_symbols = list(set(ciphertext))
107     alphabet = list(string.ascii_uppercase)
108
109     current_key = {}
110     for sym in unique_symbols:
111         if sym in locked_keys:
112             current_key[sym] = locked_keys[sym]
113         else:
114             current_key[sym] = random.choice(alphabet)
115
116     unlocked_symbols = [s for s in unique_symbols if s not in
            locked_keys]
117     current_text = apply_key(ciphertext, current_key)
118     current_score = calculate_fitness(current_text, log_probs)
119
120     best_text = current_text
121     best_score = current_score
122     T = T_start
123
124     if not unlocked_symbols:
125         return best_text, best_score
126
127     for epoch in range(epochs_needed):
128         for _ in range(iter_per_epoch):
129             sym = random.choice(unlocked_symbols)
130             old_val = current_key[sym]
131             new_val = random.choice(alphabet)
132
133             if old_val == new_val: continue
134
135             current_key[sym] = new_val
136             new_text = apply_key(ciphertext, current_key)
137             new_score = calculate_fitness(new_text, log_probs)
138
```

```
            delta_S = new_score - current_score
            alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                else math.exp(delta_S / T))

            if random.random() < alpha:
                current_score = new_score
                if current_score > best_score:
                    best_score = current_score
                    best_text = new_text
            else:
                current_key[sym] = old_val
        T *= cooling_rate

    return best_text, best_score

if __name__ == "__main__":
    plaintext = (
        "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHMETHATWA"
        "SNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTMEIAMNOTAFRAI"
        "DOFTHEGASCHAMBERBECAUSEITWILLSENDMETOPARADISEALLTHESO"
        "ONERBECAUSEINOWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVE"
        "RYONEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHEYAREAFRA"
        "IDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWTHATMYNEWLIFEWILLBEA"
        "NEASYONEINPARADISEDEATH"
    )

    ciphertext, real_key = encrypt_homophonic(plaintext)

    fixed_cribs = {
        "I": 0,
        "CATCH": 36,
        "THAT": 43,
        "THE": 56,
        "ABOUT": 84,
        "BECAUSE": 118,
        "THEYREACH": 231
    }

    locked_keys = {}
    for word, start_idx in fixed_cribs.items():
        for i in range(len(word)):
            simbolo_cifrato = ciphertext[start_idx + i]
            lettera_chiaro = word[i]
            locked_keys[simbolo_cifrato] = lettera_chiaro

    log_probs = train_language_model(k_smoothing=1.0)
    restarts = 5
    global_best_score = float('-inf')
    global_best_text = ""

    for run in range(restarts):
        text, score = decrypt_mcmc_sa_crib_epoch(
            ciphertext, log_probs, T_start=15.0, T_fin=0.5,
                cooling_rate=0.90,
            iter_per_epoch=15000, locked_keys=locked_keys
        )
```

```
193             if score > global_best_score:
194                 global_best_score = score
195                 global_best_text = text
196 
197         matches_final = sum(1 for a, b in zip(plaintext,
                global_best_text) if a == b)
198         accuracy_final = (matches_final / len(plaintext)) * 100
199         print(f"\nACCURATEZZA FINALE (CRIB): {accuracy_final:.2f}%
                delle lettere corrette")
```

Appendice F

Codice Sorgente: Decrittazione a 4-grammi

In questa appendice viene riportato il codice Python completo sviluppato per l'addestramento del modello linguistico a tetragrammi e per l'esecuzione della catena MCMC tramite *Simulated Annealing* a epoche dinamiche.

```
import gensim.downloader as api
import random
import math
import string
import collections

# ==========================================================
# FASE 1: MODELLO LINGUISTICO A 4-GRAMMI (PROBABILIT  ASSOLUTA E FLOOR)
#==========================================================
def train_language_model(n_gram=4):
    print(f"Caricamento del corpus 'text8' per modello a {n_gram}-grammi...")
    dataset = api.load("text8")

    ngram_counts = collections.defaultdict(int)
    alphabet = set(string.ascii_uppercase)

    total_ngrams = 0
    buffer = []

    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    buffer.append(char)
                    if len(buffer) == n_gram:
                        ngram = "".join(buffer)
                        ngram_counts[ngram] += 1
                        total_ngrams += 1
                        buffer.pop(0)
```

```
31
32      log_probs = {}
33      for ngram, count in ngram_counts.items():
34          log_probs[ngram] = math.log(count / total_ngrams)
35
36      # Il Floor Penalty per i tetragrammi inventati/non visti.
37      floor_penalty = math.log(0.01 / total_ngrams)
38
39      print(f"Modello pronto. Trovate {len(log_probs)} combinazioni
            uniche.")
40      print(f"Totale {n_gram}-grammi nel corpus: {total_ngrams}")
41      print(f"Floor Penalty matematica calcolata a: {floor_penalty
            :.2f}\n")
42      return log_probs, floor_penalty
43
44  #================================================================
45  # FASE 2: FUNZIONE FITNESS
46  #================================================================
47  def calculate_fitness(text, log_probs, floor_penalty, n_gram=4):
48      score = 0.0
49
50      for i in range(len(text) - (n_gram - 1)):
51          chunk = text[i:i+n_gram]
52          score += log_probs.get(chunk, floor_penalty)
53
54      counts = collections.Counter(text)
55      entropy = 0.0
56      for count in counts.values():
57          p = count / len(text)
58          entropy -= p * math.log(p)
59
60      if entropy < 2.5:
61          score -= 5000.0
62
63      return score
64
65  #================================================================
66  # FASE 3: CIFRARIO OMOFONICO
67  #================================================================
68  def encrypt_homophonic(plaintext):
69      #symbol_string = "
            "
70      symbol_string = "inserisci qui i tuoi simboli"
71      pool = list(symbol_string)
72      random.shuffle(pool)
73
74      extra_homophones = {
75          'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
76          'S': 1, 'R': 1, 'H': 1
77      }
78
79      cipher_dict = {}
80      for char in string.ascii_uppercase:
81          num_symbols = 1 + extra_homophones.get(char, 0)
82          cipher_dict[char] = [pool.pop() for _ in range(
```

```
            num_symbols)]

    ciphertext = ""
    for char in plaintext:
        if char in cipher_dict:
            ciphertext += random.choice(cipher_dict[char])

    return ciphertext, cipher_dict

def apply_key(ciphertext, key):
    return ''.join(key.get(c, c) for c in ciphertext)

#================================================================
# FASE 4: SIMULATED ANNEALING A EPOCHE (DINAMICO)
#================================================================
def decrypt_mcmc_sa_epoch(ciphertext, log_probs, floor_penalty,
    n_gram=4, T_start=15.0, T_fin=0.5, cooling_rate=0.90,
    iter_per_epoch=15000):

    # Calcolo dinamico basato su matematica termodinamica
    epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
        log(cooling_rate))

    unique_symbols = list(set(ciphertext))
    alphabet = list(string.ascii_uppercase)

    current_key = {sym: random.choice(alphabet) for sym in
        unique_symbols}
    current_text = apply_key(ciphertext, current_key)
    current_score = calculate_fitness(current_text, log_probs,
        floor_penalty, n_gram)

    best_text = current_text
    best_score = current_score

    T = T_start
    history = []

    for epoch in range(epochs_needed):
        for i in range(iter_per_epoch):
            sym = random.choice(unique_symbols)
            old_val = current_key[sym]
            new_val = random.choice(alphabet)

            if old_val == new_val:
                continue

            current_key[sym] = new_val
            new_text = apply_key(ciphertext, current_key)
            new_score = calculate_fitness(new_text, log_probs,
                floor_penalty, n_gram)

            delta_S = new_score - current_score

            if delta_S > 0:
                alpha = 1.0
```

```
132             else:
133                 alpha = 0.0 if T < 1e-10 else math.exp(delta_S /
                        T)
134
135             if random.random() < alpha:
136                 current_score = new_score
137                 if current_score > best_score:
138                     best_score = current_score
139                     best_text = new_text
140             else:
141                 current_key[sym] = old_val
142
143             if (i + 1 + epoch * iter_per_epoch) % 1000 == 0:
144                 history.append(current_score)
145
146         T *= cooling_rate
147
148     return best_text, best_score, history, epochs_needed
149
150 #=============================================================
151 # ESECUZIONE
152 #=============================================================
153 if __name__ == "__main__":
154     plaintext = (
155         "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME"
156         "THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME"
157         "IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO"
158         "PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES"
159         "TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY"
160         "REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID"
161         "BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN"
162         "PARADISEDEATH"
163     )
164
165     print("="*75)
166     print("TEST CRITTANALISI: TETRAGRAMMI CON FLOOR PENALTY (N=4)
            ")
167     print("="*75)
168
169     ciphertext, real_key = encrypt_homophonic(plaintext)
170
171     n_gram_size = 4
172     log_probs, floor_penalty = train_language_model(n_gram=
            n_gram_size)
173
174     restarts = 5
175     global_best_score = float('-inf')
176     global_best_text = ""
177
178     T_INIZIALE = 15.0
179     T_FINALE = 0.5
180     LAMBDA_COOLING = 0.90
181     ITER_EPOCA = 15000
182
183     for run in range(restarts):
184         print(f"\n--- AVVIO TENTATIVO {run + 1} DI {restarts} ---
```

```
            ")
185
186         text, score, hist, epoche_usate = decrypt_mcmc_sa_epoch(
187             ciphertext, log_probs, floor_penalty, n_gram=
                n_gram_size,
188             T_start=T_INIZIALE, T_fin=T_FINALE, cooling_rate=
                LAMBDA_COOLING, iter_per_epoch=ITER_EPOCA
189         )
190
191         matches_partial = sum(1 for a, b in zip(plaintext, text)
                if a == b)
192         accuracy_partial = (matches_partial / len(plaintext)) *
                100
193
194         print(f"Epoche calcolate: {epoche_usate} | Iterazioni
                Totali: {epoche_usate * ITER_EPOCA}")
195         print(f"Risultato: {text[:50]}... | Score: {score:.2f} |
                Accuratezza: {accuracy_partial:.2f}%")
196
197         if score > global_best_score:
198             global_best_score = score
199             global_best_text = text
200
201     print("\n" + "="*75)
202     print("RISULTATO FINALE DELLA CATENA MCMC (Migliore dei 5
            tentativi):")
203     print(f"OBIETTIVO : {plaintext}")
204     print(f"OTTENUTO  : {global_best_text}")
205
206     matches_final = sum(1 for a, b in zip(plaintext,
            global_best_text) if a == b)
207     accuracy_final = (matches_final / len(plaintext)) * 100
208     print(f"\nACCURATEZZA FINALE (MIGLIORE): {accuracy_final:.2f
            }% delle lettere corrette")
```

Appendice G

Analisi del Raffreddamento per quadrigrammi

Il seguente script implementa il motore MCMC per confrontare l'efficienza del decadimento continuo della temperatura rispetto a un approccio termodinamico a epoche fisse, generando un grafico a sismografo.

```
import random
import math
import string
import collections
import matplotlib.pyplot as plt
import numpy as np
import os
import gensim.downloader as api

def train_language_model(n_gram):
    print(f"Caricamento del corpus 'text8' per modello a {n_gram}-grammi...", flush=True)
    dataset = api.load("text8")
    ngram_counts = collections.defaultdict(int)
    alphabet = set(string.ascii_uppercase)
    buffer = []
    total_ngrams = 0

    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    buffer.append(char)
                    if len(buffer) == n_gram:
                        ngram = "".join(buffer)
                        ngram_counts[ngram] += 1
                        total_ngrams += 1
                        buffer.pop(0)

    log_probs = {}
    for ngram, count in ngram_counts.items():
        log_probs[ngram] = math.log(count / total_ngrams)
```

```
33
34      floor_penalty = math.log(0.01 / total_ngrams)
35
36      print(f"Modello pronto. Trovate {len(log_probs)} combinazioni
            .\n", flush=True)
37      return log_probs, floor_penalty
38
39  def calculate_fitness(text, log_probs, floor_penalty, n_gram):
40      score = 0.0
41      for i in range(len(text) - (n_gram - 1)):
42          chunk = text[i:i+n_gram]
43          score += log_probs.get(chunk, floor_penalty)
44
45      counts = collections.Counter(text)
46      entropy = 0.0
47      for count in counts.values():
48          p = count / len(text)
49          entropy -= p * math.log(p)
50
51      if entropy < 2.5:
52          score -= 5000.0
53
54      return score
55
56  def encrypt_homophonic(plaintext):
57      #symbol_string = "
            "
58      symbol_string = "Inserisci qui i tuoi simboli"
59      pool = list(symbol_string)
60      random.shuffle(pool)
61
62      extra_homophones = {
63          'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
64          'S': 1, 'R': 1, 'H': 1
65      }
66
67      cipher_dict = {}
68      for char in string.ascii_uppercase:
69          num_symbols = 1 + extra_homophones.get(char, 0)
70          cipher_dict[char] = [pool.pop() for _ in range(
                num_symbols)]
71
72      ciphertext = ""
73      for char in plaintext:
74          if char in cipher_dict:
75              ciphertext += random.choice(cipher_dict[char])
76
77      return ciphertext, cipher_dict
78
79  def apply_key(ciphertext, key):
80      return ''.join(key.get(c, c) for c in ciphertext)
81
82  def mcmc_epoch_dynamic(ciphertext, log_probs, floor_penalty,
        n_gram, T_start, T_fin, cooling_rate, iter_per_epoch):
83      epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
```

```
        log(cooling_rate))
    total_iterations = epochs_needed * iter_per_epoch

    unique_symbols = list(set(ciphertext))
    alphabet = list(string.ascii_uppercase)
    current_key = {sym: random.choice(alphabet) for sym in
        unique_symbols}
    current_score = calculate_fitness(apply_key(ciphertext,
        current_key), log_probs, floor_penalty, n_gram)

    T = T_start
    history = []
    global_iter = 0

    for epoch in range(epochs_needed):
        for _ in range(iter_per_epoch):
            sym = random.choice(unique_symbols)
            old_val = current_key[sym]
            new_val = random.choice(alphabet)

            if old_val != new_val:
                current_key[sym] = new_val
                new_score = calculate_fitness(apply_key(
                    ciphertext, current_key), log_probs,
                    floor_penalty, n_gram)
                delta_S = new_score - current_score
                alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                    else math.exp(delta_S / T))

                if random.random() < alpha:
                    current_score = new_score
                else:
                    current_key[sym] = old_val

            global_iter += 1
            if global_iter % 100 == 0:
                history.append(current_score)

        T *= cooling_rate

    return history, total_iterations, epochs_needed

def mcmc_continuous_dynamic(ciphertext, log_probs, floor_penalty,
    n_gram, T_start, T_fin, cooling_rate):
    total_iterations = math.ceil(math.log(T_fin / T_start) / math
        .log(cooling_rate))

    unique_symbols = list(set(ciphertext))
    alphabet = list(string.ascii_uppercase)
    current_key = {sym: random.choice(alphabet) for sym in
        unique_symbols}
    current_score = calculate_fitness(apply_key(ciphertext,
        current_key), log_probs, floor_penalty, n_gram)

    T = T_start
    history = []
```

```
130
131     for i in range(total_iterations):
132         sym = random.choice(unique_symbols)
133         old_val = current_key[sym]
134         new_val = random.choice(alphabet)
135
136         if old_val != new_val:
137             current_key[sym] = new_val
138             new_score = calculate_fitness(apply_key(ciphertext,
                    current_key), log_probs, floor_penalty, n_gram)
139             delta_S = new_score - current_score
140             alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                    else math.exp(delta_S / T))
141
142             if random.random() < alpha:
143                 current_score = new_score
144             else:
145                 current_key[sym] = old_val
146
147         T *= cooling_rate
148
149         if (i + 1) % 100 == 0:
150             history.append(current_score)
151
152     return history, total_iterations
153
154 if __name__ == "__main__":
155     plaintext = (
156         "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHMETHATWA"
157         "SNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTMEIAMNOTAFRAI"
158         "DOFTHEGASCHAMBERBECAUSEITWILLSENDMETOPARADISEALLTHESO"
159         "ONERBECAUSEINOWHAVEENOUGHSLAVESTOWORKFORMEWHEREASEEVE"
160         "RYONEELSEHASNOTHINGWHENTHEYREACHPARADISESOTHEYAREAFRA"
161         "IDOFDEATHIAMNOTAFRAIDBECAUSEIKNOWTHATMYNEWLIFEWILLBEA"
162         "NEASYONEINPARADISEDEATH"
163     )
164     ciphertext, _ = encrypt_homophonic(plaintext)
165
166     N_GRAM_SIZE = 4
167     T_START = 15.0
168     T_FIN = 0.5
169     ITER_EP = 15000
170
171     log_probs, floor_penalty = train_language_model(n_gram=
            N_GRAM_SIZE)
172     tests_data = []
173
174     print(f"--- Esecuzione TEST CONTINUO 1 (0.99995) in corso..."
            , flush=True)
175     h_c1, i_c1 = mcmc_continuous_dynamic(ciphertext, log_probs,
            floor_penalty, N_GRAM_SIZE, T_START, T_FIN, cooling_rate
            =0.99995)
176     tests_data.append({"hist": h_c1, "iters": i_c1, "label": "
            Continuo Ottimale ($\lambda=0.99995$)", "color": "#1f77b4"
            , "lw": 1.0})
177
```

```
    print(f"--- Esecuzione TEST CONTINUO 2 (0.999) in corso...", flush=True)
    h_c2, i_c2 = mcmc_continuous_dynamic(ciphertext, log_probs, floor_penalty, N_GRAM_SIZE, T_START, T_FIN, cooling_rate=0.999)
    tests_data.append({"hist": h_c2, "iters": i_c2, "label": "Continuo Veloce ($\lambda=0.999$)", "color": "#9467bd", "lw": 1.5})

    print(f"--- Esecuzione TEST EPOCHE 1 (0.78) in corso...", flush=True)
    h_e1, i_e1, ep_e1 = mcmc_epoch_dynamic(ciphertext, log_probs, floor_penalty, N_GRAM_SIZE, T_START, T_FIN, cooling_rate=0.78, iter_per_epoch=ITER_EP)
    tests_data.append({"hist": h_e1, "iters": i_e1, "label": f"Epoche ($\lambda=0.78$, {ep_e1} ep)", "color": "#2ca02c", "lw": 1.0})

    print(f"--- Esecuzione TEST EPOCHE 2 (0.85) in corso...", flush=True)
    h_e2, i_e2, ep_e2 = mcmc_epoch_dynamic(ciphertext, log_probs, floor_penalty, N_GRAM_SIZE, T_START, T_FIN, cooling_rate=0.85, iter_per_epoch=ITER_EP)
    tests_data.append({"hist": h_e2, "iters": i_e2, "label": f"Epoche ($\lambda=0.85$, {ep_e2} ep)", "color": "#ff7f0e", "lw": 1.0})

    print(f"--- Esecuzione TEST EPOCHE 3 (0.90) in corso...", flush=True)
    h_e3, i_e3, ep_e3 = mcmc_epoch_dynamic(ciphertext, log_probs, floor_penalty, N_GRAM_SIZE, T_START, T_FIN, cooling_rate=0.90, iter_per_epoch=ITER_EP)
    tests_data.append({"hist": h_e3, "iters": i_e3, "label": f"Epoche ($\lambda=0.90$, {ep_e3} ep)", "color": "#d62728", "lw": 1.0})

    print("\n" + "=" * 50, flush=True)
    print("RISULTATI TERMODINAMICI", flush=True)
    print("=" * 50, flush=True)

    for test in tests_data:
        raw_hist = test["hist"]
        max_abs = max(raw_hist)

        top_5_count = max(1, len(raw_hist) // 20)
        top_scores = sorted(raw_hist, reverse=True)[:top_5_count]
        media_massimi = sum(top_scores) / len(top_scores)

        tail_count = max(1, len(raw_hist) // 10)
        media_plateau = sum(raw_hist[-tail_count:]) / tail_count

        print(f"{test['label']}:")
        print(f"  Max Assoluto     : {max_abs:.2f}")
        print(f"  Media Max (5%)   : {media_massimi:.2f}")
        print(f"  Plateau Finale   : {media_plateau:.2f}\n", flush=True)
```

```
213
214     max_iters = max(test["iters"] for test in tests_data)
215     max_len = max_iters // 100
216     x_axis = np.arange(100, max_iters + 1, 100)
217
218     plt.figure(figsize=(14, 8))
219     for test in tests_data:
220         hist = test["hist"]
221         if len(hist) < max_len:
222             hist.extend([hist[-1]] * (max_len - len(hist)))
223         label_with_iters = f"{test['label']} | {test['iters
            ']//1000}k iter"
224         plt.plot(x_axis, hist, label=label_with_iters, color=test
            ["color"], linewidth=test["lw"], alpha=0.8)
225
226     plt.title(f"Sismografo Termodinamico a {N_GRAM_SIZE}-Grammi",
        fontsize=15, fontweight='bold')
227     plt.xlabel("Iterazioni Totali", fontsize=12)
228     plt.ylabel("Energia (Log-Fitness Corrente)", fontsize=12)
229     plt.legend(loc='lower right', fontsize=11)
230     plt.grid(True, linestyle='--', alpha=0.6)
231
232     save_path = os.path.expanduser('~/grafico_sismografo.png')
233     plt.savefig(save_path, dpi=300, bbox_inches='tight')
234     print(f"Grafico salvato in: {save_path}", flush=True)
```

Appendice H

Confronto tra Add-k-Smoothing Ibrido e Interpolazione Lineare per 4-Grammi

Questo algoritmo definisce e addestra due modelli linguistici a 4-grammi distinti: il Laplace Ibrido (con applicazione rigorosa del *Floor Penalty*) e l'Interpolazione Lineare a 4 livelli, misurandone le performance comparative in termini di accuratezza (multi-restart) per la decrittazione omofonica.

```
import random
import math
import string
import collections
from collections import defaultdict
import matplotlib.pyplot as plt
import numpy as np
import os
import gensim.downloader as api

# ============================================================
# ADDESTRAMENTO DEI DUE MODELLI LINGUISTICI (4-GRAMMI)
# ============================================================
def train_comparative_models_tetragrams(l4=0.4, l3=0.3, l2=0.2, l1=0.1):
    print(f"Caricamento corpus 'text8' per 4-Grammi (Lambda: {l4}, {l3}, {l2}, {l1})...", flush=True)
    dataset = api.load("text8")

    uni_counts = defaultdict(int)
    bi_counts = defaultdict(int)
    tri_counts = defaultdict(int)
    tetra_counts = defaultdict(int)
    alphabet = string.ascii_uppercase

    total_unigrams = 0
    total_tetragrams = 0
    prev1 = prev2 = prev3 = None

```

```
    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    uni_counts[char] += 1
                    total_unigrams += 1
                    if prev1 is not None:
                        bi_counts[prev1+char] += 1
                        if prev2 is not None:
                            tri_counts[prev2+prev1+char] += 1
                            if prev3 is not None:
                                tetra_counts[prev3+prev2+prev1+
                                    char] += 1
                                total_tetragrams += 1
                    prev3, prev2, prev1 = prev2, prev1, char

    print("Calcolo delle probabilit  in corso (generazione di
        456.976 combinazioni per l'Interpolazione)...", flush=True
        )

    # 1. Modello Laplace Ibrido (MLE puro per le combinazioni
        esistenti + Floor Penalty)
    log_probs_hybrid = {}
    for tetra, count in tetra_counts.items():
        log_probs_hybrid[tetra] = math.log(count /
            total_tetragrams)

    floor_penalty = math.log(0.01 / total_tetragrams)

    # 2. Modello Interpolazione Lineare a 4 livelli
    log_probs_interp = {}
    for a in alphabet:
        for b in alphabet:
            for c in alphabet:
                for d in alphabet:
                    tetra = a + b + c + d
                    tri_pre = a + b + c
                    tri_suf = b + c + d
                    bi_pre = b + c
                    bi_suf = c + d

                    p4 = tetra_counts[tetra] / tri_counts[tri_pre
                        ] if tri_counts[tri_pre] > 0 else 0
                    p3 = tri_counts[tri_suf] / bi_counts[bi_pre]
                        if bi_counts[bi_pre] > 0 else 0
                    p2 = bi_counts[bi_suf] / uni_counts[c] if
                        uni_counts[c] > 0 else 0
                    p1 = uni_counts[d] / total_unigrams

                    prob_interp = (l4 * p4) + (l3 * p3) + (l2 *
                        p2) + (l1 * p1)
                    if prob_interp > 0:
                        log_probs_interp[tetra] = math.log(
                            prob_interp)

```

```
    print("Modelli a 4-Grammi pronti\n", flush=True)
    return log_probs_hybrid, floor_penalty, log_probs_interp

# ============================================================
# FUNZIONI DI FITNESS E CIFRARIO
# ============================================================
def calculate_fitness(text, log_probs, fallback_penalty, n_gram=4):
    score = 0.0
    for i in range(len(text) - (n_gram - 1)):
        score += log_probs.get(text[i:i+n_gram], fallback_penalty)

    counts = collections.Counter(text)
    entropy = 0.0
    for count in counts.values():
        p = count / len(text)
        entropy -= p * math.log(p)

    if entropy < 2.5:
        score -= 5000.0
    return score

def encrypt_homophonic(plaintext):
    #symbol_string = "

       "
    symbol_string = "Inserisci qui i tuoi simboli"
    pool = list(symbol_string)
    random.shuffle(pool)

    extra_homophones = {
        'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
        'S': 1, 'R': 1, 'H': 1
    }

    cipher_dict = {}
    for char in string.ascii_uppercase:
        num_symbols = 1 + extra_homophones.get(char, 0)
        cipher_dict[char] = [pool.pop() for _ in range(num_symbols)]

    return "".join(random.choice(cipher_dict[c]) for c in plaintext)

def apply_key(ciphertext, key):
    return ''.join(key.get(c, c) for c in ciphertext)

# ============================================================
# MOTORE MCMC DINAMICO CON TRACCIAMENTO ACCURATEZZA
# ============================================================
def decrypt_mcmc_tracked_epochs(ciphertext, log_probs, plaintext, fallback_penalty, n_gram, T_start=15.0, T_fin=0.5, cooling_rate=0.90, iter_per_epoch=15000):
    epochs_needed = math.ceil(math.log(T_fin / T_start) / math.log(cooling_rate))
```

```
121
122     unique_symbols = list(set(ciphertext))
123     alphabet = list(string.ascii_uppercase)
124     current_key = {sym: random.choice(alphabet) for sym in
            unique_symbols}
125
126     best_text = apply_key(ciphertext, current_key)
127     current_score = calculate_fitness(best_text, log_probs,
            fallback_penalty, n_gram)
128     best_score = current_score
129     T = T_start
130
131     history_acc = []
132
133     for epoch in range(epochs_needed):
134         for _ in range(iter_per_epoch):
135             sym = random.choice(unique_symbols)
136             old_val = current_key[sym]
137             new_val = random.choice(alphabet)
138
139             if old_val == new_val: continue
140
141             current_key[sym] = new_val
142             new_text = apply_key(ciphertext, current_key)
143             new_score = calculate_fitness(new_text, log_probs,
                    fallback_penalty, n_gram)
144
145             delta_S = new_score - current_score
146             alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                    else math.exp(delta_S / T))
147
148             if random.random() < alpha:
149                 current_score = new_score
150                 if current_score > best_score:
151                     best_score = current_score
152                     best_text = new_text
153             else:
154                 current_key[sym] = old_val
155
156         T *= cooling_rate
157         matches = sum(1 for a, b in zip(plaintext, best_text) if
                a == b)
158         history_acc.append((matches / len(plaintext)) * 100)
159
160     return history_acc, epochs_needed
161
162 # ============================================================
163 # ESECUZIONE DEL TEST
164 # ============================================================
165 if __name__ == "__main__":
166     plaintext = (
167         "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME"
168         "THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME"
169         "IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO"
170         "PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES"
171         "TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY"
```

```
        "REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID"
        "BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN"
        "PARADISEDEATH"
    )

    l4, l3, l2, l1 = 0.7, 0.2, 0.05, 0.05
    ciphertext = encrypt_homophonic(plaintext)

    log_probs_hybrid, floor_penalty, log_probs_interp =
        train_comparative_models_tetragrams(l4=l4, l3=l3, l2=l2,
        l1=l1)

    runs = 10
    label_interp = f"Interpolazione ($\lambda_4={l4}, \lambda_3={
        l3}, \lambda_2={l2}, \lambda_1={l1}$)"

    N_GRAM_SIZE = 4
    T_START = 15.0
    T_FIN = 0.5
    LAMBDA_COOL = 0.90
    ITER_EP = 15000

    print(f"\n--- Parametri Fissi: T_start={T_START}, T_fin={
        T_FIN}, Lambda={LAMBDA_COOL}, Iter={ITER_EP} ---", flush=
        True)

    print(f"\n--- Esecuzione Test 1: LAPLACE IBRIDO (con Floor
        Penalty) ---", flush=True)
    results_hybrid = []
    epochs_hybrid = 0
    for r in range(runs):
        acc, ep = decrypt_mcmc_tracked_epochs(ciphertext,
            log_probs_hybrid, plaintext, fallback_penalty=
            floor_penalty, n_gram=N_GRAM_SIZE, T_start=T_START,
            T_fin=T_FIN, cooling_rate=LAMBDA_COOL, iter_per_epoch=
            ITER_EP)
        results_hybrid.append(acc)
        epochs_hybrid = ep

    best_hybrid_idx = np.argmax([res[-1] for res in
        results_hybrid])
    best_hist_acc_hybrid = results_hybrid[best_hybrid_idx]

    print(f"\n--- Esecuzione Test 2: {label_interp.upper()} ---",
         flush=True)
    results_interp = []
    epochs_interp = 0
    for r in range(runs):
        acc, ep = decrypt_mcmc_tracked_epochs(ciphertext,
            log_probs_interp, plaintext, fallback_penalty=-50.0,
            n_gram=N_GRAM_SIZE, T_start=T_START, T_fin=T_FIN,
            cooling_rate=LAMBDA_COOL, iter_per_epoch=ITER_EP)
        results_interp.append(acc)
        epochs_interp = ep

    best_interp_idx = np.argmax([res[-1] for res in
```

```
        results_interp])
213     best_hist_acc_interp = results_interp[best_interp_idx]
214
215     plt.figure(figsize=(10, 6))
216     plt.plot(best_hist_acc_hybrid, label="Laplace Ibrido (Floor
            Penalty)", color='blue', linewidth=2)
217     plt.plot(best_hist_acc_interp, label=label_interp, color='
            orange', linewidth=2)
218     plt.title("Evoluzione Accuratezza a 4-Grammi")
219     plt.xlabel("Epoche")
220     plt.ylabel("Accuratezza (%)")
221     plt.legend()
222     plt.grid(True, linestyle='--', alpha=0.7)
223     plt.savefig('grafico_accuratezza_4grammi_T15.png', dpi=300,
            bbox_inches='tight')
```

Appendice I

Impatto dei Riavvii Multipli (Restarts) a 4-Grammi

Questo script analizza il comportamento della Log-Fitness e dell'accuratezza in risposta all'utilizzo dei riavvii multipli (restarts) per superare i minimi locali generati dal paesaggio energetico difficile del Laplace Ibrido con Floor Penalty.

```
import gensim.downloader as api
import random
import math
import string
import collections
from collections import defaultdict
import matplotlib.pyplot as plt
import os

# ============================================================
# FASE 1: MODELLO LINGUISTICO A 4-GRAMMI CON FLOOR PENALTY
# ============================================================
def train_language_model_4grams():
    print("Caricamento del corpus 'text8' per 4-grammi in corso
        ...", flush=True)
    dataset = api.load("text8")

    tetra_counts = defaultdict(int)
    alphabet = set(string.ascii_uppercase)
    buffer = []
    total_tetragrams = 0

    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    buffer.append(char)
                    if len(buffer) == 4:
                        tetra = "".join(buffer)
                        tetra_counts[tetra] += 1
                        total_tetragrams += 1
                        buffer.pop(0)
```

```
33
34      log_probs = {}
35      for tetra, count in tetra_counts.items():
36          log_probs[tetra] = math.log(count / total_tetragrams)
37
38      floor_penalty = math.log(0.01 / total_tetragrams)
39
40      print(f"Modello linguistico a 4-Grammi pronto. Trovate {len(
            log_probs)} combinazioni.\n", flush=True)
41      return log_probs, floor_penalty
42
43  # ============================================================
44  # FASE 2: FUNZIONE FITNESS A 4-GRAMMI
45  # ============================================================
46  def calculate_fitness(text, log_probs, floor_penalty):
47      score = 0.0
48      for i in range(len(text) - 3):
49          tetra = text[i:i+4]
50          score += log_probs.get(tetra, floor_penalty)
51
52      counts = collections.Counter(text)
53      entropy = 0.0
54      for count in counts.values():
55          p = count / len(text)
56          entropy -= p * math.log(p)
57
58      if entropy < 2.5:
59          score -= 5000.0
60
61      return score
62
63  # ============================================================
64  # FASE 3: CIFRARIO OMOFONICO
65  # ============================================================
66  def encrypt_homophonic(plaintext):
67      #symbol_string = "
            "
68      symbol_string = "Inserisci qui i tuoi simboli"
69      pool = list(symbol_string)
70      random.shuffle(pool)
71
72      extra_homophones = {
73          'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, 'N': 2,
74          'S': 1, 'R': 1, 'H': 1
75      }
76
77      cipher_dict = {}
78      for char in string.ascii_uppercase:
79          num_symbols = 1 + extra_homophones.get(char, 0)
80          cipher_dict[char] = [pool.pop() for _ in range(
                num_symbols)]
81
82      ciphertext = ""
83      for char in plaintext:
84          if char in cipher_dict:
```

```
            ciphertext += random.choice(cipher_dict[char])

    return ciphertext, cipher_dict

def apply_key(ciphertext, key):
    return ''.join(key.get(c, c) for c in ciphertext)

# ============================================================
# FASE 4: METROPOLIS-HASTINGS & SIMULATED ANNEALING
# ============================================================
def decrypt_mcmc_sa_epoch(ciphertext, log_probs, floor_penalty,
    T_start=15.0, T_fin=0.5, cooling_rate=0.90, iter_per_epoch
    =15000):
    epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
        log(cooling_rate))

    unique_symbols = list(set(ciphertext))
    alphabet = list(string.ascii_uppercase)

    current_key = {sym: random.choice(alphabet) for sym in
        unique_symbols}
    current_text = apply_key(ciphertext, current_key)
    current_score = calculate_fitness(current_text, log_probs,
        floor_penalty)

    best_text = current_text
    best_score = current_score

    T = T_start

    for epoch in range(epochs_needed):
        for _ in range(iter_per_epoch):
            sym = random.choice(unique_symbols)
            old_val = current_key[sym]
            new_val = random.choice(alphabet)

            if old_val == new_val:
                continue

            current_key[sym] = new_val
            new_text = apply_key(ciphertext, current_key)
            new_score = calculate_fitness(new_text, log_probs,
                floor_penalty)

            delta_S = new_score - current_score

            if delta_S > 0:
                alpha = 1.0
            else:
                if T < 1e-10:
                    alpha = 0.0
                else:
                    alpha = math.exp(delta_S / T)

            if random.random() < alpha:
                current_score = new_score
```

```
                if current_score > best_score:
                    best_score = current_score
                    best_text = new_text
            else:
                current_key[sym] = old_val

        T *= cooling_rate

    return best_text, best_score, epochs_needed

if __name__ == "__main__":
    plaintext = (
        "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME"
        "THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME"
        "IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO"
        "PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES"
        "TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY"
        "REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID"
        "BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN"
        "PARADISEDEATH"
    )

    ciphertext, real_key = encrypt_homophonic(plaintext)
    log_probs, floor_penalty = train_language_model_4grams()

    restarts = 10
    global_best_score = float('-inf')
    global_best_text = ""
    current_best_acc = 0.0

    T_INIZIALE = 15.0
    T_FINALE = 0.5
    LAMBDA_COOLING = 0.90
    ITER_EPOCA = 15000

    x_axis = list(range(1, restarts + 1))
    y_fitness_cumulativa = []
    y_accuratezza_correlata = []

    for run in range(restarts):
        text, score, epoche_usate = decrypt_mcmc_sa_epoch(
            ciphertext, log_probs, floor_penalty,
            T_start=T_INIZIALE, T_fin=T_FINALE, cooling_rate=
                LAMBDA_COOLING, iter_per_epoch=ITER_EPOCA
        )

        matches_run = sum(1 for a, b in zip(plaintext, text) if a
            == b)
        acc_run = (matches_run / len(plaintext)) * 100

        if score > global_best_score:
            global_best_score = score
            global_best_text = text
            current_best_acc = acc_run

        y_fitness_cumulativa.append(global_best_score)
```

```
189             y_accuratezza_correlata.append(current_best_acc)
190
191     fig, ax1 = plt.subplots(figsize=(10, 6))
192     color1 = 'tab:blue'
193     ax1.set_xlabel('Numero di Riavvii (Restarts)')
194     ax1.set_ylabel('Log-Fitness Massima Cumulativa', color=color1
            )
195     ax1.step(x_axis, y_fitness_cumulativa, where='post', color=
            color1, linewidth=2.5, marker='o')
196     ax1.tick_params(axis='y', labelcolor=color1)
197     ax1.set_xticks(x_axis)
198     ax1.grid(True, linestyle='--', alpha=0.6)
199
200     ax2 = ax1.twinx()
201     color2 = 'tab:green'
202     ax2.set_ylabel('Accuratezza (%) associata al Best Score',
            color=color2)
203     ax2.step(x_axis, y_accuratezza_correlata, where='post', color
            =color2, linewidth=2.5, linestyle='--', marker='x')
204     ax2.tick_params(axis='y', labelcolor=color2)
205     plt.title("Impatto dei Riavvii multipli (10 Restarts) - 4
            Grammi")
206     plt.savefig('grafico_esperimento_restarts_4grammi.png', dpi
            =300, bbox_inches='tight')
```

Appendice J

Metodo Crib per 4-grammi

L'ultimo algoritmo integra il congelamento delle parole ricavate dal codice generale (Appendice G), vincolando termodinamicamente l'algoritmo al fine di restringere lo spazio di ricerca e aggirare i minimi locali.

```
import gensim.downloader as api
import random
import math
import string
import collections
from collections import defaultdict

# ============================================================
# FASE 1: MODELLO LINGUISTICO A 4-GRAMMI
# ============================================================
def train_language_model_4grams():
    print("Caricamento corpus 'text8' per 4-grammi...", flush=
        True)
    dataset = api.load("text8")

    tetra_counts = defaultdict(int)
    alphabet = set(string.ascii_uppercase)
    buffer = []
    total_tetragrams = 0

    for words in dataset:
        for word in words:
            word = word.upper()
            for char in word:
                if char in alphabet:
                    buffer.append(char)
                    if len(buffer) == 4:
                        tetra_counts["".join(buffer)] += 1
                        total_tetragrams += 1
                        buffer.pop(0)

    log_probs = {}
    for tetra, count in tetra_counts.items():
        log_probs[tetra] = math.log(count / total_tetragrams)

```

```
35      floor_penalty = math.log(0.01 / total_tetragrams)
36      return log_probs, floor_penalty
37
38  def calculate_fitness(text, log_probs, floor_penalty):
39      score = 0.0
40      for i in range(len(text) - 3):
41          score += log_probs.get(text[i:i+4], floor_penalty)
42
43      counts = collections.Counter(text)
44      entropy = 0.0
45      for count in counts.values():
46          p = count / len(text)
47          entropy -= p * math.log(p)
48
49      if entropy < 2.5:
50          score -= 5000.0
51      return score
52
53  # ============================================================
54  # FASE 2: CIFRARIO OMOFONICO
55  # ============================================================
56  def encrypt_homophonic(plaintext):
57      #symbol_string = "
            "
58      symbol_string = "Inserisci qui i tuoi simboli"
59      pool = list(symbol_string)
60      random.shuffle(pool)
61
62      extra_homophones = {'E': 6, 'T': 4, 'A': 3, 'O': 3, 'I': 3, '
            N': 2, 'S': 1, 'R': 1, 'H': 1}
63      cipher_dict = {}
64      for char in string.ascii_uppercase:
65          num_symbols = 1 + extra_homophones.get(char, 0)
66          cipher_dict[char] = [pool.pop() for _ in range(
                num_symbols)]
67
68      ciphertext = "".join(random.choice(cipher_dict[c]) for c in
            plaintext)
69      return ciphertext, cipher_dict
70
71  def apply_key(ciphertext, key):
72      return ''.join(key.get(c, c) for c in ciphertext)
73
74  # ============================================================
75  # FASE 3: MOTORE MCMC CON CONGELAMENTO DEI CRIB (PAROLE CORRETTE)
76  # ============================================================
77  def decrypt_mcmc_sa_crib(ciphertext, log_probs, floor_penalty,
        crib_dict=None, T_start=15.0, T_fin=0.5, cooling_rate=0.90,
        iter_per_epoch=15000):
78      epochs_needed = math.ceil(math.log(T_fin / T_start) / math.
            log(cooling_rate))
79      unique_symbols = list(set(ciphertext))
80      alphabet = list(string.ascii_uppercase)
81
82      current_key = {}
```

```
    crib_dict = crib_dict or {}
    locked_symbols = set(crib_dict.keys())
    mutable_symbols = [sym for sym in unique_symbols if sym not
        in locked_symbols]

    for sym in unique_symbols:
        if sym in locked_symbols:
            current_key[sym] = crib_dict[sym]
        else:
            current_key[sym] = random.choice(alphabet)

    current_text = apply_key(ciphertext, current_key)
    current_score = calculate_fitness(current_text, log_probs,
        floor_penalty)

    best_score = current_score
    best_text = current_text
    T = T_start

    for epoch in range(epochs_needed):
        for _ in range(iter_per_epoch):
            if not mutable_symbols:
                break
            sym = random.choice(mutable_symbols)

            old_val = current_key[sym]
            new_val = random.choice(alphabet)
            if old_val == new_val: continue

            current_key[sym] = new_val
            new_text = apply_key(ciphertext, current_key)
            new_score = calculate_fitness(new_text, log_probs,
                floor_penalty)

            delta_S = new_score - current_score
            alpha = 1.0 if delta_S > 0 else (0.0 if T < 1e-10
                else math.exp(delta_S / T))

            if random.random() < alpha:
                current_score = new_score
                if current_score > best_score:
                    best_score = current_score
                    best_text = new_text
            else:
                current_key[sym] = old_val

        T *= cooling_rate

    return best_text, best_score

if __name__ == "__main__":
    plaintext = (
        "IHOPEYOUAREHAVINGLOTSOFFUNINTRYINGTOCATCHME"
        "THATWASNTMEONTHETVSHOWWHICHBRINGSUPAPOINTABOUTME"
        "IAMNOTAFRAIDOFTHEGASCHAMBERBECAUSEITWILLSENDMETO"
        "PARADISEALLTHESOONERBECAUSEINOWHAVEENOUGHSLAVES"
```

```
        "TOWORKFORMEWHEREASEEVERYONEELSEHASNOTHINGWHENTHEY"
        "REACHPARADISESOTHEYAREAFRAIDOFDEATHIAMNOTAFRAID"
        "BECAUSEIKNOWTHATMYNEWLIFEWILLBEANEASYONEIN"
        "PARADISEDEATH"
    )

    ciphertext, _ = encrypt_homophonic(plaintext)
    log_probs, floor_penalty = train_language_model_4grams()

    # 1. Definiamo i frammenti di testo che l'algoritmo aveva gia' decifrato correttamente
    good_cribs = [
        "IHOPE", "LOTSOFFUN", "BRINGSUP", "ABOUT", "BECAUSE",
        "ALLTHESOONERBECAUSE", "ENOUGHS", "EVERYONEELSEHASNOTHING",
        "REACH", "LIFE", "ONEIN"
    ]

    crib_dict = {}
    for crib in good_cribs:
        crib_start = plaintext.find(crib)
        if crib_start != -1:
            for i in range(len(crib)):
                ct_sym = ciphertext[crib_start + i]
                pt_char = crib[i]
                crib_dict[ct_sym] = pt_char

    RESTARTS = 3 # 3 restart sufficienti con la riduzione dello spazio delle chiavi
    T_INIZIALE = 15.0
    T_FINALE = 0.5
    LAMBDA_COOLING = 0.90
    ITER_EPOCA = 15000

    best_score_with_crib = float('-inf')
    best_text_with_crib = ""

    for run in range(RESTARTS):
        text, score = decrypt_mcmc_sa_crib(
            ciphertext, log_probs, floor_penalty, crib_dict=crib_dict,
            T_start=T_INIZIALE, T_fin=T_FINALE, cooling_rate=LAMBDA_COOLING, iter_per_epoch=ITER_EPOCA
        )
        if score > best_score_with_crib:
            best_score_with_crib = score
            best_text_with_crib = text

    final_acc = (sum(1 for a, b in zip(plaintext, best_text_with_crib) if a == b) / len(plaintext)) * 100
    print(f"\nACCURATEZZA FINALE (CON CRIB A 4-GRAMMI): {final_acc:.2f}% delle lettere corrette")
    print(f"TESTO DECIFRATO:\n{best_text_with_crib}")
```

Bibliografia

[1] Jiankang Chen and Jeffrey S. Rosenthal. Decrypting classical cipher text using markov chain monte carlo. *Statistics and Computing*, 22(2):397–413, 2012.

[2] Jarl Van Eycke David Oranchak, Sam Blake. The solution of the zodiac killer's 340-character cipher. March 27, 2024.

[3] Olle Häggström. *Finite Markov Chains and Algorithmic Applications*. Cambridge University Press, 2002.

[4] Martin J.H. Jurafsky D. *Speech and language processing*. Cambridge University Press, 2026.

[5] David A. Levin and Yuval Peres. *Markov Chains and Mixing Times*. American Mathematical Society, 2nd edition, 2017.

[6] Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer-Verlag, New York, 2 edition, 2004.

[7] Claude E. Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4):656–715, 1949.

[8] Claude E. Shannon. Prediction and entropy of printed english. *Bell System Technical Journal*, 30(1):50–64, 1951.

[9] Douglas R. Stinson and Maura B. Paterson. *Cryptography: Theory and Practice*. CRC press, 4 edition, 2018.